嵌入式系统概论 — 课件整理与期末复习笔记

> **来源**：2025 春《嵌入式系统概论》全部课件（00~12 讲）＋ 期末复习文件（90-期末重点） **用途**：考前系统复习。按课件顺序从头到尾整理全部知识点，并结合复习文件标注重点。
>
> 标注图例
>
> - ⭐【重点】：复习文件标注为 “+” 的知识点 / 章节 → 笔试重点关注，务必掌握
> - 📌【了解】：复习文件标注为 “-” 的知识点 / 章节 → 不在本次笔试考察范围，通读了解即可
> - 无标记：正常掌握（笔试可能覆盖的基础内容）
> - `(Pxx)`：知识点对应的原课件页码

目录

- 第0~1章 课程介绍与嵌入式系统概述（00 课程介绍 / 01 嵌入式系统概述）
- 02 嵌入式微处理器 📌【了解】
- 03 存储器架构（部分条目 📌【了解】）
- 04 基于总线的计算机系统 📌【了解】
- 05 嵌入式软件系统
- 06 嵌入式操作系统 ⭐【重点】
- 07 同步、通信与存储管理
- 08 建模
- 09 FSM ⭐【重点】
- 10 嵌入式系统设计方法 📌【了解】
- 11 物联网 ⭐【重点】
- 12 BSP 与 Bootloader
- 附录A 期末重点速查表
- 附录B 复习文件原文摘录
- 附录C 考试形式与成绩构成
- 附录D 实验部分简介（不在笔试范围）
- 附录E 参考书目

第0~1章 课程介绍与嵌入式系统概述

00 课程介绍

课程定位（P2-P4）

- 嵌入式系统不是"流行与否"的问题，而是"不可或缺"：不像手机、AI 大模型那样是"显性热点"，却是所有智能硬件的"隐形骨架"（P2）
- 应用场景从"看不见"到"离不开"，覆盖全领域（P2）
- 覆盖领域（P3）：通信、移动互联网、互联网＋、M2M、工业互联网；传感器网络、物联网（IoT，Internet of Things）、信息物理系统（CPS，Cyber-Physical Systems）；云计算、大数据；人工智能、机器学习、深度学习；无人驾驶（无人车、无人机）、智能机器人（尤其是人形机器人）；智能硬件、AR、VR；工业 4.0、中国制造 2025 等
- Alan Kay 名言（P4）：“预见未来的最好方式就是亲手创造未来”“对待软件严肃认真的人，应该制造自己专属的硬件”

预备知识（P5）

- 计算机程序设计（包括软件工程基础）、操作系统基础、计算机网络基础、计算机组成原理、算法
- 编程语言 C/C++/Python/Java/Rust：任一即可

目标与定位（P6）

- 目标：嵌入式软件开发、产品增值、系统设计
- 软件系统越来越复杂：95%+ 的软件系统实际上是嵌入的
- 代码量实例：1960 年代战机约 5 万行，F22 猛禽 250 万行，新型汽车超过一亿行代码

课程内容（P7）

- 原理：定义、基础知识、支撑技术等
- 掌握嵌入式系统的软件设计
- 掌握嵌入式系统的一般设计过程，重点在于嵌入式软件的开发

组织方式（P8）

- 课程讲授 + 课程实践
- 单周上课，地点：费楼 711、701 室；需自带笔记本

考核方式（P9）

- 笔试、实践、课上提问与点名

在研平台（**P10-P13**）

- DIY 平台示例：OSVehicle（P10）
- 所需技能：Linux、C/C++、Python（P11）
- 简单功能示例：车道保持（P12）；复杂功能示例：SLAM（P13）

展望未来（**P14**）

- IoT、机器人（尤其是人形机器人）、无人驾驶、元宇宙、人工智能热潮再度兴起

参考书目（**P15**）

- [德] Peter Marwedel 著，张凯龙译：《嵌入式系统设计：CPS 与物联网应用》（原书第 3 版），机械工业出版社，2020
- Robert Oshana、Mark Kraeling 著，单波等译：《嵌入式系统软件工程》，清华大学出版社，2016
- [美] Hassan Gomaa 著，郭文海、林金龙译：《实时嵌入式系统软件设计》，机械工业出版社，2018
- [美] K.C.Wang 著，徐坚、李佳蓓、吴文峰译：《嵌入式与实时操作系统》，机械工业出版社，2020
- Wayne Wolf 著，李仁发译：《嵌入式计算系统设计原理》（第 3 版），机械工业出版社
- [美] Edward Ashford Lee、Sanjit Arunkumar Seshia：《嵌入式系统导论：CPS 方法》（原书第 2 版），机械工业出版社，2018

01 嵌入式系统概述

1. 定义

计算机发展的三大阶段（**P2**）

1. 第一阶段（始于 50 年代）：大型机，由 IBM、Burroughs、Honeywell 等公司率先研制
2. 第二阶段（始于 70 年代）：个人计算机
3. 第三阶段：计算机正迈入"后 PC 时代"或"无处不在的计算机"阶段，充满机遇

术语演化（**P3**）

- 泛在计算（ubiquitous computing）：预言任何时间、任何地点的计算
- 不可见计算机（invisible computer）：预言计算机将被集成到物件中
- 普适计算（pervasive computing）：计算设备对日常生活的渗透
- 环境智能（ambient intelligence）：主要强调未来家庭或智能楼宇中的通信技术
- Mark Weiser（施乐公司 Palo Alto 研究中心主任）观点：从长远看 PC 机和计算机工作站将衰落，因为计算机变得无处不在——在墙里、在手腕上、在手写电脑中（像手写纸一样），随用随取、

伸手可及

嵌入式系统概念（**P4**）

- 随着计算机、网络、微电子技术快速发展，人们进入后 PC 时代；后 PC 时代是嵌入式系统（Embedded System）的网络时代，嵌入式技术将主宰后 PC 时代
- "嵌入式系统"实际上是"嵌入式计算机系统"的简称
- "嵌入式系统"有各种不同的定义

嵌入式系统定义（**P5**）

- 国内普遍认同的定义：嵌入式系统是"以应用为中心，以计算机技术为基础，软硬件可裁减，适用于应用系统对功能、可靠性、成本、体积、功耗有严格要求的专用计算机系统"
- 补充理解：嵌入式系统就是具有特定功能或用途、隐藏在某种设备中的计算机软硬件集合体，没有固定的特征形状

嵌入式系统定义（英文，**P6**）

- 通用定义（IEEE technav）：嵌入式系统是用于控制、监视或辅助设备、机器或工厂运行的装置；嵌入式系统在更大的系统中将硬件与嵌入式软件结合以实现特定功能
- 简单方便的定义：嵌入式系统是包含在产品中、而该产品不被描述为计算机的任何计算机系统

嵌入式系统定义（其他，**P7**）

- 嵌入封闭产品中的信息处理系统
- wiki 定义：为解决少数几个非常特定的问题而构建、且不易改变的计算机
- 简单表述：所有通用计算机（带显示器、键盘等）之外的计算系统都是嵌入式系统

嵌入式系统三要素（**P8**）

- 嵌入性：嵌入到对象体系中，可以是单个对象、系统或过程
- 专用性：软、硬件按对象要求设计、裁减
- 计算机：实现对象的智能化功能

CPS 信息物理系统（**P9**）

- Cyber-Physical System：计算进程与物理进程的集成
- 更多地强调嵌入式系统与物理学的深度结合：CPS = ES + Physics
- 人机物融合系统：深度融合传感器、嵌入式计算、云计算、网络通信、软件等各类信息技术
- 3C 能力：计算（Computer）、通信（Communication）、控制（Control）高度协同和自治，使生产应用系统自主、智能、动态、系统化地监视并改变物理世界的性状

通信的重要性（**P10**）

- 通信技术：联网、分布式应用、服务质量
- 嵌入式系统：机器人、控制系统、实时、可靠性
- 普适/泛在计算：随时随地的信息、互联、协同

无线传感器网络 **WSN**（**P11**）

- 定义：由许多在空间中分布的自动装置组成的一种无线通信计算机网络，这些装置使用传感器协作地监控不同位置的物理或环境状况（如温度、声音、振动、压力、运动或污染物）
- 起源：战场监测等军事应用；现广泛用于民用领域——环境与生态监测、健康监护、家居自动化、交通控制等

物联网 **IoT**（**P12-P13**）

- 定义：通过射频识别（RFID）、红外感应器、全球定位系统、激光扫描器等信息传感设备，按约定的协议把任何物品与互联网相连接，进行信息交换和通信，以实现智能化识别、定位、跟踪、监控和管理
- 万物相连的互联网：是互联网基础上的延伸和扩展，将各种信息传感设备与网络结合形成巨大网络，实现任何时间、任何地点人、机、物的互联互通
- 面向生产的物联网技术开发被称为"工业 4.0"
- 发展方向：万物互联 → 万物智联（P13，示意）

嵌入式系统的组成（**P14-P17**）

- 一般由嵌入式硬件和软件组成
- 硬件：以微处理器为核心，集成存储器和系统专用的输入/输出设备
- 软件：初始化代码及驱动、嵌入式操作系统和应用程序等，有机地结合在一起，形成系统特定的一体化软件
- 分层结构（P14，自下而上）：系统硬件 → 驱动 → 运行/实时内核 → 中间件 → 应用软件
- 典型构件示意（P15，图）
- 基本工作流程：感知 → 决策 → 控制（P16，图）
- TI 的提法：Sense-Understand-Act（感知—理解—行动）（P17，图）

通用计算机与嵌入式系统的区别（**P18**）

特征	嵌入式系统（典型）	通用计算机（PC/服务器类）
形式和类型	看不见的计算机；形式多样，按应用分类	看得见的计算机；按体系结构、运算速度和规模分为大、中、小型机和微机

特征	嵌入式系统（典型）	通用计算机（PC/服务器类）
组成	面向应用的嵌入式微处理器，总线和外部接口多集成在处理器内部，软件与硬件紧密集成	硬件模块化；通用处理器、标准总线和外设；软件和硬件相对独立
计算模型（MoC）	多个模型	主要是冯诺依曼模型
优化目标	多个（能耗、规模、实时……）	以平均性能为主导
是否为安全关键的	可能是	通常不是
实时相关	通常是	几乎不是
设计时应用是否已知	对于实时系统而言是	仅某些（如 Word 等应用）
二次开发性	一般不能再编程	应用程序可重新编制

- 结论：尽管差异显著，但随着技术发展，二者呈现"边界模糊"的融合趋势

2. 特征、分类、发展趋势

嵌入式系统的共性特征（**P22**）

1. 形式多样、面向特定应用（P23）
2. 高度制约的环境（P24）
3. 与外部环境的交互，包含传感器和执行器（P25）
4. 实时性的要求（P26）
5. 安全性和可靠性（P27）
6. 关注成本（P28）
7. 并发性（P29）

形式多样、面向特定应用（**P23**）

- 一般用于特定任务，硬件和软件都必须高效率地设计、可剪裁
- 采用多种类型的处理器和处理器体系结构
- 通常具有低功耗、体积小、集成度高等特点，能把通用微处理器中许多由板卡完成的任务集成在芯片内部

高度制约的环境（**P24**）

- 通常工作在资源高度受限的环境中，使系统设计极具挑战性
- 大多数嵌入式系统受处理器速度、存储器容量和用户接口的限制

- 很多运行于不可控的恶劣环境中，需克服高温、潮湿、振动、冲击甚至腐蚀
- 必须在满足计算环境和完成任务的前提下，优化尺寸、重量、可靠性、性能、成本和能耗等方面的设计

与外部环境的交互，包含传感器和执行器（**P25**）

- 与外部环境的交互在很大程度上是无人干预的
- 通过传感器接收来自外部环境的数据，通过执行器向外部环境输出数据并控制外部环境
- 传感器定义（国家标准 GB7665-87）：“能感受规定的被测量并按照一定的规律（数学函数法则）转换成可用信号的器件或装置，通常由敏感元件和转换元件组成”
- 执行器：提供实时计算机系统能够控制外部设备或机械装置的方法

实时性要求（**P26**）

- 一方面大多数实时系统都是嵌入式系统；另一方面嵌入式系统多数有实时性要求，具有时间约束条件，必须在给定时间范围内处理完事件
- 交互系统中系统响应延迟可能只造成不便，而实时系统的延迟可能是灾难性的
- 实时嵌入式系统通常包括实时控制：在无人干预的情况下，基于输入数据和当前状态做出控制决策

安全性和可靠性（**P27**）

- 某些实时嵌入式系统是安全至上且必须高度可靠
- 安全性：“远离事故和损失”，关注没有故障以及在单点故障情况下的安全
- 可靠性：系统或部件在给定时间和给定条件下实现要求功能的能力
- 一般要求具有出错处理和自动复位功能；极端环境下运行的嵌入式系统其可靠性设计尤其重要
- 软件中常包含硬件看门狗定时器、软件内存保护和重启动机制等

关注成本（**P28**）

- 系统成本是产品竞争的关键因素之一，尤其是量大的消费类数字化产品
- 成本构成：一次性开发成本 NRE（Non-Recurring Engineering，非经常性工程成本）+ 产品成本（硬件 BOM、外壳包装、软件版税等）
- 公式：批量产品总体成本 = NRE 成本 + 每个产品成本 × 产品总量；每个产品最后成本 = NRE 成本/产品总量 + 每个产品成本

并发性（**P29**）

- 并发性：在一个系统中将多个计算同时执行并潜在地交互的特性
- 实际问题域存在天然并发性（如物理环境多个任务同时发生），并发任务设计是实时嵌入式系统设计的有效解决方案

- 多处理器系统（如多核系统）为嵌入式系统的计算能力和性能提供显著提升

嵌入式系统的分类（**P30**）

- 五种分类方式：按嵌入式处理器的位数、按应用、按速度、按确定性、按软件复杂程度

按嵌入式处理器的位数分类（**P31**）

- 4 位、8 位、16 位、32 位、64 位嵌入式系统
- 32 位为主流；64 位已开始用于高度复杂、高速的嵌入式系统；4/8/16 位目前仍大量应用

按应用分类（**P32**）

- 讲义为示意图，按应用领域划分（如消费电子、工业控制、汽车电子等）

按速度分类（**P33**）

- 强实时系统：响应时间在毫秒或微秒级
- 一般实时系统：响应时间在几秒数量级，实时性要求比强实时系统差一些
- 弱实时系统：响应时间约为数十秒或更长，响应时间可能随系统负载轻重而变化

按确定性分类（**P34**）

- 硬实时系统：对响应时间有严格要求，若不满足会引起系统崩溃或致命错误
- 软实时系统：对响应时间有要求，但不满足不会导致致命的错误或崩溃

按软件复杂程度分类（**P35**）

- 循环轮询系统 → 有限状态机系统 → 前后台系统 → 单处理器多任务系统 → 多处理器多任务系统（由简单到复杂）

发展趋势（**P36**）

- 嵌入式人工智能
- 安全
- 互连的需求
- 不断增加的计算需求、不断增加的复杂性（如机顶盒多媒体处理、高清电视、深度学习等）
- 日益增长的灵活性需求：在不断变化的标准下进入市场的时间（Time-To-Market，TTM）
- 更高的集成度：在同一芯片上集成更多功能
- IP 重用、基于平台的设计、NoC 与总线（用于 ASIC 或 FPGA 中预先设计好的电路功能模块）
- 云、边缘计算

嵌入式人工智能（**P37**）

- 广义：嵌入式设备上实现的具有自主分析和决策的能力
- 狭义：嵌入式设备上实现的、等同于通用计算机上实现的人工智能
- “端智能”：嵌入式设备无处不在、SoC 芯片日益增长，最终嵌入式人工智能也将无处不在

嵌入式系统安全（**P38**）

- 数据存储不安全、敏感数据泄漏；服务端控制措施部署不当
- 传输过程中没有加密；身份认证措施不当
- 密钥保护措施不当；会话处理不当

互联的价值（**P39**）

- 提高对各种服务、内容和信息的访问能力
- 为动态修改嵌入式软件提供可能：修改系统代码或"固件"、增添新的应用软件模块
- 增强系统和设备的可管理性
- 带来新业务/功能、新的利润增长点

3. 应用

嵌入式系统无处不在（**P42**）

- 运输与移动性：汽车电子、航电设备、铁路、舰船、海洋技术与海事系统
- 工厂自动化、健康领域、智能楼宇、智能电网、公共安全、结构健康监测、灾难恢复
- 机器人、农业与畜牧业、军事应用、航天、消费电子等

运输与移动性（**P43-P49**）

- 现代汽车：多达 100 个微处理器运行复杂软件；BMW 745i Sedan 超过 70 个微处理器（P43）
- 一辆汽车超过 30% 的成本在电子设备上，90% 的创新基于电子系统（P43）
- 汽车电子应用领域（典型 CPS，P44）：防抱死制动系统（ABS）、电子稳定控制（ESP）、安全气囊、高效自动变速箱、防盗智能钥匙、盲角警报系统、高级驾驶辅助系统（ADAS）等
- 无人车/智能驾驶（P45，图）
- 自动驾驶分级（P46-P47）：依据美国高速公路安全管理局（NHTSA）和美国汽车工程师学会（SAE）标准；按 SAE 划分，自动驾驶分为五个等级（P47 为汽车驾驶自动化分级示意图）
- 飞行汽车（P48，图）
- Boeing 777（P49）：板载 1280 个 CPU、700 万行代码；航电设备包括飞行控制系统、防撞系统、供电系统、飞行员信息系统、襟翼控制系统、娱乐系统等

可穿戴与机器人（**P50-P59**）

- 可穿戴计算 Wearable Computing（P50，图）
- VR/AR（P51，图）
- BMI（脑机接口）（P52，图）
- 可穿戴机器人：微元外骨骼行走助力机器人（P53）
- 无人机（P54，图）
- 仿生机器人（P55，图）
- 服务机器人（P56，图）
- 人形机器人、具身智能（P57、P59，图；P58 为示意图）

智能家居与航天（**P60-P62**）

- 智能家居（P60，图；P61 为示意图）
- 航天领域（P62，图）

02 嵌入式微处理器（Embedded Microprocessor）📌【了解】

> 整章不在笔试考察范围，内容照常整理供了解。

硬件平台

- 嵌入式系统硬件（CPS，信息物理系统）包含：CPU、总线（bus）、存储器（memory）、I/O 设备（网络、传感器、执行器等）（P2-P3）
- 核心问题：需要多大/多快？（P3）

主流的微处理器

- ARM：消费电子、车载、边缘 AI（P4）
- RISC-V：高性能嵌入式领域的新增长点，用于定制化场景、新兴边缘设备（P4）
- X86：工业控制、边缘服务器（P4）
- 专用架构：实时控制（P4）
- MIPS：“无互锁流水级的微处理器”（Microprocessor without interlocked piped stages），已边缘化（P4）

冯诺依曼结构与哈佛结构

- 冯诺依曼结构（P5）：程序指令存储器与数据存储器合并的存储器结构；指令存储地址和数据存储地址指向同一存储器的不同物理位置，因此指令和数据的宽度相同；CPU 寄存器包括程序计数

器（PC）、指令寄存器（IR）、通用寄存器等

- 哈佛结构（P6）：使用两个独立的存储器模块分别存储指令和数据，每个存储模块都不允许指令和数据并存；使用独立的两条总线分别作为 CPU 与每个存储器之间的专用通信路径，两条总线之间毫无关联
- 改进的哈佛结构（P6）：仍使用两个独立存储器模块；具有一条独立地址总线和一条独立数据总线，用公用地址总线访问两个存储模块，公用数据总线完成程序/数据存储模块与 CPU 之间的数据传输；两条总线由程序存储器和数据存储器分时共用
- 大多数 DSP 使用哈佛架构处理流数据：更大的内存带宽、更可预测的带宽（P6）

RISC 与 CISC

- CISC（复杂指令集计算机，Complex Instruction Set Computer）：多种寻址模式（P7）
- RISC（精简指令集计算机，Reduced Instruction Set Computer）：load/store 结构；流水线指令（P7）

RISC 与 CISC 主要特征对比（P8）：

比较内容	CISC	RISC
指令系统	复杂、庞大	简单、精简
指令数目	一般 > 200	一般 < 100
指令格式	一般 > 4	一般 < 4
寻址方式	一般 > 4	一般 < 4
指令字长	不固定	等长
访存指令	不加限制	只有 LOAD/STORE 指令
指令使用频率	相差很大	相差不大
指令执行时间	相差很大	绝大多数在一个周期内完成
优化编译实现	很难	较容易

微处理器架构类型（P9）：

- von Neumann + CISC：X86、Motorola 68000
- von Neumann + RISC：ARM7、Cortex-M0、SPARC、MIPS、PowerPC
- Harvard + CISC：SHARC（DSP）
- Harvard + RISC：ARM9、PIC、Cortex-M3/M4

RISC-V

- 特点：开源开放、简洁、模块化、可定制、可扩展，降低芯片设计的周期和成本（P10）
- 将基准指令和扩展指令分开，可通过扩展指令做定制化的模块和扩展（P10）
- 基准指令确定后不会再变化，这是 RISC-V 稳定性的重要保障（P10）
- 支持 32、64、128 位指令集（P10）
- 相较 x86、ARM，RISC-V 在生态系统、性能和优化、硬件设计和验证挑战等方面仍有挑战，但高速发展（P10）
- 兼容性：2015 年非盈利性组织 RISC-V 基金会（RISC-V Foundation）成立（P11）
- 设计系统芯片之前需要考虑的五件事（P12）：成本、生态系统、碎片化风险、安全性、设计保证、社区支持

嵌入式微处理器的分类

- 按位数：4 位、8 位、16 位、32 位、64 位（P13）
- 按功能分四种：嵌入式微处理单元（MPU）、嵌入式微控制器（MCU）、嵌入式 DSP 处理器、嵌入式 SoC（P13）

嵌入式微处理单元（MPU）（P14）：

- 与通用计算机处理器对应的 CPU，功能和微处理器基本一样；32 位以上，性能较高
- 体积小、功耗少、成本低、可靠性高，有的可提供工业级应用
- 流行产品：ARM（Cortex-A15/A77/A78/X1/X4/X925）、MIPS
- 图（P15）：基于 MPU 的时间和温度系统——以 MPU 为核心的测控系统结构示意

嵌入式微控制器（MCU）（P16）：

- 将整个计算机系统的主要硬件集成到一块芯片：内部集成 ROM/EPROM、RAM、总线、总线逻辑、定时/计数器、Watchdog、I/O、串行口等必要功能和外设
- 特点：一个系列具有多种衍生产品；单片化，体积大大减小，功耗和成本降低、可靠性提高；是目前嵌入式工业的主流，约占嵌入式系统 50% 的份额；多为 8、16、32 位处理器；追求 time-to-market 的完整方案
- 流行产品：Cortex-M0/M35P/M55/M85、RISC-V、8051、Coldfire 的 MC683xx（32 位）
- 图（P17）：基于 MCU 的时间和温度系统——单芯片集成式的测控系统结构示意
- 图（P18）：NXP i.MX RT 系列跨界处理器——兼具 MCU 易用性与 MPU 高性能的跨界产品示例

嵌入式 DSP（P19）：

- 专门用于信号处理的处理器，在系统结构和指令算法方面进行特殊设计，具有很高的编译效率和指令执行速度
- 应用领域：数字滤波、频谱分析、FFT

嵌入式 SoC（片上系统）（P20）：

- 追求产品系统最大包容的集成器件，绝大多数系统构件都在一个系统芯片内部
- 以 IP 模块为基础的设计技术，IP 是 SoC 应用的基础；IP 核（Intellectual Property Core，知识产权核）指已验证、可重复利用、具有某种确定功能的芯片设计模块
- 特点：设计生产效率高；降低功耗（集中管理电力供应，实现更精细的电源管理）；降低成本（减少外部元件和接口的需求）；小型化（集成度高，适合移动设备、物联网设备等对尺寸要求高的场景）
- 流行 SoC：高通骁龙（Snapdragon）、海思
- 图（P22）：华为 Ascend310 AI 处理器逻辑架构——SoC 内部集成 AI 计算单元、CPU、存储等模块的架构示例

FPGA（现场可编程门阵列）（P23）：

- Field-Programmable Gate Array，具有可编程特性的集成电路，是专用集成电路（ASIC）领域中的一种半定制电路
- 相比 ASIC 开发周期更短、开发难度更低，但存在成本过高、性能较差、资源利用率远不及 ASIC 等问题，不能真正替代 ASIC

xPU 的兴起（P24）：

- GPU：大规模并行计算
- NPU（Neural Processing Units，神经网络处理器）：专门为人工智能（AI）计算设计的处理器，高效执行神经网络运算（矩阵乘法、卷积、激活函数等）；相较传统 CPU/GPU 在能效比和计算速度上更具优势，尤其适合移动设备、边缘计算和嵌入式 AI 场景；如华为昇腾 910 系列
- TPU（Tensor Processing Unit，张量处理器）：谷歌 2016 年推出，主要用于低精度运算；专为机器学习设计的 ASIC，通过硬件与算法的深度协同优化，在性能、能效比和扩展性上实现突破
- DPU（Deep-Learning Processing Unit，深度学习处理器/数据处理单元）：通过硬件加速和任务卸载，释放 CPU 算力以支持核心业务应用；主要作用：AI 推理加速（如智能摄像头中实现实时目标检测）、科学计算（通过 RDMA 技术优化跨节点数据传输，提升分布式计算效率）
- BPU（Brain Processing Unit，大脑处理器）：地平线公司主导的嵌入式处理器架构

嵌入式微处理器的特点

- 基础是通用微处理器（P25）
- 与通用微处理器的区别：体积小、重量轻；功耗低；成本低（片上存储、引脚与封装、代码密度）；在工作温度、抗电磁干扰、可靠性等方面增强（P25）

ARM 公司

- 成立于 1990 年，前身是英国剑桥的 Acorn Computers（P28）

- 设计 ARM 架构的 RISC 处理器内核，同时开发配套技术：Mali GPU、符合 AMBA 协议的总线设计、常见外设 IP 及配套软件等一整套 SoC 解决方案（P28）
- 商业模式为 IP 授权：通过知识产权授权收取一次性技术授权费用和版税提成；ARM 只专注设计 CPU/GPU 等 IP，代工或生产由被授权客户自行解决；收入包括前期授权费（license）和版税（Royalty，按使用 ARM 芯片的出货量按比例抽成）（P29）
- 授权方式（P30）：
  - 处理器授权：最低授权层级，可使用 ARM 设计好的处理器，不能改变原有设计，但可根据需要调整频率、功耗等
  - POP（Processor Optimization Pack，处理器优化包）授权：处理器授权的高级形式，ARM 出售优化后的处理器，方便厂商在特定工艺下设计、生产出性能有保证的处理器
  - 架构/指令集层级授权：可对 ARM 架构或指令集进行改造以自行设计处理器，如高通的 Krait 架构和苹果的 Swift 架构
- 全球合作伙伴众多（P31）
- 主要应用领域（P32）：以低功耗为特点，在嵌入式和移动处理器领域具有绝对优势；包括消费类电子产品、无线通讯领域、网络应用、工业控制领域
- ARM 处理器的架构不断发展演进（P33）

ARM 处理器分类——应用特征（P34）：

- Cortex-A 系列：应用处理器（Application Processors），主要面向移动计算、智能手机；支持基于内存管理单元（MMU）的虚拟内存系统体系结构（VMSA）
- Cortex-R 系列：实时处理器（Real-time Processors），面向实时应用的高性能系列，如硬盘控制器、汽车传动系统和 Modem 基带；支持基于内存保护单元（MPU）的受保护内存系统体系结构（PMSA）
- Cortex-M 系列：微控制器处理器（Microcontroller Processors），通常设计成面积很小、能效比很高；流水线很短，最高时钟频率相对较低；主要面向 IoT 物联网领域
- SecureCore 系列：基于 ARM-M 架构，增强嵌入式安全；主要有 SC000 和 SC300，分别基于 Cortex-M0（v6-M）和 Cortex-M3（v7-M）
- Neoverse 系列：服务器端，面向云端到边缘的 HPC、AI/ML 加速等领域；是 Cortex 之外面向服务器和基础设施设备的核心 IP

ARM 架构

- 典型的 RISC 结构（P35）：丰富的寄存器；加载/存储体系结构；简单寻址模式；统一和固定长度的指令字段
- 增强功能（P36）：每条指令控制 ALU 和移位器；自动递增和自动递减寻址模式；多个加载/存储；条件执行
- 结果（P37）：高性能、低代码大小、低功耗、低硅面积

流水线与分支预测

- 流水线用于提高速度（P38）：3 级（ARM7TDMI 及更早版本）；5 级（ARM9、ARM9TDMI）；13 级流水线（ARMv7 架构，如 Cortex-A8）；15 级流水线（ARMv8-A 架构，Cortex-A73）；深流水线 + 超标量（三发射），配合更复杂的分支预测和乱序执行引擎
- 流水线的挑战（P39）：
  - 分支指令的影响：可能产生流水线清空，导致执行速度变慢
  - 分支预测技术：静态预测；动态预测（基于历史执行记录猜测跳转方向）
  - 数据相关性、数据冒险：ARM 通过寄存器重命名（将依赖的寄存器临时映射到空闲寄存器）和乱序执行（调整指令执行顺序，优先执行无依赖的指令）缓解这一问题

设备分类

- 根据其能力和性能，嵌入式/物联网设备分为三类（P40）：
  - 高端设备：如车载设备
  - 中端设备：包括 Raspberry Pi 等单板机（SBC）和智能手机
  - 低端设备：资源有限，无法运行传统操作系统；流行例子包括 Arduino、Econotag、Zolertia Z1、IoT-lab M3 节点、Open-Mote 节点和 TelosB motes
- 低端设备分类——因特网工程任务组（IETF）根据内存容量将其标准化为三个子类（P41）：
  - 0 级设备：拥有最小资源（<< 10KB 的 RAM 和 << 100KB 的 Flash），如无线传感器网络（WSN）中的一种专用微粒
  - 1 级设备：中等资源（RAM 约 10KB、Flash 约 100KB），可运行比一般设备更丰富的应用程序和更高级的功能，如路由、安全通信协议等
  - 2 级设备：拥有更多资源，但与中高端物联网设备和传统互联网主机相比仍受到很大限制

选择微处理器的准则

- 高效、经济地满足任务的计算需求（P42）
- 考虑因素：速度、ROM 和 RAM 的数量、I/O 端口和定时器的数量、尺寸、封装、功耗（P42）
- 易于升级；单位成本（P42）
- 软件开发工具的可用性：汇编器、调试器、C 编译器、仿真器、模拟器、技术支持（P42）
- 广泛的可用性和可靠的微控制器来源（P42）

选择微处理器的十个步骤

- 步骤 1：列出所需的硬件接口（P43）
- 步骤 2：检查软件架构（P44）
- 步骤 3：选择架构（P44）

- 步骤 4：确定存储器需求（P44）
- 步骤 5：开始搜索微控制器（P44）
- 步骤 6：检查成本和功率限制（P44）
- 步骤 7：检查部件供货情况（P44）
- 步骤 8：选择开发套件（P44）
- 步骤 9：调查编译器和工具（P44）
- 步骤 10：开始实验（P44）

03 存储器架构

存储系统（复杂性）

- 对许多应用来说，存储系统对整个系统性能的影响更大（P2）
- 复杂性体现在（P2）：在同一个嵌入式系统中，通常需要组合各种存储技术；至少需要一些非易失性存储和一些易失性存储；存在多种选择；层次结构往往是必须的；处理器架构的地址空间被分解成若干子空间来提供不同类型的存储器访问

易失性存储器 📌【了解】

- 断电时内容消失的存储器（P3）
- 随机存取存储器（RAM）包括：
  - SRAM（静态 RAM）：速度快，面积大（P3）
  - DRAM（动态 RAM）：保持数据的时间很短，需要定期刷新；比 SRAM 更不稳定（P3）

在嵌入式系统中的应用 📌【了解】

- 大多数嵌入式系统都包括一个 SRAM，许多也会包括 DRAM，因为只利用 SRAM 不能提供足够大的存储容量（P4）
- 影响程序执行时间的因素（P4）：被访问的存储器地址是映射到 SRAM 还是 DRAM；DRAM 可能在忙于刷新时被请求访问，因此 DRAM 刷新周期会引起访问时间的变化；访问历史也可以影响存取时间

非易失性存储器 📌【了解】

- 不需要持续供电来保留存储在计算设备中的数据或程序代码（P5）
- 类型（P5）：只读存储器（ROM），或掩模 ROM（Mask ROM，内容在芯片工厂就已经固定）；电可擦除可编程 ROM（Electrically Erasable Programmable ROM，EEPROM）；快闪存储器（Flash）；磁盘存储器

固件、可固化（ROMable）📌【了解】

- 固件（firmware）（P6）：一般存储于设备中的电可擦除只读存储器 EEPROM 或 FLASH 芯片中的程序，一般可由用户通过特定的刷新程序进行升级
- 可固化（ROMable）（P6）：可被编程到 ROM 芯片中；代码将从 ROM 正确执行；不需要复制到 RAM，但 RAM 可能更快；代码和数据不能混用（除了常量数据）；作为"只读"芯片不能更新，可固化程序必须使用 RAM 或磁盘来保存变化的数据

Flash 📌【了解】

- 特性（P7）：非易失性；无机械结构；高速存取；有比较快的读取时间，但比 SRAM 和 DRAM 慢；频繁访问的数据在程序使用之前从 flash 转移到 RAM 中；写入时间大大超过读取时间，而且写入的次数是有限的
- 类型：NOR Flash（或非门闪存）、NAND Flash（与非门闪存）（P8）

NOR Flash（P9）：

- 支持随机地址访问，读取速度快
- 可直接执行存储在其中的代码（芯片内执行/就地执行"XIP，eXecute In Place"特性），无需先加载到 RAM
- 擦写效率低：按块（Block）擦除（通常每块 64KB-512KB）；写入操作需按"字节（Byte）"或"字（Word）"进行；擦除和写入较慢
- 成本高、容量小：存储密度低于 NAND Flash，单位容量成本较高，常见容量为 1MB-512MB
- 典型应用：嵌入式系统的启动程序（Bootloader，如路由器、机顶盒、汽车 ECU 的启动代码，需快速随机读取并直接执行）；固件（Firmware，如 BIOS/UEFI、打印机固件，容量需求小但对可靠性和读取速度要求高）

NAND Flash（P10）：

- 不支持随机地址访问
- 存储密度高、成本低
- 以页（Page）为单位读写、块（Block）为单位擦除，且"先擦后写"
- 典型应用：消费电子（U 盘、SD 卡、固态硬盘 SSD、手机/平板的内置存储）；企业级存储（数据中心的 SSD 阵列、云存储节点）；工业设备（监控摄像头的录像存储、物联网设备的本地数据缓存）

NOR 与 NAND Flash 的对比（P11，部分单元格原文提取缺失，按讲义上下文补全）：

对比项	NAND	NOR
主要应用	文件存储	代码执行
存储容量	高	低

对比项	NAND	NOR
每比特成本	低	高
动态功耗	低	高
待机功耗	低	高
写入速度	快	慢
读取速度	慢	快
eXecute In Place（XIP）	不支持	支持
可靠性	高	中

存储器系统的层次结构

- 精确的分层结构取决于技术参数，也取决于应用领域（P12）
- 层次结构（从处理器向外）（P12）：处理器寄存器 → 缓冲存储器（包括高速缓存 Cache、地址转换高速缓存 TLB 也称快表、暂存存储器 SPM）→ 工作存储器（或主存储器、主存，实现处理器存储地址所涵盖的存储器，容量通常在 MB 到 GB 之间，并且是易失的）→ flash、磁盘等非易失性存储，也可基于互联网的存储器解决方案（如云）

cache（高速缓存）（P13）：

- 高速缓存的架构对应用程序的执行时间具有很大影响
- 高速缓存潜在地提高了存储系统的能效
- 在设计时预测缓存是否命中是很困难的，这对实时性能的精确预测而言也是一个负担
- 图（P13）：CPU 与 cache、cache controller、main memory 之间的 data/address 连接示意

多级 cache（P14）：

- 结构：CPU → L1 cache → L2 cache → 主存
- 参数：h1 = cache 命中率（hit rate），h2 = L2 命中率
- 平均访存时间公式：tav = h1·tL1 + h2·tL2 + (1 - h1 - h2)·tmain

暂存/便签存储器（Scratchpad Memory，SPM）（P15）：

- 通常用于嵌入式系统和实时处理器中，以提供比缓存更可预测的访问时间
- SPM 和主存统一编址，每当某个简单的地址解码器给出一个 SPM 地址范围内的地址时，SPM 就被访问
- 应用软件自主管理的片上存储器技术；功耗低、速度快

SPM 与 Cache 比较（P16）：

对比项	**SPM**	**Cache**
功耗	Low	High
芯片面积	Small	Big
访问时间	Fixed	Unpredicted
地址空间	Separate with Main memory	Same with Main memory
管理	software	Hardware

存储器访问时间难以预测

- 很难预测或理解访问存储器的时间需要多久，因而嵌入式系统设计人员通常比一般程序员需要更深入地理解存储器系统（P17）
- 虚拟存储器（Virtual Memory）：使各种存储技术看起来是一个连续的地址空间（P17）
- 地址转换：把地址空间的逻辑地址转换成一种存储技术上的物理地址，转换通常由专门硬件协助完成，称为转换后备缓冲器（Translation Lookaside Buffer，TLB），用于加速地址转换（P17）

存储墙与功耗墙 📌【了解】

- 传统冯·诺依曼架构是"存算分离"架构：数据需在存储和计算单元之间频繁搬运，造成高延迟、高能耗（P18）
- 存储墙（Memory Wall）：存储速度无法跟上计算速度（P18）
- 功耗墙（Power Wall）：数据在计算单元和存储单元之间频繁的移动也会带来不小的功耗（P18）
- 存算一体（P19-P20）：将计算融入存储、减少数据搬移的应对思路；图（P19-P20）：连续两页展示存算一体的架构/原理示意图

4. 基于总线的计算机系统 📌【了解】

> 本章（含总线及分类、常用总线、IO 设备、可编程 I/O、中断、可重入、竞态条件等）在复习文件中标记为 “-”，不在笔试范围，以下内容供完整了解。

4.1 总线（Bus）（P1-P2）

- 总线是计算机各种功能部件之间传送信息的**公共通信干线**（P2）。
- 总线 = **一组传送线路 + 相关的通信协议**（P2）。
- 按功能可分为 CPU 总线等不同类别（P1-P2）。

4.2 总线协议（Bus Protocol）（P3）

- 总线协议**决定设备如何通信**。
- 总线上的设备经历一系列状态；协议由**状态机（FSM）** 指定，**协议中的每个参与者都有一个状态机**。
- 可包含**异步逻辑行为**。

4.3 总线复用（P4）

- 图：CPU 通过地址线（adrs）与数据线（data）连接设备，并配以 data enable、adrs enable 等使能信号——地址/数据信号分时复用，由使能信号控制有效时段。

4.4 系统总线配置（P5）

- **多总线允许并行**：一条总线连接慢速设备，另一条独立总线连接高速设备（如 memory、high-speed device）。
- 两条总线之间通过**桥（bridge）** 连接（如 CPU ↔ 高速设备/内存总线 ↔ 慢速设备总线）。
- 图示：CPU 经桥分别挂接高速内存/设备与多个慢速设备。

4.5 AMBA 总线（ARM）（P6-P7）

AMBA（Advanced Microcontroller Bus Architecture，高级微控制器总线架构），包含：

总线	全称	特点
AHB	Advanced High-performance Bus（高级高性能总线）	针对高效率、高频宽及快速系统模块设计；可连接微处理器、片内/片外内存模块、DMA 等高效率模块（P6）
APB	Advanced Peripheral Bus（高级外围总线）	速度更慢、成本更低，用于低速且低功率的外围；可针对外围设备优化功耗与复杂接口；在 AHB 与低带宽外围设备之间提供通信桥梁（P6）
AXI	Advanced eXtensible Interface（高级可拓展接口）	AMBA 3.0 新增，用于 ARM 与 FPGA 的高速数据交互；高速度、高带宽、管道化互联、单向通道、只需首地址、读写并行、支持乱序、支持非对齐操作、有效支持初始延迟较高的外设；缺点是连线非常多（P7）
CHI	Coherent Hub Interface	用于多核处理器之间的缓存一致性通信（如 big.LITTLE 大小核协同），支持分布式共享内存（P7）
ACE	AXI Coherency Extensions	AXI 的缓存一致性扩展，确保 CPU、GPU、加速器等访问共享内存时的数据一致性（P7）

4.6 常用总线对比（P8）

常用总线：UART、CAN、USB、SPI、I2C。

总线	优点	缺点
UART	认知度高、简单、低成本、大型系列、经济有效	功能有限、点对点、有限速率
CAN	安全、快速、简单、低成本	源自汽车、无即插即用软件需指定驱动、无即插即用硬件、没有固定标准
USB	快速、即插即用硬件、广泛接受、即插即用、大型系列	复杂、固件昂贵、需要强大主机
SPI	快速、简单	无即插即用（软件/硬件）、没有固定标准
I2C	认知度高、广泛接受、经济有效	有限速率、无即插即用

4.7 示例：Arduino Uno R3（P9-P10）

- 典型基于总线的微控制器开发板示例，其上通过各类总线（I2C、SPI、UART 等）与传感器、外设交互。

5. I/O 接口与设备（P11-P12）

5.1 I/O 设备的分类（P12）

- **输入设备**：将信息从外部输入到内部系统，如触摸面板、条形码扫描仪等。
- **输出设备**：将嵌入式系统处理后的信息输出到外部世界，如液晶显示器、扬声器等。

5.2 I/O 设备的组成（P13）

- I/O 设备通常由两部分组成：
  - **机械部件**（机电部分）；
  - **电子部件**——称为**设备控制器（device controller）或适配器（adapter）**。

5.3 应用实例：8251 UART（P14）

- UART（Universal Asynchronous Receiver Transmitter，通用异步收发器）是**异步串行通信口的总称**。
- 包括 RS232、RS449、RS423、RS422、RS485 等接口标准规范和总线标准规范。
- 规范内容：通信口的**电气特性、传输速率、连接特性和接口的机械特性**等。

5.4 I/O 接口寄存器（P15）

- I/O 接口充当设备与计算机的桥梁，需要多个寄存器：
  - **数据寄存器（Data Register）**：暂存传输的数据；
  - **控制寄存器（Control Register）**：接收 CPU 的控制命令（如 pinMode）；
  - **状态寄存器（Status Register）**：反映设备状态。
- 图示：CPU ↔（数据/控制/状态寄存器）↔ 设备。

6. 可编程 I/O（Programmed I/O）（P16-P17）

- 在通信过程中选择控制寄存器或数据缓冲区的三种方法：
  1. **独立 I/O 端口（Isolated I/O）**；
  2. **内存映射 I/O（Memory-mapped I/O）**；
  3. **混合解决方案（Hybrid）**：混合模型 = 内存映射的 I/O 数据缓冲区 + 用于控制寄存器的独立 I/O 端口。
- 架构差异：Intel x86 提供 IN、OUT 指令（独立 I/O）；ARM、MIPS、RISC-V 等 RISC 架构使用内存映射 I/O（P16）。
- 图：(a) 独立 I/O 和内存空间；(b) 内存映射 I/O；© 混合方案（P17）。

6.1 独立 I/O 端口（P18）

- 优点：
  - I/O 独立编址，**不占用内存空间**；
  - 使用专门的 I/O 指令，程序清晰，容易看出是 I/O 操作还是存储器操作；
  - 译码电路比较简单（I/O 端口地址空间一般较小，所用地址线较少）。
- 缺点：
  - 只能用专门的 I/O 指令，访问端口的方法少。

6.2 内存映射 I/O（P19）

- 优点：
  - 设备控制寄存器只是内存中的变量，可像其他变量一样在 C 语言中寻址；
  - I/O 设备驱动程序可以**完全用 C 语言编写**；
  - 编址空间大。
- 缺点：

- **缓存设备控制寄存器会导致灾难**——必须为硬件提供选择性禁用缓存的功能，增加软硬件复杂性；
- 可能浪费部分内存地址；
- 安全性问题。

6.3 ARM 内存映射 I/O 示例（P20）

- 定义设备地址：`DEV1 EQU 0x1000`；
- 读：`LDR r0,[r1]`；写：`STR r0,[r1]`——设备寄存器按内存地址直接存取。

6.4 高级语言接口：peek / poke（P21）

- 传统高级语言访问 I/O 的通用接口：
  - `int peek(char *location)`——读指定地址（`return *location;`）；
  - `void poke(char *location, char newval)`——写指定地址（`(*location) = newval;`）。

6.5 忙等 I/O（Busy/Wait I/O）（P22-P23）

- **最基本的方式**：使用指令测试设备忙闲，忙则循环等待。
- 输出示例：`while (peek(OUT_STATUS) != 0);`——等待输出设备空闲后再写入下一字符（P22）。
- 输入→输出复制示例：`while (peek(IN_STATUS) == 0);` 等输入就绪 → 读 IN_DATA → 写 OUT_DATA → `poke(OUT_STATUS,1)` 通知输出 → 等待输出完成（P23）。
- **缺点**：CPU 在测试设备时不能做其他工作，很难同时进行 I/O，效率非常低。

7. 中断 I/O（Interrupt-driven I/O）（P24-P31）

7.1 动机与原理（P24-P25）

- 忙/等 I/O 非常低效；中断允许**设备改变 CPU 中的控制流**，调用子例程来处理设备（P24）。
- 中断行为**基于子程序调用机制**：中断强制下一条指令是预定义地址的子程序调用（P25）。

7.2 中断物理接口（P26）

- CPU 与设备之间通过 CPU 总线连接，CPU 与设备握手：
  1. 设备发出**中断请求（Interrupt Request）**；
  2. CPU 在能够处理中断时**确认中断（Acknowledge）**。

7.3 示例：字符 I/O 中断处理（P27-P28）

- 输入处理程序 `input_handler()`：读 IN_DATA → 置 `gotchar = TRUE` → `poke(IN_STATUS,0)` 清状态（P27）。
- 中断驱动主程序：主循环轮询 `gotchar`，为真则把字符写到 OUT_DATA、置 OUT_STATUS、清 `gotchar`，其余时间做其他处理（P28）。

7.4 示例：带缓冲区的中断 I/O（P29-P31）

- 使用**字符缓冲队列**（环形/指针队列）：**留一个空槽以便检测缓冲区满**（head/tail 指针）（P29）。
- 基于缓冲区的输入处理程序：缓冲区满则置 error；否则读 IN_DATA 入队；若 `nchars == 1` 则立即取出一个字符送输出（P30）。
- Arduino 示例：`while (Wire.available()) { char c = Wire.read(); Serial.print(c); }`——I2C 数据可用性轮询读取（P31）。

8. 中断优先级与中断向量（P32-P37）

- 两种使中断更灵活的机制（**正交**，大多数 CPU 都同时提供）：
  - **优先级（Priority）**：决定哪个中断首先得到 CPU；
  - **向量（Vector）**：决定每种类型的中断调用什么代码。

8.1 优先级中断（P33-P34）

- 图示：多个设备（device 1…n）经优先级线（L1…Ln）向 CPU 提出中断请求，CPU 经 interrupt acknowledge 响应（P33）。
- **屏蔽（Masking）**：在挂起的中断完成之前，不识别优先级低于当前优先级的中断；通过当前程序状态寄存器 **CPSR 中的中断屏蔽位**实现（P34）。
- **不可屏蔽中断（NMI，Non-Maskable Interrupt）**：最高优先级，从不被屏蔽，通常用于断电等紧急情况（P34）。
- 应用示例：
  - 实时控制系统：工业机器人中**急停信号中断**优先级需高于普通传感器采样中断，确保危险时立即响应；
  - 嵌入式设备：智能手表中**按键中断**优先级高于环境光传感器中断，保证用户操作实时响应。

8.2 中断向量（P35-P37）

- **中断向量**：用于快速定位中断服务程序（ISR）的关键机制；允许不同设备由不同代码处理，提高灵活性（P36）。

- 向量号作为**中断向量表（Interrupt Vector Table）**的索引；向量表存储在内存中（handler 0/1/2/3…）。
- ARM 架构：中断向量表通常位于**内存起始地址 0x00000000**，或通过配置寄存器重定位；表项直接存储跳转指令（如 B 指令）。
- 中断向量获取时序（P37）：device → CPU 依次经历 receive request → receive ack → receive vector。

9. 通用中断机制（P38-P39）

- 通用中断处理流程（P38，流程图）：
  1. 检查是否有中断请求（intr?）；无则继续执行；
  2. 有则进行**优先级选择**：中断优先级 > 当前优先级？否则忽略（ignore）并继续执行；
  3. 是则发出 **ack**；
  4. 询问是否带向量（vector?）：带则 `call table[vector]` 调用向量表对应处理程序；不带则等待/超时（timeout?），超时则总线错误（bus error）。
- **中断序列（Interrupt Sequence）**（P39）：
  1. **CPU 确认请求**：CPU 在每条指令开始前检查未决中断；
  2. **设备发送向量**：收到 ACK 后发送；
  3. **CPU 调用中断处理程序**：以向量作为索引查找地址；
  4. **软件处理请求**；
  5. **CPU 恢复后台程序状态**：中断返回指令恢复 PC 及其他自动保存的状态，返回被中断的代码处继续执行。

10. 中断开销与设计（P40-P49）

10.1 中断开销的来源（P40）

- 中断处理程序执行时间；
- 中断机制开销；
- 寄存器保存/恢复；
- 流水线相关的开销；
- 缓存相关的开销。

10.2 中断设计指南（P41-P42）

- 糟糕的代码很难调试，而**糟糕的 ISR 实际上是不可调试的**。

- 做法：
  1. 在布局中断映射之前不要写任何代码：**列出每个中断，并描述程序应该做什么**；
  2. 中断设计图是一份**预算**，预计在什么情况下中断需要花费的时间；粗略估计每个 ISR 的复杂度；
  3. **ISR 的基本原则是保持处理程序简短**——"短"是用**时间**衡量的，不是用代码大小衡量的；
  4. 避免循环，避免冗长复杂的指令（重复动作、复杂数学等）；
  5. 在 ISR 中**尽快重新启用中断**：先做硬件关键和不可重入的事情，然后执行中断启用指令，给其他 ISR 机会。
- 中断设计图：对每个中断源预估其响应时间预算与 ISR 复杂度。

10.3 中断向量表填充（P43）

- 用指向**空例程的指针**填充所有未使用的中断向量（汇编示例，_start 处的向量表）：
  - Reset_Handler（复位）、Undefined_Handler（未定义指令）、SVC_Handler（Supervisor 调用）、PrefAbort_Handler（预取终止）、DataAbort_Handler（数据终止）、NotUsed_Handler（未使用）、IRQ_Handler（IRQ 中断）、FIQ_Handler（快速中断）。
- 每个表项用 `ldr pc, =Handler` 直接加载跳转地址。

10.4 C 语言还是汇编语言（P44）

- 汇编：可将指令条数换算为时间；若平均一条指令需 x 微秒，易得代码允许复杂度的临界估计。
- C 语言更麻烦：**“用 C 语言科学地编写中断处理程序是不可能的”**——难以估计一行 C 代码的运行时间（每行代码时间变化很大）：
  - 字符串比较可能导致运行库调用，结果完全不可预测；
  - FOR 循环可能只是简单整数比较，也可能带来大量处理开销；
  - 可能发现编译器在中断处理方面很慢——换编译器或改用汇编。
- 结论：**尽量使用 C 语言**（作为工程折中）。

10.5 调试 INT/INTA 周期（P45）

- 中断-确认（INT/INTA）周期流程：
  1. 设备硬件产生中断脉冲；
  2. 中断控制器（若有）以优先级处理多个同时发生的请求，向处理器发出单个中断；
  3. CPU 响应一个中断确认周期；
  4. 控制器在总线上发出一个中断向量；
  5. CPU 读取向量并计算向量所指向的内存地址，获取该值；

6. CPU 推入当前上下文、禁用中断、跳转到 ISR。

10.6 查找丢失的中断（P46）

- 用单个上行/下行计数器构建小电路：对每个中断计数 +1，每个中断确认 -1；若计数器总是显示 0 或 1 则一切正常。
- 经验法则：**在最早的安全点重新启用 ISR 中的中断**，以最小化丢失中断。

10.7 避免 NMI（P47）

- NMI 用于电源故障、系统关闭和即将发生的灾难。
- NMI 甚至会破坏编码良好的中断处理程序：大多数 ISR 在服务硬件的前几行代码中都是**不可重入**的，NMI 也会阻碍堆栈管理工作。
- NMI 与大多数工具混合效果差：调试任何 ISR（NMI 或其他）都相当困难，很少有工具能很好地在 ISR 内单步执行和设置断点。

10.8 断点问题（P48）

- 断点对嵌入式代码就像**海森堡不确定性原理**：停止代码有时是危险的（如控制机械臂时）。
- 解决：使用**实时 trace（real-time trace）**——所有仿真器和一些智能逻辑分析仪都具有的功能。

10.9 简易 ISR 调试（P49）

- 调试 ISR 最快的方法：**肉眼检查（不用复杂工具）**——如果 ISR 只有 10 或 20 行代码，直接审查即可，不要启动各种复杂和不可预测的工具。
- 核心：**保持处理程序简单和简短**。

11. 可重入（Reentrant）（P50-P55）

11.1 定义（P50）

- **可重入**：一个函数或子程序在被中断后，能够**安全地再次被调用**（包括被自身递归调用），且不会导致数据错误或逻辑混乱。
- 嵌入式世界中，例程可重入必须满足的条件：
  1. **以原子方式使用所有共享变量**，除非将每个共享变量分配给函数的特定实例；
  2. **不调用不可重入的函数**；
  3. **不以非原子的方式使用硬件**。

11.2 原子变量（Atomic）（P51-P52）

- "原子（atomic）“来自希腊语，意为"不可分割的”；计算机世界中指**不能被中断的操作**。

- 示例：`foobar += 1;` 高级语言中常被编译为多条指令（如 `mov ax,[foobar]; inc ax; mov [foobar],ax`），**不是原子的**——可能被中断破坏（P51-P52）。
- x86 的 **`lock` 指令前缀**：确保多线程或多处理器环境下的原子性操作（如 `lock inc [foobar]`）（P52）。

11.3 三条规则的深意（P53）

- 规则 2：调用函数**继承**被调用函数的可重入问题；涉及使用的软件包（如协议栈）需确保可重入。
- 规则 3：硬件看起来很像一个变量；如果处理一个设备需要**多个 I/O 操作**，就会出现重入问题。

11.4 保持代码可重入性（P54）

- 消除不可重入代码的最佳选择：
  1. **避免共享变量**——全局变量是没完没了的调试问题和代码失败的根源；
  2. 使用**自动变量或动态分配内存**；
  3. 最常见方法：**在不可重入代码期间禁用中断**（缺点是增加系统延迟、削弱及时处理外部事件的能力）；
  4. 使用**信号量（Semaphore）**。

11.5 递归函数（P55）

- 递归：函数调用自己。**所有递归函数都必须是可重入的……但并非所有可重入函数都是递归的**。

11.6 异步硬件/固件示例（P56）

- 32 位定时器由硬件低 16 位 + 固件高 16 位（timer_hi）构成：中断每次溢出 `++timer_hi`；`timer_read()` 先读硬件寄存器（low），再读 timer_hi（high），返回 `(high<<16)+low`——这个读法存在竞态隐患。

12. 竞态条件（Race Condition）（P57-P59）

12.1 定义与示例（P57）

- **竞态条件**：设备或系统出现**不恰当的执行时序，而得到不正确的结果**。
- timer_read 的竞态示例：
  1. 读取硬件，得到 0xFFFF；
  2. 在从变量 timer_hi 获取高十六位之前，硬件再次增加，溢出到 0x0000；
  3. 溢出触发中断，ISR 运行，timer_hi 变为 0x0001（而不是几纳秒前的 0x0000）；

4. ISR 返回；timer_read 不知道发生了中断，把新的 0x0001 与先前读到的 0xFFFF 组合，返回 0x1FFFF——一个非常不正确的值。

12.2 解决方法（P58-P59）

1. **先停止计时器再读**：最简单，但会丢失时间；期间关闭中断可消除不必要任务，但增加系统延迟和复杂性（P58）。
2. **双读校验法**：先读 timer_hi，再读硬件计时器，再重读 timer_hi；若两次 timer_hi 值不同则发生了中断，**迭代直到两次读值相等**（P58）。
   - 优点：数据正确、中断保持开启、系统不丢失计数；
   - 缺点：在重负载多任务环境中，例程可能循环相当长时间才获得两个相同读数，**函数执行时间不确定**。
3. **禁用中断法**：在读取前后禁用中断；先读 low 与 high，再检查溢出标志，若溢出则 `++high` 并重读 low，最后恢复中断状态（P59）——消除竞态且执行时间确定。

05 嵌入式软件系统

嵌入式软件开发路线图（P2）

- 系统越来越复杂，嵌入式系统的软件开发逐渐变成软件专家的工作（P2）
- 嵌入式软件团队包含不同领域专家：网络、用户界面设计、硬件设备、传统嵌入式软件设计等（P2）
- 最近 10-15 年，嵌入式软件规模和复杂性爆炸性增长，专门的软件开发团队进入视野；软件团队不断壮大不仅是因为要完成所需代码（P2）

嵌入式软件与桌面软件的对比

对比总览

维度	嵌入式软件（ES）	要点
内存	有限	影响编程语言选择及开发工具（如编译器优化）（P3）
CPU 处理能力	恰好满足要求	受成本和功耗约束，采用保守设计方案（P3）
操作系统	OS（Windows/Linux）、RTOS、专有 OS 或裸机	可选范围宽（P3）

维度	嵌入式软件（ES）	要点
实时行为	可预测（确定性）	不一定要运行得很快，但一定可预测；影响 OS 选择和程序设计（P4）
开发流程	交叉开发	编辑/编译/调试循环相同，但"执行程序"一步复杂（P4）
执行流程	开机即运行直到关机	程序可存于 ROM，也可从非易失性存储器转移到 RAM 执行（P5）
开发工具	交叉编译器、灵活链接器、多形态调试器	与宿主机开发显著不同（P6）
软件组件	嵌入式专用库	可重入、可存 ROM；需网络、存储、GUI 支持（P7）

内存、CPU、操作系统（P3）

- 内存有限，影响编程语言选择与开发工具的使用（如编译器优化）
- 受成本和功耗考虑，ES 不得不采用保守设计方案，CPU 往往只是恰好满足要求
- 操作系统可选：OS（Windows/Linux）、RTOS、专有 OS 或裸机

实时行为（P4）

- 实时系统不一定会运行得很快，但一定是可预测的，通常的术语叫**确定性**
- 实时性要求对 OS 的选择和程序设计都有影响

开发流程（P4）

- 采用交叉开发，对工具选择影响很大
- ES 往往没有足够资源进行软件开发
- 编辑/编译/调试的循环与普通开发相同，但执行程序一步非常复杂：需把代码转移到目标机上，或在某种环境下运行

执行流程（P5）

- 多数嵌入式设备从开机开始就运行某个程序，一直运行到系统关机；程序可能存于 ROM，也可能从非易失性存储器转移到 RAM 中执行
- 每一个嵌入式设备都是不同的：
  - 技术层面：不同的 CPU 架构、内存、外设、应用程序和操作系统
  - 商业运作层面：如手机开发/生产的商业模式与核磁共振扫描仪完全不同

开发工具（P6）

- **交叉编译器**：嵌入式应用中有时直接包含小部分汇编代码
- **链接器**：嵌入式内存映射复杂、精确性要求严格，链接器必须灵活应对，负责在内存中正确定位代码和数据
- **调试器**：不是单一组件，而是一系列对应不同运行环境的工具：
  - 借由在本地或指令集模拟器中运行代码，代码可在宿主机上运行
  - 调试器可与目标板连接（JTAG 或其他方式如以太网）并执行代码
  - 其他针对性工具：分析实时性能、代码执行时设备功耗情况的分析器

软件组件（P7）

- 嵌入式编译器提供的库应适合嵌入式环境：可以重入（reentrant）、可以被存储在 ROM 中
- 其他库（尤其 C++ 标准模板库 STL）可能并不适合嵌入式应用，使用时需小心
- 与硬件交互是难点之一，OS 内部设备驱动提供这样的接口
- ES 互联性不断提高，需不同网络支持：TCP/IP 或总线 CAN、I2C、SNMP、Zigbee、Bluetooth、WiFi；最重要的是检查有效性、确保符合标准、能与其他系统互通互联
- 嵌入式设备对有组织的数据存储有需求，如能应对电源故障、支持多线程
- LCD（往往是触摸式）成本降低使其进入嵌入式设计，软件面临新挑战：支持复杂的图像和用户界面

软件主导硬件、软硬件权衡 📌【了解】

软件主导硬件（P8）

- 只有少数工程师软硬件都懂；可能有一个软硬件都做的工程师，更有可能有两个团队
- 有关硬件的决策会对软件产生持久影响：一开始的决定一般暗藏了最后通过的设计，并总会在最终产品中得到实现
- 硬件中克扣费用或功耗，可能增加额外编码时间；过分夸大需求会浪费硬件、增加单位产品成本

软件/硬件的权衡（P9）

- 微处理器选择：可能影响软件效率；高性能软件开发工具的可用性反过来影响处理器选择
- 内存大小和组合：尽可能晚地决定存储器的确切数量和搭配（ROM/RAM 混用）；ROM/RAM 具互换性，调试时可用 RAM 替换大部分 ROM
- 外设：需认真考虑设计中包含哪些外设，重点考虑硬件费用；有时可由软件替代（如定时器）

调试（P10）

- **在线仿真器 ICE（In-Circuit Emulator）**：模拟 CPU 功能，可完全仿真芯片行为；价格昂贵、难以普及，未广泛使用
- **监控调试器**：需要通信通道；因占用系统资源，在严格场合不适合使用
- **片上调试**：价格便宜、易于实现、广泛使用；例如 JTAG（边界扫描测试标准协议）、SWD（Serial Wire Debug）仿真器

自检（P11）

- I/O 电路："loop back"（回环）测试是有用的
- 板载开关：用于配置或模式选择
- 状态显示：可能是字符或只是 LED；LED 至少可以有 3 种状态：开、关、闪烁

嵌入式软件架构（P12）

- 分为两部分：**业务逻辑 + 实时依赖硬件的逻辑**
- 抽象层将所需操作的高级请求转换为操作所需的低级命令
- 整体结构：应用业务逻辑架构（独立于硬件）→ 抽象层 → 实时软件架构（依赖硬件）

嵌入式软件架构模式（P13）

- 基于微控制器的系统最常用的架构模式：非结构化单体架构、分层架构、事件驱动架构、微服务架构

非结构化单体架构（P14）

- 很容易构建，但很难维持规模和移植；与应用层的应用程序紧密耦合
- 图示（P14）：应用之下平铺众多模块（Module），模块间无清晰边界、高耦合

分层架构（P15）

- 当今嵌入式应用程序中最常用的架构
- 将应用程序逻辑划分为若干独立的层，仅通过定义良好的抽象层进行交互；旨在改善非结构化单体架构的高耦合性
- 层次（自上而下）：应用层 → 中间件层 → RTOS/OS 层 → 硬件驱动层 → 硬件（P15 图）
- 软件系统层次结构图（P16）：展示嵌入式软件自应用层到硬件驱动的分层组织关系

事件驱动架构（P18）

- 对实时嵌入式应用程序和与能耗相关的应用程序非常有意义
- 通常利用中断立即响应事件；系统通常处于空闲或低功耗状态，除非有事件触发要执行的操作

- 通常使用消息队列、信号量和事件标志来表示系统中发生了事件
- 图示（P18）："按钮点击"产生事件 → 生成消息 → 处理器处理 → 唤醒系统执行，随后回到等待状态
- **优势**：具有相对可扩展性，是可伸缩和可移植的；软件模块通常高内聚；低耦合——每个事件最小化依赖关系，模块与消息传递系统解耦
- **缺点**：无论何时需要做任何事情，都有额外的开销和复杂性

微服务架构（P19）

- 将应用程序构建为为业务领域开发的小型自治服务的集合
- 微服务本质上是低耦合的，易于维护和测试，开发人员可以快速扩展或移植
- 围绕系统的业务逻辑组织；业务逻辑（有时称业务功能）是系统行为的业务规则和用例
- 对嵌入式系统而言，微服务可以远远超出系统业务逻辑
- 图示（P19）：气象站微服务示例——用户接口、命令服务、电机服务、传感器服务、遥测服务等多个自治服务

微服务架构的不足（P20）

- 架构上增加设计的复杂性
- 因具有其他体系结构中可能不需要的通信特性，可能增加额外开销和内存需求
- 架构的分散性使实时的、确定性的行为更具挑战性，实时和响应可能有额外抖动
- 可能增加开发时间和预算

嵌入式软件设计模式（P21）

实时嵌入式软件常用的设计模式（P22）

- 单核：轮询（Polling）、中断处理（ISR）、DMA
- RTOS 模式、发布和订阅模型（Pub/Sub）、低功耗设计、多核

管理外设数据（P23）

- 主要设计理念之一：**数据决定设计**
- “如何将数据从外围设备获取到应用程序？”——三种基本机制：轮询、中断、直接存储器访问（DMA）
- 这些机制都有若干设计模式，可用于确保不会丢失数据

外设轮询（Polling）（P24-P27）

- **适用场景**：从外设收集数据的最直接机制；外设数据量小、实时性要求不高、单任务简单系统
- **机制**：应用程序定期（循环）轮询外设，查看是否有任何数据可供管理和处理

    - Control Loop 模式（P25）：主循环中一切都是函数调用——`while(TRUE){ if(device_A requires service) service; if(device_B...) ... }`，依次服务所有设备后从头再来
    - 轮询变化（P26）：低优先级任务需要放慢速度——在循环中多次检查高优先级设备（如 device_A 被反复服务），确保其按时被服务
- 示例（P28）：Arduino 按下按钮点亮 LED——`loop()` 中 `digitalRead(BUTTON)` 轮询按钮输入，为 HIGH 则 `digitalWrite(LED, HIGH)`
- 示例（P29）：Arduino 看门狗 Watchdog——`wdt_enable(TIMEOUT)` 使能看门狗（如 WDTO_8S），主循环定期 `wdt_reset()` 喂狗；若超时未喂狗则触发系统复位
- **总结**：优先级——无，一切按顺序运行；响应时间——所有任务的总和；变化的影响——显著（修改任务执行时间或添加任务会影响所有其他任务）
- **优点**：简单；没有共享数据问题；定时检查外设状态寄存器以查看数据是否可用
- **缺点**：浪费处理周期，在资源受限或低功耗系统中这些周期可能显著增加；处理外设时可能有很多延迟和抖动

有限状态机（FSM）（P30-P32）

- **适用场景**：行为可划分为有限状态、事件驱动转移的控制类应用（如按键处理）
- **机制**：类似于轮询，但只执行当前状态；每个状态决定下一个状态（非顺序执行）
    - 示例代码（P30）：`while(1){ switch(state){ case IDLE: 检查按键/显示，有按键则 state=SHOW; case SHOW: 累加显示值，state=IDLE; } }`
- **总结**：优先级——每个状态决定下一个状态的优先级；响应时间——所有任务的总和；变化影响——显著（修改任务执行时间或添加任务影响所有其他任务）；简单性——没有共享数据问题
- **优点**：结构清晰、易于实现、无共享数据问题
- **缺点**：一次只处理一个状态，响应时间为所有任务总和；状态变化影响全局（P32）

中断设计模式（ISR）（P33）

- **机制**：中断应用程序的正常流程，以允许中断处理程序（ISR）运行代码处理系统中发生的事件；例如数据可用、已接收甚至已传输时可能触发外设中断
- **适用场景**：需要及时响应外设事件、避免轮询浪费 CPU 周期的场景
- **ISR 设计要点**：
    - 希望中断尽可能快地运行（以最小化中断）
    - 避免内存分配操作：不声明非静态变量、不操作堆栈、不使用动态内存
    - 尽量减少函数调用，以避免时钟周期开销、不可重入函数或阻塞函数的问题

前后台系统（Foreground/Background）（P34）

- 前台（ISR）处理紧急事件，后台（主循环）处理非紧急任务——由"中断 + 主循环轮询"构成的前后台模型

带有中断的轮询（P35-P37）

- **机制**：ISR 处理高优先级设备的紧急需求并置标志位（如 `flag_A = TRUE`），主循环检查标志后做后续（低优先级）处理，再轮询其他设备
- **总结（P36）**：优先级——中断优先于主循环（中断有自己的优先级）；响应时间——所有任务的总和或中断执行时间；变更影响——对 ISR 影响较小，与主循环的轮询相同；共享数据——必须处理与中断服务例程共享的数据；中断不能进行参数传递
- **优点**：不需要浪费 CPU 周期检查数据是否准备好；获取数据的延迟是确定的；抖动被最小化
- **缺点**：中断的设置可能比较复杂；必须小心不要使用频繁触发的中断；增加了软件设计的复杂性；使用中断接收数据时开发人员必须仔细管理 ISR 中所做的工作——常需 ISR 处理即时操作，把非紧急工作卸载给应用程序

数据获取/存储相关的中断设计模式（P38）

- 包含：线性数据存储、乒乓缓冲/双缓冲（Double Buffer）、环形/循环缓冲区（Ring Buffer）、带信号量的循环缓冲区、带事件标志的循环缓冲区、消息队列

线性数据存储设计模式（P39）

- **机制**：ISR 直接访问的共享内存位置，用于在 ISR 与应用程序之间传递数据
- **适用场景**：最简单的中断数据传递；但危险：是经常遇到竞态条件的地方
- **要点**：用于存储应用程序和 ISR 之间数据的共享变量必须声明为 `volatile`，防止编译器优化；数据存储必须由互斥锁（M）保护，防止竞态条件
- 图示（P39）：外部设备 → ISR → 数据存储 → 应用程序，数据存储处加互斥锁 M 保护

乒乓缓冲区/双缓冲区设计模式（P40）

- **适用场景**：缓解线性数据存储遇到的竞态条件问题
- **机制**：设置两块缓冲区交替使用——ISR 写入一块的同时应用程序处理另一块，写满后切换
- **优点**：降低读写冲突与竞态条件；**缺点**：需要两倍缓冲区内存（P40）
- 图示（P40）：外部设备 → ISR → 缓冲区（大小）→ 应用程序，数据先入缓冲区再被应用消费

环形缓冲区设计模式（P41）

- **定义**：环形缓冲区（Circular Buffer），也称循环缓冲区（Cyclic Buffer）或环形队列（Ring Buffer），是一种在内存中形成环形存储空间的数据结构
- **特点**：终点和起点相连，形成环状结构

- **适用场景**：处理流数据、实现数据缓存等；中断中接收的实时数据可从外设移除并存入循环缓冲区
- **机制**：中断尽可能快地运行（仅负责把数据放入缓冲区），应用程序代码自行处理循环缓冲区
- **优点**：有助于确保数据不丢失；中断速度快；数据得到合理处理
- 图示（P41）：外部设备 → ISR → 缓冲区（大小）→ 应用程序

带有通知的循环缓冲区设计模式（P42）

- **机制**：在环形缓冲区基础上，由 ISR 向应用程序发出"有新数据"的信号，避免应用反复轮询缓冲区（如 Arduino `Wire.available()`）
- **通知方式**：信号量（S）和事件标志（EF）；在大多数实时操作系统中，使用事件标志比使用信号量更有效
- 图示（P42）：外部设备 → ISR → 缓冲区 → 应用程序，附 EF（事件标志）或 S（信号量）通知通道

消息队列设计模式（P43）

- **机制**：类似于使用带信号量的线性缓冲区，但通过消息队列传递结构化的消息
- **缺点**：通常需要更多的 RAM、ROM 和处理能力
- 图示（P43）：外部设备 → ISR → 消息队列 → 应用程序

直接存储器访问（DMA）（P44）

- **定义**：DMA（Direct Memory Access），无需 CPU 交互即可在 RAM 与外设之间以及内部传输数据
- **传输类型**：RAM 到 RAM、外设到 RAM、外设到外设
- 图示（P44）：CPU、DMA 控制器、外部设备、RAM 通过总线（Bus）连接，DMA 控制器可独立于 CPU 搬运数据

DMA 控制器将外设数据传输到循环缓冲区的设计模式（P45）

- **适用场景**：高数据吞吐量的外设（如模数转换器 ADC）批量数据搬运
- **机制**：DMA 控制器直接把外设数据搬到循环缓冲区，传输完成触发 ISR 通知（如信号量 S），应用程序处理缓冲区
- **优点**：显著提高外设和应用程序之间的数据吞吐量；减轻 CPU 运行 ISR 传输数据的负担，最大限度地减少浪费的计算周期
- **示例**：ADC 中特别常见——利用 DMA 执行和传输所有转换后的模拟通道，然后将它们复制到缓冲区
- 图示（P45）：外部设备 → DMA → 缓冲区（大小）→ 应用程序，ISR 与信号量（S）负责完成通知

RTOS 应用程序设计模式（P46）

- RTOS 应用中通常有两种类型的同步：
  - **资源同步**：决定对共享资源的访问是否安全
  - **活动同步**：决定执行是否已达到特定状态
- 资源同步和活动同步设计模式有多种形式和大小

资源同步（P47）

- **目标**：确保需要访问资源（如内存位置）的多个任务，或任务与中断，以协调的方式进行，避免竞态条件和内存损坏
- **三种处理方式**：中断锁定（中断锁）、抢占锁定（抢占锁）和互斥锁

中断锁（P48）

- **机制**：系统任务禁用中断，以在任务和中断之间提供资源同步
- **适用场景**：简单、要求低的临界区保护
- **缺点**：在实时嵌入式系统中可能导致许多问题——例如读取操作期间禁用中断，中断可能被错过，或最好情况下延迟执行，给系统增加不必要的延迟甚至抖动

抢占锁（P49）

- **机制**：确保任务在执行临界区期间不间断——临界区期间临时禁用 RTOS 内核抢占调度程序；关键的系统中断仍然允许运行
- **适用场景**：任务间（非中断）临界区保护
- **潜在问题**：给系统带来额外延迟和抖动；由于内核抢占式调度器被禁用，高优先级任务的执行可能被延迟

互斥锁（P50）

- **机制**：保护共享资源最安全、最推荐的方法之一；通过创建对象保护临界区，检查该对象状态以确定是否可以安全访问共享资源，其唯一目的是为共享资源提供互斥
- **特点**：不会禁用中断，不会禁用内核的抢占调度程序
- **潜在问题**：开发人员需要知道它的存在！（忘记使用或未正确使用则失去保护）

活动同步（P51）

- **目标**：协调任务的执行
- **示例**：开发获取传感器数据并用传感器值驱动电机的系统——需要向运动任务发出信号"有新数据可用"，这样任务就不会对陈旧的数据采取行动
- 使用 RTOS 时，可用许多活动同步模式协调任务执行

单向同步（任务对任务）（**P52**）

- 使用二值信号量或事件标志来同步任务

单向同步（中断到任务）（**P53**）

- 在中断和任务之间同步、协调任务执行；不同之处在于：ISR 给出信号量或事件标志之后，ISR 将继续执行，直到完成为止
- 也可以与计数信号量一起使用

双向同步（**P54**）

- 两个任务在它们之间的两个方向上进行协调

同步多个任务之广播设计模式（**P55**）

- **机制**：允许多个任务阻塞，直到给定信号量、出现事件标志，甚至将消息放入消息队列；任务或 ISR 可提供由多个任务使用的二值信号量广播
- **注意事项**：广播可能无法在所有实时操作系统中实现，必须检查 RTOS 是否支持
- **替代方案**：若不支持广播，可创建由任务或中断给出的多个信号量；从设计角度看使用多个信号量并不优雅，从实现角度看也不高效，但有时软件中只能如此

发布和订阅模型（**Pub/Sub**）（**P56-P57**）

- **适用场景**：物联网领域广泛使用；ROS 也使用该模型
- **机制**：在许多情况下，物联网设备启动电源 → 连接到云 → 订阅它想要接收的消息主题，同时也可以发布特定主题
- **缺点**：会导致应用程序占用更大的内存
- **优点（IoT 例子，P57）**：打破了传感器任务和任何需要使用数据的任务之间的依赖关系；系统完全可扩展——如果突然需要使用传感器数据进行故障处理，不必回头重新构建系统或添加另一个依赖，只需创建一个订阅传感器数据的故障任务，该任务即可获得操作和检测故障所需的数据

低功耗应用程序设计模式（**P58**）

- **核心原则**：主要模式是尽可能地保持设备关闭
- **机制**：事件驱动中，事件之间没有实际工作要做时，将微控制器置于适当的低功耗状态，并关闭任何非必要的电子设备；除非发生唤醒事件，系统处于低功耗状态；实际工作完成后，系统恢复到低功耗状态
- **建议**：
  - 使用内置 tickless 模式的 RTOS，或可扩展系统 tick 以使微控制器休眠更长时间的 RTOS
  - 例如 FreeRTOS 有一个可以启用的 tickless 模式
  - 开发人员可以添加自定义代码来管理低功耗状态

- 当系统进入睡眠状态时，系统节拍（tick）被自动缩放，因此发生的系统节拍更少

多核架构（P59）

- **同构多核**：对称多核处理，具有相同处理器架构的两个核
- **异构多核**：每个处理核心都有不同的底层架构
- **示例（P59）**：Cypress PSoC 64——用户应用程序用 Arm Cortex-M4，安全处理器用 Arm Cortex-M0+；两个核心不必以相同时钟速度运行（M4 工作频率 150MHz，M0+ 为 100MHz）

人工智能用例（P60）

- 在微控制器上运行机器学习（ML）推理是可以做到的，但计算周期密集；理念是一个核心运行 ML 推理，另一个核心做实时控制
- **示例**：STM32H7 系列多核部件——Arm Cortex-M7 运行机器学习推理，Arm Cortex-M4 做标准实时工作（电机控制、传感器采集和通信）；M4 内核向 M7 提供运行 AI 算法所需的数据，M7 向 M4 提供结果

实时控制用例（P61）

- 多核微控制器系统中经常使用的用例：让每个核心管理实时控制能力
- **示例**：呼吸机系统使用多核微控制器 STM32H7——M7 内核驱动 LCD 和管理内存，M4 运行控制泵和阀门所需的各种算法

安全解决方案用例（P62）

- 多核的另一个流行用例：应用程序管理安全解决方案
- **机制**：利用多核内置的硬件隔离，将其中一个核作为安全处理器，处理安全操作和信任根；另一个核为正常的应用空间
- 数据可以通过共享内存在内核之间共享，但内核之间只能通过处理器间通信（IPC）请求进行交互

参考文献（P63）

- Jacob Beningo, *Embedded Software Design: A Practical Approach to Architecture, Processes, and Coding Techniques*. Apress, 2022. Chapter 2 and 5

第6章 嵌入式操作系统 ⭐【重点】

6.1 RTOS、物联网操作系统

嵌入式操作系统概述（P2）

- 伴随人工智能快速发展，嵌入式操作系统在智能系统中发挥越来越大的作用
- 历史悠久、门类繁多、用途广泛：不仅包括嵌入式实时多任务操作系统（RTOS）、开源 Linux、机器人和路由器操作系统，还包括新型的物联网操作系统以及边缘计算操作系统平台
- 近年国内嵌入式操作系统市场规模整体保持稳定：2022 年我国嵌入式操作系统市场规模约 469 亿元（工信部数据）

实时系统（P3）

- 定义：计算的正确性不仅取决于程序的逻辑正确性，也取决于结果产生的时间；如果系统的时间约束条件得不到满足，将会发生系统出错

术语（P4）

- 确定性（Determinism）：如果一个系统始终会为某个已知输入产生相同的输出，则该系统是确定性的
- 非确定性系统的输出具有随机变化特征
- 截止时限（Deadline）：必须完成某项任务的有限时间窗口，指明计算何时必须结束

实时操作系统（P5）

- 实时操作系统（Real-Time Operating System, RTOS）是支持构建实时系统的操作系统
- 许多嵌入式系统都是实时系统，因此用于该类系统的操作系统必须是实时操作系统
- 实时计算系统同时需要实时运行的操作系统和提供确定性执行的用户代码
- 非实时操作系统上的确定性用户代码、实时操作系统上的非确定性用户代码，都不会产生实时性能

RTOS 与 GPOS 的相似功能（P6）

- 多任务级别
- 软件和硬件资源管理
- 为应用提供基本的 OS 服务
- API 接口抽象

RTOS 与 GPOS 的不同功能（P7）

- 更快的特性
- 满足应用需要的剪裁能力
- 减少内存需求
- 为实时嵌入式系统提供可剪裁的调度策略
- 嵌入式应用上下文中具有更好的可靠性
- 支持无盘化嵌入式系统，允许从 ROM 或 RAM 上引导和运行
- 对不同硬件平台具有更好的可移植性

RTOS 关键要求（P8）

- 操作系统的时间行为必须是可预测的
- 任何调度策略都必须是确定性的
- 为避免关键事件处理过程中的不可预测延迟，禁止中断的时间必须尽可能短
- 操作系统必须管理线程和进程的调度
- 一些系统要求操作系统管理时间：对时间精度的要求可能不同；一些与环境的连接（如 GPS 或移动网络）可用来获取精确的时间信息
- 操作系统必须是快速的
- 可靠性
- 简洁紧凑

为何使用 RTOS（P9）

- 可被复用的标准软件组件
- 灵活性
- 响应时间

RTOS 类别（P10）

- 快速专有内核：µC/OS、FreeRTOS
- 对标准操作系统的实时扩展：
  - 单内核：PREEMPT_RT——Linux 内核补丁，将 Linux 调度器修改为完全可抢占（fully preemptible）；2015 年 Linux 基金会创建 Real-Time Linux (RTL) 协作项目以协调将其合并到主线内核；实时抢占式补丁（PREEMPT_RT）于 2024 年 9 月被纳入主线内核
  - 双内核：Xenomai——符合 POSIX 标准的协同内核（co-kernel）或管理程序，与 Linux 内核协作的实时内核，Linux 内核被视为实时内核调度器的空闲任务（最低优先级任务）；RTAI——协同内核（co-kernel）的替代解决方案

对标准操作系统的实时扩展（P11）

- 优点：可配备标准操作系统的 API，具有 GUI、文件系统等；标准 OS 的增强功能也可很快速地应用在嵌入式领域；非实时进程不会影响实时进程
- 缺点：设备驱动程序存在一些问题，为避免冲突，有必要将设备划分为由实时进程处理和由标准 OS 处理的两大类型；实时进程不能使用标准 OS 的服务，因此文件系统访问、GUI 等优秀特性对实时进程一般不可用

最流行的嵌入式操作系统（P12）

- 图：2023 年 embedded.com 嵌入式调查显示最流行的嵌入式操作系统排名（来源：https://www.embedded.com/embedded-survey-2023-more-ip-reuse-as-workloads-surge/）

国产嵌入式操作系统（P13）

- HUAWEI Lite OS 鸿蒙操作系统：用于可穿戴、智能家居、车联网
- “道系统”操作系统 DeltaOS：实时，可替换 VxWorks 6.8/6.9；用于军用、工业互联网、物联网等
- SylixOS 实时操作系统：硬实时，借鉴了 RTEMS、VxWorks、ThreadX 等的设计思想；用于航空航天、国防安全、网络设备、汽车电子、机器人等
- AliOS Things：轻量级物联网嵌入式操作系统，云端一体
- RT-Thread：物联网操作系统，开源；集 RTOS 内核、中间件组件和开发者社区于一体的技术平台，组件丰富

RTOS 内核系统服务（P14）

- 任务管理、同步与通信、内存管理、时间管理、IO 管理、异常与中断管理

物联网操作系统（P15）

- 物联网（IoT）源于大量廉价、小巧、节能的通信设备（或“物”）
- 从硬件上看，物联网是由异构硬件组成的
- 图：低端 IoT 设备示例——Arduino Due、Zolertia Re-Mote、IoT-LAB-M3、Atmel SAM R21，展示物联网硬件的异构性

物联网操作系统的要求——内存/异构硬件（P16）

- 内存占用小：需在性能、方便的 API、操作系统内存占用小之间作出正确的权衡
- 支持异构硬件：物联网设备基于各种微控制器（MCU）架构和家族，包括 8 位（Intel 8051/52、Atmel AVR）、16 位（TI MSP430）、32 位（ARM7、ARM Cortex-M、MIPS32、甚至 x86），64 位架构未来也可能出现；设备可配备各种各样的通信技术；调查发现嵌入式环境中更换处理器频次比以前想象的更为常见

物联网操作系统的要求——网络连接/节能（P17）

- 网络连接：不仅包括各种低功耗无线电技术（IEEE 802.15.4、蓝牙/BLE、LoRa 等），还包括各种有线技术（PLC、以太网或几种总线系统）
- 节能：许多物联网设备使用电池或其他有限的能源；全球预计部署的设备数量巨大（数百亿），必须考虑节能
- 关键需求：为上层提供节省能源的方案，尽量利用功能本身，如使用无线电周期工作技术，或尽量减少需要执行的周期性任务的数目

物联网操作系统的要求——实时（P18）

- 实时功能：RTOS 设计目的是保证最坏情况下的执行时间和最坏情况下的中断延迟，这通常意味着内核函数必须使用确定性的运行时进行操作
- ITRON（Industrial the Real-Time Operation System Nucleus，工业实时操作系统中心）：日本实时操作系统的开放标准，在该领域很受欢迎，尽管其目标主要是消费电子产品

物联网操作系统的要求——安全（P19）

- 一些物联网系统是具有生命安全影响的关键基础设施或工业系统的一部分；另一方面设备与互联网相连，普遍期望满足高安全性和隐私标准
- 安全挑战：除主要的信任管理挑战外，还包括物联网架构中各个部分的数据完整性、身份验证和访问控制
- 需求（和挑战）：提供必要的机制（密码库和安全协议），同时保持灵活性和可用性

通用架构和模块化（P20）

- 图：低端物联网设备操作系统的典型组件，包括通用的低功耗 IPv6 协议栈

调度模型（P21）

- 两种调度器：抢占式调度器；非抢占（或合作）调度器

内存分配（P22）

- 内存以静态还是动态的方式分配是一个重要的问题
- 动态内存分配存在的问题：malloc() 和相关函数在标准 C 库中通常以时间不确定性的方式实现，将破坏任何实时保证（为实时应用提供动态内存分配，操作系统必须提供特殊的确定性 malloc() 实现）；动态内存分配需要在运行时处理内存不足等情况，这可能很难处理；基于堆的 malloc 实现通常会导致内存碎片，从而使系统更快地耗尽内存

网络缓存管理（P23）

- 两种可能的解决方案：复制内存（memcpy()）——从资源的角度看是昂贵的；在几个层之间传递指针——产生谁负责分配内存的问题

编程模型（P24）

- 物联网操作系统领域的典型编程模型：事件驱动系统；多线程系统

编程语言（P25）

- 主要选择：一种标准的编程语言，通常是 ANSI C 或 C++；特定于操作系统的语言（如 nesC、TinyOS）

驱动模型和硬件抽象层（P26）

- 灵活、合理方便的驱动接口对物联网操作系统至关重要
- 物联网系统以多种方式与环境交互：被动方式（通过传感器感知）、主动方式（通过执行器，如电机或照明系统）
- 这些系统的 MCU 通常配备各种外围设备：ADC/DAC、接口（SPI、I2C、CAN 总线或串行线）和 GPIO

调试工具（P27）

- 编程语言的选择也预先决定了可能使用的工具（包括调试工具），完善的工具链通常包括相应的调试工具

- 例：围绕 GNU 编译器集合（GCC）的工具链包含 GNU 调试器（GDB）
- 为运行实时调试系统，目标板必须提供适当接口，如 JTAG 或 Spy-Bi-Wire（两线制 JTAG）
- 常见辅助工具：使用 printf() 等工具在串行接口上进行简单调试；有时甚至一个简单的 LED 闪烁算法也可以作为原始的调试替代

特征集（**P28**）

- 操作系统可以分为内核和更高级别的功能：内核通常提供调度器、任务模型、互斥（mutex）和其他形式的同步以及计时器；如果支持多线程，API 通常也包含 IPC 函数
- 较高层：系统库如 shell、日志、密码函数或网络栈；低端物联网设备还包括空中更新 OTA、动态加载和链接、轻量级加密/解密库
- 由于 IoT 设备上通常缺少 MMU，应用程序和应用程序库通常与内核运行在相同的地址空间中，因此会降低系统的稳定性

测试（**P29**）

- 物联网系统测试的挑战来自：系统的分布式特性；典型的嵌入式以及常常是受限系统的事实
- 处理与硬件相关的测试部分（如设备驱动程序的测试）的广泛使用方法是硬件仿真工具，如 MSPSim 或 Emul8

非技术属性（**P30**）

- （开放的）标准、认证、文档、代码成熟度、代码许可、操作系统提供商

（开放的）标准（**P31**）

- 开发标准化 API（如可移植操作系统接口 POSIX，由 IEEE 和 Open Group 指定），以简化操作系统之间的软件移植
- 低端 IoT 设备上因软件大小限制，设计实现类似 Linux 等通用操作系统的标准 API 可能很困难（事实上即使在 PC 上也很少有操作系统能声称完全符合 POSIX）
- 标准不仅在系统层面重要，在网络层面也不可避免：IETF

认证（**P32**）

- 对于某些用例（如建筑自动化等应用中的关键系统），关键属性包括实时功能、健壮性或确定性、安全功能，通过独立机构认证成为不可避免的要求
- IEC 61508 标准：《电气/电子/可编程电子安全相关系统的功能安全》
- “IPv6 Ready” Logo 测试认证：IPv6 论坛发起的国际测试认证计划，验证 IPv6 协议一致性和互连互通性，提供公正性测试规范
- IPSO（IP for Smart Objects）联盟合规和认证程序：推动 IP 协议作为网络互联技术连接传感器节点或其它智能物件以传输信息；智能物件（Smart objects）指物理世界中在嵌入式设备协助下能传输状况和环境信息（如温度、光强、移动、健康状况）的物体，信息被传输到中心节点进行分析并与其它信息关联以做出进一步行动

文档（P33）

- 完整且易于理解的文档对任何软件都非常重要，对操作系统更加重要（操作系统是运行在系统上每个软件的基础）
- 对嵌入式软件来说，对完整文档的需求更加迫切：受各种约束条件，这些软件通常不得不做出妥协，而妥协的原因乍一看很难把握
- 典型指标：每行代码的文档百分比（不一定是太有意义的度量）

代码成熟度（P34）

- 比文档质量更难衡量
- 非常粗略的指标：项目年龄与贡献者和用户数量相结合
- 认证在许多情况下主要是一种法律保障，系统的实际健壮性和正确性要难以评估得多

许可证（P35）

- 一般来说有三种许可证类型：收费；开放源码（如 BSD、MIT 或 Apache 许可，给予开发者和用户高度自由，通常比公共版权许可更容易被业界接受）；版权许可（copyleft，如 GPL 和 LGPL）

OS 提供者（P36）

- 操作系统的代码可能以不同形式和由不同实体提供：由实际开发软件的供应商提供，或由第三方提供（第三方也可能提供商业支持）
- 开源解决方案：代码通常由开发人员社区通过版本控制系统的存储库（如 Git、Subversion 或 Mercurial）提供
- 社区通常通过在线论坛、开放 Issue 跟踪管理系统和邮件列表提供最佳支持；强烈建议选择当前活跃社区的开源项目
- 有时不仅为商业操作系统，也为免费的开源操作系统提供专业的软件咨询

物联网操作系统候选（P37）

- 开源操作系统、闭源操作系统、物联网的其他软件库或中间件
- 如果没有另外提到，所有操作系统都用 C 编程语言编写，而一些特定于硬件的部分可以用汇编语言实现

开源与闭源操作系统（P38-P40）

- 开源操作系统（P38）：图为开源操作系统列举
- 闭源操作系统（P39）：ThreadX、QNX、VxWorks、PikeOS、embOS、Nucleus RTOS
- 关注开源操作系统的原因（P40）：通过物联网设备上运行代码的透明度实现安全性和可信赖性；在多方之间分摊开发成本（类似于 Linux）的预期需要

与物联网相关的操作系统分类（P41）

- 事件驱动的操作系统、多线程操作系统、纯 RTOS

事件驱动 OS（P42）

- 最初针对无线传感器网络领域开发的操作系统最常见的方法
- 核心思想：系统上所有处理都由（外部）事件触发，通常由中断发出信号；内核大致相当于一个无限循环，处理同一上下文中发生的所有事件；事件处理程序通常运行到结束
- 在内存消耗和低复杂度方面有效，但对程序员施加了一些重要约束（例如并非所有程序都能容易地表示为有限状态机）
- 代表：Contiki、TinyOS、OpenWSN

多线程 OS（P43）

- 通常会引入内存开销（堆栈预留空间）和运行时开销（上下文切换）
- 代表：RIOT、NuttX、ChibiOS；RIOT 更专注于物联网需求，是这类操作系统的一个很好的代表

纯 RTOS（P44）

- 在工业/商业环境中主要关注实现实时保证的目标，正式的验证、认证和标准化通常至关重要
- 为允许模型检查和形式化验证，其编程模型通常对开发人员施加严格约束，常使操作系统变得相当不灵活，移植到其他硬件平台可能相当困难
- 代表：FreeRTOS、eCos、RTEMS、ThreadX 及一系列商业产品（通常是封闭源代码）；FreeRTOS 是最著名的物联网设备开源 RTOS

各类别代表的关键特征（P45）

- 图：事件驱动、多线程、纯 RTOS 各类别代表操作系统的关键特征对比（参考文献：Hahm 等《Operating Systems for Low-End Devices in the Internet of Things: a Survey》）

6.2 实时调度

实时系统所需的调度策略 📌【了解】（P49）

- 任务是一个任务集合，需要调度策略
- 涉及：优先级排序约束、时间约束、截止时限
- 任务可能被要求在不早于某一特定时间执行
- 任务可能相互依赖，也可能相互合作形成一个应用程序；也可能不相关，只是共享处理器资源

调度程序 📌【了解】（P50）

- 当面临并发程序或一组程序要执行时，调度程序决定接下来执行哪个任务
- 多处理器调度程序：决定下一步执行哪个任务，以及由哪个处理器来执行（处理器分配）
- 调度决策包括：分配（应该由哪个处理器执行任务）、排序（每个处理器按什么顺序执行相应任务）、定时（每项任务什么时候执行）

调度程序分类（根据决策制定的时间）📌【了解】（P51-P52）

- 完全静态调度程序（P51）：在系统设计时制定三项决策；明确规范；通常不需要信号和互斥锁，可通过定时来实现互斥和优先序约束；在大多数现代微处理器上难以实现（执行时间难以准确预测，而且通常具有数据依赖性）
- 静态顺序调度程序/离线调度程序（P51）：在设计时完成任务的分配和排序，但直到任务运行时才确定每个任务的物理执行时间；可能受是否获取互斥锁、是否满足优先序约束等因素影响；不会根据信号或互斥锁的状态改变任务的顺序
- 在线调度程序（P52）：
    - 静态分配调度程序：在设计时制定任务的分配，其他决策在任务运行时制定；每个处理器被分配一组执行任务，运行时调度程序（run-time scheduler）在任务执行过程中决定接下来执行哪个任务
    - 完全动态调度程序：在任务运行时制定所有的决策

任务模型（P53）

- 调度程序可能做出很多有关任务的假设，这样的假设集称为调度程序的任务模型
- 假设调度开始之前知道所有要执行的任务；支持任务的到达；支持任务反复执行，可能是无休止的，也可能是周期性的
- 突发性任务：重复出现但时间不规律，任务的两次执行时间间隔有一个下界
- 优先序约束：如果执行 i 必须优先于 j，可以写成 i<j
- 任务执行可以有一些先决条件才开始或恢复执行（如得到互斥锁）

任务的周期（P54）

- 作业（job）：一个任务的每次执行被称为一个作业
- 周期任务：每隔 Ti 个时间单元释放一次的任务，Ti 被称为周期
- 偶发任务：连续释放任务的间隔长度有一个下界；实际应用中偶发任务经常被当作周期任务处理，其周期为相邻任务实例到达时刻的最小时间间隔
- 非周期任务：既不是周期的也不是偶发的任务
- 超周期：周期或偶发任务系统的超周期被定义为每个独立任务周期的最小公倍数

与任务执行相关的时间（P55）

- 图（时间线示意）：释放时间 ri、开始时间 si、抢占、恢复、完成时间 fi、时限 di、执行时间 ei、响应时间 Oi
- 执行时间 ei：任务实际执行的时间；响应时间 Oi：任务从释放到完成的时间

时限约束（P56）

- 硬截止时限：如果错过，系统将失败；具有硬时限的调度称为硬实时调度
- 软截止时限：不需要严格执行的设计决策，最好满足时限要求，但错过了也不算错误；软实时调度

- 也许会试图采取补偿性措施：使用近似数据或切换到特殊安全模式
- 简单的措施避免错误数据扩散：如电话中插入一段寂静，或者干脆置之不理

例子：航天飞机软件错误（P57）

- 航天飞机的首次发射因软件计时错误而推迟
- 主控制系统 PASS 和备用系统 BFS：BFS 与 PASS 同步失败
- 一些对代码的小改动增加了一个延时，从而导致系统错误地计算了启动时间
- 有 1/67 概率会发生时限问题

优先级（P58）

- 假设每个任务都有优先级号，调度程序总是执行优先级最高的任务（通常用最小的优先级号表示）
- 固定优先级：任务整个执行过程中其优先级保持不变
- 动态优先级：在执行过程中任务的优先级是可变的

非抢占式调度与抢占式调度程序（P59）

- 非抢占式优先级调度程序：通过优先级决定当前任务完成之后执行哪个任务，不中断一个正在执行的任务而去执行另外一个
- 抢占式优先级调度程序：任务优先级可静态分配（在任务的持续时间内是常量）或动态分配（可以变化）；内核跟踪哪些任务是“激活的”
- 抢占式调度：在任何时刻支持任务的到达，执行最高优先级的被激活任务；当任何任务改变优先级或激活状态时，内核可以调度一个新任务

调度程序的度量（P60）

- 可行调度、CPU 利用率、调度开销（制定调度决策所需的时间）、延迟、总完成时间

可行调度（P61）

- 调度策略的选择取决于应用程序的目标
- 最简单的目标：所有任务执行都符合其时限（$f_i \le d_i$），满足以上条件的调度称为可行调度
- 对于具有可行调度的每个任务集（符合任务模型）都产生可行调度的调度程序称为可行性最优

利用率（P62）

- 指处理器用于执行任务的时间（对闲置时间）的百分比；适合周期性执行的任务
- 每当处理器的使用率小于或等于 100% 时都能实现可行调度的调度算法无疑是可行性最优的
- 通常假设没有调度开销：U = (CPU time for useful work) / (total available CPU time)

延迟（P63）

- 对于任务集 T，最大延迟定义为：$L_{max} = \max(f_i - d_i)$，其中 f_i 为任务 i 的完成时间

- 对于一个可行调度，值为 0 或负数；也可用于比较非可行调度
- 对于软实时程序，可以容许最大延迟值为正数，只要不是太大

总完成时间/总时程（P64）

- M = max(fi) − min(ri)
- 是性能指标，而非实时要求

嵌入式调度与通用调度（P65）

- 通用操作系统：以提升系统整体资源利用率和系统吞吐量为核心，以公平性为核心设计理念（公平性 = 对 CPU 的访问）
- 嵌入式系统：必须“满足任务的时间约束”，即满足截止期限；低优先级的进程可能很长时间不会运行

调度程序何时被调用（P66）

- 非抢占式调度程序：任务完成时被调用
- 抢占式调度程序：在计时器中断时、在 I/O 中断时、操作系统服务被调用时、任务试图获得互斥锁时、任务测试信号时等

6.3 RMS 与 EDF ⭐【重点】（P68–P80；P81 起优先序约束等内容见后半部分笔记）

单调速率调度 RMS（Rate Monotonic Scheduling）（P69）

- 基于如下假设：n 个周期性任务，周期 T1…Tn（施加实时约束）；所有任务都是独立的；对于所有任务硬截止时限 Di=Ti；最坏执行时间（WCET）C1…Cn；假定没有互斥锁、信号量或阻塞 I/O；没有优先序约束；固定优先级；上下文切换所需的时间可以忽略不计；抢占式调度
- RMS：任务的优先级是周期的单调减函数——周期短的任务拥有高优先级，长周期的任务被分配为低优先级
- 定理：如果一个任务集能够被静态调度，那么 RMS 算法就能够调度这个任务集；RMS 是最优的静态调度算法，RMS 是可行性最优的

RMS CPU 利用率（P70）

- 单个处理器 n 个任务下，利用率为：Usum = Σ(Ci/Ti) ≤ n(2^(1/n) − 1)
- n 更大时，利用率趋于 0.7（即 ln2 ≈ 0.693）

示例（P71）

- 任务参数：T1=5、C1=3；T2=8、C2=3
- Usum = 3/5 + 3/8 = 39/40 = 0.975；而 n(2^(1/n) − 1) 大约为 0.828
- 0.975 > 0.828，不能确保 RMS 调度的可调度性

RMS 的优点（P72）

- 该算法是单处理器系统中最优的固定优先级抢占式调度算法
- 基于静态优先级，从而可以应用于具有固定优先级的操作系统

任务执行分配（临界时刻定理）（P73）

- 临界时刻定理：对于固定优先级调度，如果任务 T1 与所有更高优先级的任务同时释放，那么每个任务在单处理器系统上执行的响应时间就是最大化的
- 当低优先级任务的开始相位与高优先级任务的开始相位一致时，低优先级任务的完成时间是最差的
- 因此检查调度可行性时只考虑最坏的情况就足够了，即所有的任务同时开始周期式执行

最早交货期 EDD（Earliest Due Date）/ Jackson 算法（P74）

- 已知一个有限非重复任务集，截止时限分别为 d1…dn，无优先序约束
- 最大延迟定义为 Lmax = max(fi − di)，其中 fi 是任务 i 的完成时间；如果满足所有截止时限，这个值是负的
- EDD 算法：按照非递减截止时限顺序执行任务，截止时限最早的首先执行；如果两个任务的截止时限相同，则相对顺序无关紧要；不需要抢占

最大延迟最小化（P75-P76）

- 定理：EDD 在最大延迟最小化方面是最优的
- 使用交换论证证明：给定一个不是 EDD 的调度 S，必然有任务 a 和 b，其中 a 在调度中先于 b，但 da > db；可以证明没有一个非 EDD 调度的最大延迟比 EDD 调度小，因此 EDD 调度一定是最优的
- 图：非 EDD 调度 S 中任务 a、b 的交换示意（a 先于 b 但 da > db）

最早截止时限优先 EDF（Earliest Deadline First）/ Horn 算法（P77）

- EDD 是静态优先级调度算法，不支持任务到达，可通过允许任务在任何时刻“到达”（就绪）来扩展 EDD
- EDF：给定 n 个具有任意到达时间的独立任务集，在任何时刻，在所有就绪任务中执行绝对截止时限最早的任务的算法
- 在最大延迟最小化上是最优的（证明使用类似的交换论证）
- EDF 是一种动态优先级调度算法

使用 EDF（P78）

- 该算法既适用于周期性任务，也适用于非周期性任务
- 在周期性任务中使用 EDF 的最简单用法：截止时限是周期的结束
- 备选用法：分别指定截止时限（相对于周期开始时间）和周期

示例（P79）

- 任务参数：T1=5、C1=3；T2=8、C2=3
- 图：EDF 调度时序——CPU Utilization 97.5%、CPU Idle 2.5%、Hyper Period（超周期）40

EDF 与 RMS 比较（P80）

- RMS 优势：调度决策更简单；采用固定优先级（EDF 调度器必须维护一个按优先级排序的就绪任务列表）
- EDF 优势：EDF 在最大延迟最小化方面是最优的，也是可行性最优的；RMS 只是可行性最优，对于不可行的调度，RMS 完全阻塞较低优先级的任务，导致无限的最大延迟；EDF 可以实现 RMS 无法做到的 CPU 利用率（理论上 100%）；EDF 在实践中导致更少的抢占，因此减少了上下文切换的开销；EDF 的截止时限可以和周期不同

第6章 嵌入式操作系统 ⭐【重点】（后半部分，讲义 P81–P159）

6.3 RMS、EDF 及改进（续）⭐【重点】

本章前半部分已介绍 RMS（Rate Monotonic Scheduling，单调速率调度）与 EDF 的基本原理与可调度性分析；以下为优先序约束下的调度改进算法（LDF、EDF*）等内容的延续。

优先序约束（Precedence Constraints）（P81–P83）

- 优先序约束：某些任务必须在其他任务开始之前完成。
- 用**有向无环图（DAG）**表示任务之间的优先序：节点为任务，边表示"必须先完成"的关系（P81）。
- 图例说明：任务 1 必须在任务 2 和任务 3 开始之前完成（P81）。
- 例子（P82–P83）：6 个任务，每个执行时间 Ci=1，截止时限分别为 d1=2、d2=5、d3=4、d4=3、d5=5、d6=6，受 DAG 优先序约束。
- 结论：在优先序约束下**直接使用 EDF 可能是不可行的**——EDF 只按绝对截止时限调度而忽略优先序，导致某些任务违反优先序并错过截止时限（本例中任务 4 错过截止时限）（P83、P95）。

最晚时限优先 LDF（Latest Deadline First）（P84–P91）

- 提出者：Lawler，1973 年（P84）。
- 在**优先序约束下，LDF 是最优的**：调度结果遵守所有优先序并满足所有截止时限（P84）。
- 调度策略**反向构建**：给定 DAG，选择截止时限**最晚**的**叶节点**作为最后调度的任务，然后反向依次确定调度顺序（P85–P90 逐步演示）。
- 性质（P91）：
  - LDF 在**最大延迟最小化**方面是最优的；
  - **不需要抢占**；
  - 局限：要求所有任务在执行前都是已知的，且事先知道全部优先序约束（属离线调度）。

EDF*：修改释放时间与截止时限（P92–P98）

- 思想：对于具有优先序约束的任务调度，**如果存在可行调度，则 EDF* 算法能确定一个可行调度**；通过对任务参数（释放时间、截止时限）进行修改，保证存在有效调度（P92）。
- 修改后应满足（P92–P94）：
  - 任务不早于其释放时间开始执行；
  - 任务不早于其**前序任务的结束时间**开始执行（一个任务不能抢占其任何前序任务）；
  - 所有任务都在规定的截止时限内完成执行。
- **截止时限修改**（P93）：任务执行完成的时间不能晚于其后继任务的最晚开始时间（任务 j 依赖于任务 i 时，收紧 i 的截止时限）。
- **释放时间修改**（P94）：任务开始执行的时间必须不早于其释放时间，且不能早于其前序任务的最小完成时间（任务 j 依赖于任务 i 时，推迟 j 的释放时间）。
- 修改公式（以"任务 i 直接先于任务 j"为例，由讲义例子归纳）：

```
释放时间：r_j' = max(r_j, r_i' + C_i)
截止时限：d_i' = min(d_i, d_j' - C_j)
```

- 例子（P96–P97）：假设 ri=0，修改后（各 C=1）：
  - 释放时间：r1’=0、r2’=1、r3’=1、r4’=2、r5’=2、r6’=2；
  - 截止时限：d1’=1、d2’=2、d3’=4、d4’=3、d5’=5、d6’=6。
  - 使用修订后的释放时间与截止时限，EDF 调度是**最优的**，且满足所有截止时限（P97）。
- 优化结论（P98）：**具有优先序的 EDF 在最大延迟最小化方面是最优的**。

任务集不可调度时怎么办（P99）

若进程组不可调度、又必须保证在截止时限前完成，可选方案：

- **使用更快的 CPU**：不改变周期即可减少执行时间，但 CPU 利用率更低；
- **重新设计进程以减少执行时间**：需要深入了解代码；
- **重写规格说明以改变截止时限**。

（P100 参考文献：Lee & Seshia《嵌入式系统导论：CPS 方法》第 11 章；《嵌入式系统设计：CPS 与物联网应用》第 6 章。）

6.4 调度异常 📌【了解】（P101–P130）

本节主题（P102）：互斥、优先级反转、优先级继承、优先级天花板、多处理器调度、Richard 异常。

互斥与临界区（P103–P104）

- 线程访问共享资源时，需要使用**互斥（mutual exclusion）**来确保数据的完整性；互斥也会使调度复杂化（P103）。
- pthreads 互斥机制：`pthread_mutex_lock()` / `pthread_mutex_unlock()`；跨线程共享的数据访问必须是**原子**的，通过互斥锁实现（P104）。
- **临界区（critical section）**：持有锁时执行的代码（P104）。

优先级反转（Priority Inversion）（P105–P110）

- 现象定义（P105）：**低优先级任务持有高优先级任务所需的共享资源，导致高优先级任务无法执行，并可能被其他中等优先级任务抢先**，实时性难以得到保证。
- 典型场景（P105）：任务 1 优先级最高、任务 3 最低；任务 3 获取共享对象锁进入临界区后被任务 1 抢占；任务 1 试图获取锁而被阻塞；此时任务 2（中等优先级）在时刻 4 抢占任务 3，将高优先级的任务 1 阻塞**任意长**时间——任务 1 与任务 2 的优先级实际上被"颠倒"了。
- 实例：**火星探路者（Mars Pathfinder, MPF）**（P106–P109）：
  - 1997-07-04 着陆火星，最初工作稳定，收集气象数据后不久开始经历**全面系统重置**，每次导致数据丢失（P106）。
  - VxWorks 提供抢占式优先级调度；"信息总线"相当于共享内存区域，用于组件间传递信息；总线管理任务以高优先级频繁运行，对总线的访问用互斥锁同步（P107）。
  - 角色：高优先级——从共享内存检索数据的总线任务；中优先级——通信任务；低优先级——收集气象数据的线程（发布数据时获取互斥锁写总线）（P108）。
  - 故障链条（P109）：高优先级总线线程被阻塞等待低优先级气象线程时，中等优先级通信任务被调度；长时间运行的通信任务优先级高于气象任务，阻止气象任务运行，从而阻止被阻塞的总线任务运行；一段时间后**看门狗计时器**发现总线任务长时间未执行，推断出现严重错误，启动整个系统重置——优先级反转的典型案例。
- 解决（P110）：**优先级继承**解决了火星探路者问题。VxWorks 使用标志位调用互斥原语：标志为 on 时启用优先级继承（软件发布时被置为 off）；通过 VxWorks 调试工具将标志改为 on 予以纠正 [Jones, 1997]。

优先级继承协议 PIP（Priority Inheritance Protocol）（P111–P113）

- 提出：Sha, Rajkumar, Lehoczky, 1990（P111）。
- 假设：n 个任务通过 m 个共享资源协作；固定优先级；一个资源上的所有临界区都以 `wait(Si)` 开始、以 `signal(Si)` 结束（P111）。
- 基本思想：当任务 Ji 阻塞一个或多个高优先级任务时，它**暂时继承被阻塞任务的最高优先级**（P111）。
- 术语（P111）：固定的**名义优先级（nominal priority）Pi** 与大于等于 Pi 的**动态优先级（active priority）**；作业按名义优先级排序（J1 优先级最高）；作业不会自动中止。

- 算法要点（P112）：
  - 作业按动态优先级调度，同优先级按 **FCFS（先来先服务）**执行；
  - Ji 试图进入临界区时，若资源被较低优先级作业独占则阻塞，否则进入；
  - Ji 被阻塞时将其动态优先级**传递给持有该信号量的作业 Jk**，Jk 以 Pk=Pi 的优先级执行其临界区剩余部分（继承被它阻塞的所有作业的最高优先级）；
  - Jk 退出临界区时释放信号量，唤醒阻塞在该信号量上的最高优先级作业；若没有其他作业被 Jk 阻塞，Jk 恢复名义优先级 Pk；
  - **优先级继承是可传递的**：1 被 2 阻塞、2 被 3 阻塞，则 3 通过 2 继承 1 的优先级。
- 例子（P113）：任务 3 获取锁进入临界区后被任务 1 抢占，任务 1 尝试获取锁而阻塞；任务 3 **继承任务 1 的优先级**，从而防止被任务 2 抢占。

死锁（Deadlock）（P114）

- 定义：多个任务并发执行过程中因**资源竞争被相互永久阻塞**。
- 例子：低优先级任务先启动获取锁 a，被高优先级任务抢占；高优先级任务获取锁 b 后运行，尝试获取锁 a 时被阻塞；低优先级任务随后尝试获取锁 b 也被阻塞——两者都不可能再执行下去。
- 代码模式：线程 1 先锁 b 再锁 a、线程 2 先锁 a 再锁 b（锁顺序相反 → 死锁）。

优先级天花板协议 PCP（Priority Ceiling Protocol）（P115–P118）

- 提出：Sha, Rajkumar, Lehoczky, 1990（P115）。
- 规则（P115）：
  - 每个锁或信号量被分配一个**优先级上限（ceiling）**，等于可以锁定它的**最高优先级任务的优先级**；
  - 只有当任务 T 的优先级**严格高于**其他任务当前持有的所有锁的优先级上限时，T 才能获得锁；
  - 未被任何任务持有的锁不影响该任务；
  - 该规则可**防止一些死锁**的发生。
- 两种实现（P116）：
  - **原始优先级上限协议 OCPP**：更高优先级任务 Y 试图获取任务 X 锁定的资源时，X 的优先级被提升到该资源的优先级上限，确保 X 尽快完成其临界区并解锁资源；仅当一个任务的动态优先级高于其他任务锁定的所有资源的优先级上限时才允许其锁定资源，否则阻塞等待。
  - **立即优先级上限协议 ICPP**：任务锁定资源时，其优先级**立即**提升为资源的优先级上限，因此任何可能锁定该资源的任务都无法被调度。
- OCPP 与 ICPP 对比（P117）：

对比项	**OCPP**	**ICPP**
最坏情况调度行为	两种方案最坏情况相同	两种方案最坏情况相同
实现难度	较复杂（需监视阻塞关系）	更易实现（不需监视阻塞关系）
上下文切换	较多	**更少**（阻塞在首次执行之前进行）
优先级传递次数	**更少**（仅在实际发生阻塞时才改变优先级）	更多（发生在所有资源使用中）

- 应用示例（P118）：锁 a、b 的优先级上限都等于任务 1 的优先级；时刻 3 任务 1 试图锁定 b 未果，因为任务 2 正持有锁 a（其优先级上限等于任务 1 的优先级）。

调度的脆弱性与 Richard 异常（P119–P129）

- 脆弱性（P119）：**所有线程调度算法都是脆弱的**——微小的更改可能产生意想不到的严重后果。
- 定理（Richard Graham, 1976）：若一个具有固定优先级、执行时间和优先级约束的任务集在固定数量的处理器上按优先级调度，那么**增加处理器数量、减少执行时间或削弱优先序约束，都可能增加调度时长**（P119）。
- Richard 异常示例：9 个任务（编号较小的任务优先级较高），3 个处理器下基于优先级的调度结果（P120）：
  - **增加处理器数量**（P121）：增加到 4 个处理器，基于优先级的调度反而**执行时间更长**；
  - **减少执行时间**（P124–P125）：将所有计算时间减少 1，也导致更长的执行时间；
  - **弱化优先序约束**（P126–P127）：移除优先序约束 (4,8) 与 (4,7)，仍然导致更长的执行时间。
- 贪心调度（P122–P123）：基于优先级的调度是"贪心的"；更智能的调度程序可以推迟调度任务 5、6 或 7，使处理器空闲一个时间单元，从而缩短总执行时间；但若任务只有在其前序任务完成后才"到达"（调度器才获知），**贪心调度可能是唯一可行的选项**。
- Richard 互斥异常（P128）：任务 2 与任务 4 以独占模式共享同一资源、任务被静态分配给处理器时，任务 1 的执行时间减少，总完成时间反而增加。
- 结论（P129）：在所有已知任务调度策略下，**时限行为都是脆弱的**，小的改变会产生大的（意想不到的）后果；由于执行时间很难预测，这种脆弱性可能导致意外的系统故障；嵌入式软件因与外部物理系统的交互不可避免，对时序效果特别敏感。

6.5 实时调度实例 📌【了解】（P131–P159）

µC/OS-II（P132–P154）

概述与可靠性（P132）：

- μC/OS-II 是**可移植、可固化、可扩展、抢占式、实时确定性**的多任务内核，适用于微处理器、微控制器和 DSP。
- 面向安全关键市场的可靠性认证：航空电子 **DO-178B** 认证；药品 **FDA 510(k)/PMA** 器械；运输和核系统 **SIL3/SIL4 IEC**；99% 符合 **MISRA-C:1998** C 编码标准。

任务主函数（P133）：任务主体为无限循环，循环内调用 μC/OS-II 的服务（OSFlagPend、OSMboxPend、OSMutexPend、OSQPend、OSSemPend、OSTaskSuspend、OSTimeDly、OSTimeDlyHMSM 等）；也可在 USER CODE 后调用 OSTaskDel(OS_PRIO_SELF) 删除自身。

任务优先级（P134）：

- 最多可管理 64 个任务（2.86 版本后支持 256 个任务）；
- **优先级号越小，任务优先级越高**；
- 在 μC/OS-II 中，任务优先级号同时作为任务的标识符。

任务控制块 OS_TCB（P135–P139）：

- OS_TCB 是描述任务的核心数据结构，存放任务堆栈指针、任务状态、优先级、任务链表指针等管理信息；任务一旦建立，OS_TCB 即被赋值（P135）。
- 关键成员：栈指针、OSTCBId（任务 ID）、链表指针、OSTCBEventPtr（事件指针）、OSTCBMsg（消息指针）、OSTCBStat（任务状态）、OSTCBPrio（任务优先级）等（P135）。
- **栈指针**（P136）：OSTCBStkPtr 指向当前任务栈顶；每个任务可有自己的栈，容量任意；OSTCBStkBottom 指向任务栈底；OSTCBStkSize 表示栈容量，用**可容纳的指针数目**而非字节数（Byte）表示。
- **链表指针**（P137）：所有任务控制块分属两条链表——单向**空闲链表**（头指针 OSTCBFreeList）与双向**使用链表**（头指针 OSTCBList）；OSTCBNext/OSTCBPrev 用于将 TCB 插入/删除；任务创建时 TCB 链接入使用链表，删除时移除；双向链表使任一成员都能快速插入或删除。
- **空闲 TCB 链表**（P138）：所有 TCB 放在数组 OSTCBTbl[] 中；系统初始化时全部链接成空闲单向链表；创建任务时把 OSTCBFreeList 指向的 TCB 赋给该任务并加入使用链表，然后 OSTCBFreeList 指向空闲链表的下一个结点。
- 指针数组（P139）：图为 OSTCBPrioTbl[] 指针数组——按优先级号直接索引到对应 TCB 的映射关系。

任务状态转换（P140）：状态转换图描述任务在休眠态、就绪态、运行态、等待/阻塞态、被中断态等状态间的转换，由系统服务调用、事件等待/释放、调度等触发。

任务就绪表（P141–P146）：

- 每个任务的就绪态标志放入就绪表，就绪表含两个变量（P141）：
  - **OSRdyGrp**：任务按优先级分组，8 个任务为一组；OSRdyGrp 的每一位表示该组中是否有进入就绪态的任务；
  - **OSRdyTbl[]**：任务进入就绪态时，相应元素的相应位置 1。

- 结构（P142）：OSRdyGrp 的 8 位对应 8 组（OSRdyTbl[0…7]），组内 8 位对应 8 个优先级号；任务的优先级由 X（组内位号，低 3 位）与 Y（组号，高 3 位）确定；若 OSRdyTbl[i]（0≤i≤7）的某一位为 1，则 OSRdyGrp 的第 i 位为 1；图中标出就绪表中优先级最高任务与优先级最低任务（空闲任务）的位置。
- 置位例子（P143）：优先级 12=1100b → `OSRdyGrp |= 0x02; OSRdyTbl[1] |= 0x10;`；优先级 21=10101b → `OSRdyGrp |= 0x04; OSRdyTbl[2] |= 0x20;`。
- 查表加速（P144）：把 2^n（n=0~7）预计算存入 OSMapTbl[]：OSMapTbl[0]=2^0=0x01（0000 0001）… OSMapTbl[7]=2^7=0x80（1000 0000）。
- 使任务进入就绪态（P145）：

```
OSRdyGrp          |= OSMapTbl[prio >> 3];
OSRdyTbl[prio>>3] |= OSMapTbl[prio & 0x07];
```

  例：优先级 12 → `OSRdyGrp |= OSMapTbl[12>>3](0x02); OSRdyTbl[1] |= 0x10;`
- 使任务脱离就绪态（P146）：先将 OSRdyTbl 相应位清零，仅当该组所有位都为零（组内无就绪任务）时才将 OSRdyGrp 相应位清零：

```
if ((OSRdyTbl[prio>>3] &= ~OSMapTbl[prio & 0x07]) == 0)
    OSRdyGrp &= ~OSMapTbl[prio >> 3];
```

任务的调度（P147–P154）：

- µC/OS-II 是可抢占实时多任务内核，**总是运行就绪任务中优先级最高的那一个**；不支持时间片轮转；每个任务优先级不同且唯一，因此任务调度工作就是**查找处于就绪态的最高优先级任务并进行上下文切换**（P147）。
- **µC/OS-II 任务调度所花时间为常数，与应用程序建立的任务数无关**（P147）。
- 调度器（P148）：确定哪个任务优先级最高、选择哪个任务运行的工作由**调度器（Scheduler）**完成；任务级调度由函数 **OSSched()** 完成，中断级调度由函数 **OSIntExt()** 完成。
- 根据就绪表确定最高优先级的关键（P149–P150）：将优先级数分解为**高 3 位 + 低 3 位**分别确定（64 任务版本）；高优先级任务的优先级号更小。例：OSRdyGrp=0x08=0x00001000 → 第 3 位为 1，优先级高 3 位为 011；OSRdyTbl[3]=0x3a → 最低置位位为第 1 位（低 3 位 001），最高就绪优先级 = 3×8+2−1 = 25。
- OSSched() 代码要点（P151）：先检查 `(OSLockNesting | OSIntNesting) == 0`（确认非中断调用且允许任务调度）；`y = OSUnMapTbl[OSRdyGrp]`；`OSPrioHighRdy = (y<<3) + OSUnMapTbl[OSRdyTbl[y]]`；若 `OSPrioHighRdy != OSPrioCur`，则置 OSTCBHighRdy=OSTCBPrioTbl[OSPrioHighRdy]、OSCtxSwCtr++ 并调用 OS_TASK_SW() 进行上下文切换。
- **查表法**（P152–P153）：查表法具有确定的时间，提升了系统的可预测性，µC/OS 中所有系统调用时间都是确定的（参见 OS_CORE.C）：

```
Y    = OSUnMapTbl[OSRdyGrp];
X    = OSUnMapTbl[OSRdyTbl[Y]];
Prio = (Y << 3) + X;
```

- OSUnMapTbl[256] 例子（P153）：OSRdyGrp=01101000B（0x68）→ OSUnMapTbl[OSRdyGrp]=3（相应于第 3 位置 1）；OSRdyTbl[3]=11100100B（0xE4）→ OSUnMapTbl[OSRdyTbl[3]]=2；最高就绪任务优先级 Prio=3×8+2=26。表中 256 项常量给出参数最低置位位的位号。
- 64 个任务扩展到 256 个任务（P154）：OS_SchedNew() 中，OS_LOWEST_PRIO≤63（v2.85 之前）沿用原 8+8 查表法；否则 OSRdyGrp 为 16 位：先查低字节 `OSRdyGrp & 0xFF`，非 0 则 y=OSUnMapTbl[低字节]，否则 y=OSUnMapTbl[高字节]+8；再取 16 位 OSRdyTbl[y]（ptbl）：低字节非 0 则 OSPrioHighRdy=(y<<4)+OSUnMapTbl[低字节]，否则 (y<<4)+OSUnMapTbl[高字节]+8。

FreeRTOS（P155–P158）

- 支持的调度方法（P155）：**抢占式、协作式、时间片轮转**；高度可配置。
- 任务链表（P156）：xLIST 为链表根节点，可以是就绪链表、阻塞链表、挂起链表等；链表中的节点通过 **pvOwner** 与任务控制块联系，通过 **pvContainer** 挂载到就绪、阻塞、挂起链表上。
- 任务就绪列表（P157）：`List_t pxReadyTasksLists[configMAX_PRIORITIES];`——每个优先级对应一个就绪链表。
- 优先级配置（P158）：
  - 每个任务分配 0 到 configMAX_PRIORITIES−1 的优先级，configMAX_PRIORITIES 在 FreeRTOSConfig.h 中定义；
  - 为保证 RAM 使用效率，应取实际需要的最小值；
  - 若使用 "前导零计数" 类指令优化任务选择机制且 configUSE_PORT_OPTIMISED_TASK_SELECTION 置 1，则 configMAX_PRIORITIES 不能高于 32；其他情况下可取任何合理数值；
  - **优先级数字小表示任务优先级低**；空闲任务的优先级为 0（tskIDLE_PRIORITY）。
- µC/OS-II 与 FreeRTOS 简要对比如下（依讲义要点归纳）：

对比项	**µC/OS-II**	**FreeRTOS**
就绪任务管理	就绪表（OSRdyGrp + OSRdyTbl[]），查表常数时间	每优先级一个就绪链表（pxReadyTasksLists[]）
优先级数量	最多 64（2.86 后 256）	configMAX_PRIORITIES，可配置（取最小以省 RAM）
优先级方向	号越小优先级越高，优先级号兼作任务标识	号越小优先级越低，空闲任务为 0

对比项	**µC/OS-II**	**FreeRTOS**
调度方式	抢占式，不支持时间片轮转	抢占式 / 协作式 / 时间片轮转

Linux 增强实时能力（P159）

- 自 Linux 内核 2.6.23 起，默认进程调度器为**完全公平调度器 CFS（Complete Fair Scheduler）**，取代之前的 O(1) 调度器。
- 实时化常用方法：将 Linux 内核与所有 Linux 用户模式进程都作为**底层实时操作系统的特定任务**，仅在非实时任务需要运行时才激活。
- 传统实时调度策略：**SCHED_FIFO** 与 **SCHED_RR**。
- 硬实时支持：**SCHED_DEADLINE**（集成于 3.14（2014）版本内核中），**基于 EDF**。

第七章 同步、通信与存储管理

任务间的同步与通信机制（📌【了解】）

> 本章"实时系统任务间的同步与通信常用的机制"在复习文件中标记为"-"，不在笔试考察范围，但机制内容仍完整整理供了解（P2）。

同步与通信的基本概念（P2）

- 所有的实时系统本质上都是并发式的（concurrent）。（P2）
- 同步（Synchronization）是指任务之间按照某种规则进行协调和按序执行的过程；通信（Communication）是指任务之间交换信息和共享数据的过程。（P2）
- 常用的同步互斥与通信机制：事件（Event）、消息队列（Message Queue）、信号量（Semaphore）和互斥量（Mutex）。（P2）
- 灵活运用这些机制可实现任务间的有效通信与协作，满足不同的应用需求；RTOS 中这些机制通常是轻量级（lightweight）的实现。（P2）

µC/OS-II 事件控制块 ECB（P3-P10）

- µC/OS-II 把所有通信信号都看成是事件（event），通过事件控制块 ECB（Event Control Block）来管理每一个具体事件。（P3）
- ECB 数据结构（P3）：

```
typedef struct {
    void  *OSEventPtr;                          /* 指向消息或消息队列的指针 */
    INT8U  OSEventTbl[OS_EVENT_TBL_SIZE];       /* 等待任务列表 */
    INT16U OSEventCnt;                          /* 计数器（当事件是信号量时） */
    INT8U  OSEventType;                         /* 事件类型：信号量、邮箱等 */
    INT8U  OSEventGrp;                          /* 等待任务组 */
} OS_EVENT;
```

- ECB 是与 TCB 类似的结构，使用两个链表管理：空闲链表与使用链表。（P3）
- 事件类型由 `OSEventType` 指明：`OS_EVENT_SEM`（信号量）、`OS_EVENT_MUTEX`（互斥信号量）、`OS_EVENT_TYPE_MBOX`（邮箱）、`OS_EVENT_TYPE_Q`（消息队列）。（P4）
- ECB 数据结构图示（P5）：`OSEventTbl[8]` 为等待任务位图（8 组）、`OSEventGrp` 为组位图、`OSEventPtr` 指向消息/队列、`OSEventCnt` 为信号量计数器、`OSEventType` 标识事件类型，由 `pevent` 指针引用。

等待任务列表（**P6-P7**）

- 每个正在等待某个事件的任务被加入到该事件 ECB 的等待任务列表中，列表由 `OSEventGrp` 和 `OSEventTbl[]` 两个变量组成。（P6）
- 任务按优先级分组：8 个任务为一组，共 8 组，分别对应 `OSEventGrp` 中的 8 位；某组中有任务等待该事件时，对应位被置位，同时 `OSEventTbl[]` 中的相应位也被置位。（P6）
- 图示（P7）：任务的优先级 `prio` 的高 3 位（X）决定 `OSEventGrp` 中的位位置及 `OSEventTbl[]` 的数组下标，低 3 位（Y）决定 `OSEventTbl[]` 中相应位的位置；最低优先级任务（空闲任务）不可能处于等待状态，最高优先级任务优先级为 0。

任务进入/脱离等待状态（**P8**）

- 将一个任务插入事件的等待任务列表：`pevent->OSEventGrp |= OSMapTbl[prio >> 3]; pevent->OSEventTbl[prio >> 3] |= OSMapTbl[prio & 0x07];`（P8）
- 从等待任务列表删除一个任务：若 `OSEventTbl[prio >> 3]` 对应位清 0 后整组为 0，则同步清除 `OSEventGrp` 中对应位：`if ((pevent->OSEventTbl[prio >> 3] &= ~OSMapTbl[prio & 0x07]) == 0) { pevent->OSEventGrp &= ~OSMapTbl[prio >> 3]; }`（P8）

查找最高优先级等待任务（**P9**）

- 先查组再查位，两次查表 `OSUnMapTbl`：`y = OSUnMapTbl[pevent->OSEventGrp]; x = OSUnMapTbl[pevent->OSEventTbl[y]]; prio = (y << 3) + x;`（P9）

空闲 ECB 的管理（**P10**）

- ECB 的总数由用户需要的信号量、邮箱和消息队列总数决定，由 `OS_CFG.H` 中的 `OS_MAX_EVENTS` 定义。（P10）

- 调用 `OSInit()` 初始化系统时，所有 ECB 被链接成一个单向链表——空闲事件控制块链表（`OSEventFreeList`）。（P10）
- 每当建立一个信号量、邮箱或消息队列时，就从该链表中取出一个空闲 ECB 并对其初始化。（P10）

信号量 Semaphore（P11-P12）

- 信号量在多任务系统中的功能：①实现对共享资源的互斥访问（包括单个共享资源或多个相同的资源）；②实现任务之间的行为同步。（P11）
- 必须在 `OS_CFG.H` 中将 `OS_SEM_EN` 开关常量置为 1，μC/OS 才能支持信号量。（P11）
- 任务、ISR 和信号量的关系（P12）：任务可调用 `OSSemCreate()`、`OSSemPend()`、`OSSemPost()`、`OSSemAccept()`、`OSSemQuery()`；ISR 中只能调用 `OSSemPost()` 和 `OSSemAccept()`（不可阻塞）。

共享内存（P13）

- 共享内存是比较快速的任务通信机制，会映射到所有相关任务的地址空间。（P13）
- μC/OS-II 中内存地址空间只有一个，为所有任务所共享；为避免竞争状态（race condition），需要互斥访问：互斥锁、信号量等。（P13）

消息邮箱 MailBox（P14-P15）

- 邮箱：一个任务或 ISR 可以通过邮箱向另一个任务发送一个指针型变量，该指针指向一个包含特定"消息"（message）的数据结构。（P14）
- 必须在 `OS_CFG.H` 中将 `OS_MBOX_EN` 开关常量置为 1，μC/OS 才能支持邮箱。（P14）
- 任务、ISR 与消息邮箱的关系（P15）：任务可执行 Pend/Post/Accept/Query 等操作，ISR 只能进行 Post/Accept 等非阻塞操作。

消息队列 Message Queue（P16-P23）

- 消息队列是常用于任务间通信的数据结构：一个任务或 ISR 可以向另一个任务发送多个以指针方式定义的变量，实现任务接收来自其他任务或中断的不固定长度的消息。（P16）
- 队列为空时，读取消息的任务将被阻塞；可指定阻塞时间 timeout：在这段时间内队列仍为空，任务保持阻塞状态等待队列数据有效。（P16）
- 队列中有新消息时，被阻塞的任务被唤醒并处理新消息；等待时间超过指定阻塞时间，任务自动从阻塞态转为就绪态。（P16）
- 消息队列的体系结构（P17）：ECB 通过 `OSEventPtr` 指向队列控制块 OS_Q，OS_Q 再指向应用程序自己分配的指针数组 `MsgTbl[]`（消息缓冲区）。
- 实现消息队列时 ECB 可用的字段：`OSEventPtr`（指向队列控制块）、`OSEventTbl[]`/`OSEventGrp`（等待任务列表）等（P18）。

队列控制块 **OS_Q（P19-P21）**

- 队列控制块数据结构（P19）：

```
typedef struct os_q {
    struct os_q *OSQPtr;    // 空闲队列控制块指针
    void      **OSQStart;   // 指向消息队列的起始地址
    void      **OSQEnd;     // 指向消息队列的结束地址
    void      **OSQIn;      // 指向消息队列中下一个插入消息的位置
    void      **OSQOut;     // 指向消息队列中下一个取出消息的位置
    INT16U      OSQSize;    // 消息队列中总的单元数
    INT16U      OSQEntries; // 消息队列中当前的消息数量
} OS_Q;
```

- 消息队列的数据结构（P20）：ECB 的 `OSEventPtr` 指向 OS_Q，OS_Q 的 `OSQIn`/`OSQOut` 在应用程序分配的指针数组 `MsgTbl[]`（环形）中移动，`OSQSize` 与 `OSQEntries` 记录容量与当前消息数。
- 空闲队列控制块的管理（P21）：每个消息队列用一个队列控制块；总数由 `OS_CFG.H` 中的 `OS_MAX_QS` 定义；系统初始化时所有队列控制块被链接成单向链表——空闲队列控制块链表 `OSQFreeList`。
- 消息缓冲区支持两种模式（P22）：FIFO（先进先出）与 LIFO（后进先出）。队列控制块与事件控制块的关系见图（P23）。

µC/OS-III 事件 Event（P24-P25）

- µC/OS-III 中事件仅用于同步，不提供数据传输功能。（P24）
- 事件只与任务相关联，事件相互独立；一个 32 位（数据宽度由用户定义）的事件集合标识该任务发生的事件类型，每一位表示一种事件类型（0 表示未发生、1 表示已发生），共 32 种事件类型。（P24）
- 典型应用：①任务等待多个传感器数据就绪（如温度、湿度同时达标）；②ISR 触发多个事件，唤醒对应处理任务。（P24）
- µC/OS-III 事件控制块数据结构 `os_flag_grp`（P25）：通用成员 `Type`、`NamePtr`、`PendList`（挂起任务列表，记录阻塞在此事件上的任务）；特定成员 `Flags`（标志位）与 `TS`（事件戳）。

µC/OS-III 信号量（P26-P27）

- µC/OS-III 信号量是实现任务间通信的机制，可实现任务间同步或临界资源的互斥访问，常用于协助一组相互竞争的任务访问临界资源。（P26）
- 典型应用：①保护串口、打印机等独占资源（作互斥锁）；②生产者-消费者模型中控制缓冲区读写（作计数信号量）。（P26）

- 信号量控制块 `os_sem`（P27）：通用成员 `Type`、`NamePtr`、`PendList`；特定成员 `Ctr`（计数器）与 `TS`（事件戳）。

μC/OS-III 消息队列（P28-P31）

- μC/OS-III 消息队列的运作过程见图（P28）：任务/ISR 发送消息入队，等待任务从队列取出消息。
- 消息队列结构 `os_q`（P29）：通用成员 `Type`、`NamePtr`、`PendList`；特定成员 `MsgQ`（类型 `OS_MSG_Q`）。
- `OS_MSG_Q` 成员（P29）：`InPtr`（要插入队列的下一个 OS_MSG 指针）、`OutPtr`（提取的下一个 OS_MSG 指针）、`NbrEntriesSize`（队列允许的最大消息个数）、`NbrEntries`（当前消息个数）、`NbrEntriesMax`（消息个数峰值）。
- 消息取用支持两种模式：先进先出 FIFO（P30）与后进先出 LIFO（P31）。

FreeRTOS 消息队列（P32-P33）

- FreeRTOS 队列通常采用 FIFO 的存储缓冲机制，也提供 LIFO 的存储缓冲机制；支持值传递，也支持引用传递。（P32）
- 消息队列创建完成示意图（P33）：队列结构含 `pcHead`（起始地址）、`pcTail`（结束地址）、`pcWriteTo`（写指针）、`pcReadFrom`（读指针）、`uxMessagesWaiting`（等待中的消息数）、`uxLength`（队列长度）、`uxItemSize`（单个消息大小）、`xTasksWaitingToSend`/`xTasksWaitingToReceive`（等待发送/接收的任务链表）、`xRxLock`/`xTxLock`（锁状态，UNLOCKED）。

存储管理

动态内存管理缺陷（P36）

- 标准 C 库的 `malloc()` 和 `free()` 可以实现动态内存管理，但存在如下缺陷：（P36）
  - 具有不确定性：每次调用的时间和开销可能不同；
  - 会产生内存碎片；
  - 通常不具备线程安全特性；
  - 在小型嵌入式系统中可能不可用或效率不高；
  - 具体实现可能相对较大，占用较多代码空间；
  - 会使得链接器配置复杂。

内存碎片化（P37）

- ANSI C 中用 `malloc()`/`free()` 动态分配/释放内存，在嵌入式实时操作系统中容易产生碎片。（P37）
- 图示（P37）：堆初始状态有 A、B、C 三个 1Kb 块被 3 个任务占用；释放 A、C 后，虽然剩余空闲总量为 3Kb（两个 1Kb 块），但空间不连续（中间隔 B 的 1Kb），无法满足大于 1Kb 的连续分配请求——即产生碎片。

内存管理方法（P38）

- **静态内存分配（Static Memory Allocation）**：允许用户在编译时为任务和内核对象（如队列、信号量等）分配静态内存；可保证设备的可靠性，但需要考虑内存上限，内存使用效率低。（P38）
- **动态内存分配（Dynamic Memory Allocation）**：提供用于动态内存分配的内置函数，允许任务在运行时请求和释放内存；对需要灵活管理内存的应用非常有用，但需小心避免内存泄漏和碎片化。（P38）
- **内存池（Memory Pools）**：在系统初始化时创建的一块内存区域，用于存储固定大小的内存块；任务可以从内存池申请内存块，使用完毕后归还给内存池，有助于减少内存碎片化。（P38）

µC/OS-II 实模式存储管理（P39）

- µC/OS-II 采用实模式存储管理：不划分内核空间和用户空间，整个系统只有一个地址空间（物理内存空间），应用程序和内核程序都能直接访问所有内存单元。（P39）
- 系统中的"任务"实际上都是线程（thread）：只有运行上下文和栈是独享的，其他资源都是共享的。（P39）

µC/OS-II 固定分区存储管理（P40-P41）

- µC/OS 采用固定分区的存储管理方法：把连续的大块内存按分区管理，每个分区包含整数个大小相同的块；一个系统中可以有多个内存分区。（P40）
- 应用程序可从不同内存分区得到不同大小的内存块，但特定的内存块在释放时必须重新放回它原来所属的内存分区。（P40）
- 采用该算法后，内存碎片问题得到解决。（P40）
- 内存分区示意图（P41）：分区 1~4，每个分区由多个大小相同的内存块组成。

内存控制块 MCB（P42-P43）

- 为便于管理，µC/OS 使用内存控制块 MCB（Memory Control Block）跟踪每一个内存分区，系统中每个内存分区都有自己的 MCB。（P42）
- MCB 数据结构（P42）：

```
typedef struct {
    void  *OSMemAddr;      /* 分区起始地址 */
    void  *OSMemFreeList;  /* 下一个空闲内存块 */
    INT32U OSMemBlkSize;   /* 内存块的大小 */
    INT32U OSMemNBlks;     /* 内存块数量 */
    INT32U OSMemNFree;     /* 空闲内存块数量 */
} OS_MEM;
```

- 内存管理初始化（P43）：在 `OS_CFG.H` 中将开关量 `OS_MEM_EN` 置为 1，μC/OS-II 在系统初始化 `OSInit()` 时调用 `OSMemInit()` 对内存管理器进行初始化，建立空闲的内存控制块链表（数量由 `OS_MAX_MEM_PART` 决定）。

μC/OS 内存管理 API（P44-P47）

- **创建内存分区** `OSMemCreate()`（P44）：例如 `OS_MEM *CommTxBuf; INT8U CommTxPart[100][32]; CommTxBuf = OSMemCreate(CommTxPart, 100, 32, &err);`；参数为 `addr`（分区起始地址）、`nblks`（分区内内存块数）、`blksize`（每块字节数）、`err`（错误码指针）。
- `OSMemCreate()` 的工作（P45）：①从系统空闲内存控制块中取得一个 MCB；②将该分区所有内存块链接成单向链表；③在对应 MCB 中填写相应信息。
- **分配内存块** `OSMemGet()`（P46）：`void *OSMemGet(OS_MEM *pmem, INT8U *err);`，从已建立的内存分区申请一个内存块；`pmem` 来自 `OSMemCreate()` 返回值；如果没有空闲块可用则返回 NULL 指针；应用程序必须知道内存块大小，使用时不能超过该容量。
- **释放内存块** `OSMemPut()`（P47）：`INT8U OSMemPut(OS_MEM *pmem, void *pblk);`，将内存块释放并放回相应内存分区；注意：`OSMemPut()` 并不知道一个内存块属于哪个分区，用户应用程序必须确认将内存块放回到正确的分区中。

μC/OS-III 内存池（P48-P49）

- 内存池（Memory Pool）是用于分配大量大小相同的内存对象的技术，可极大加快内存分配/释放的速度。（P48）
- 编译时编译器静态划分一个大数组作为系统内存池，初始化时将其分成大小相等的多个内存块，内存块直接通过链表连接起来（该链表也称为空闲内存块列表）。（P48）
- 内存池一旦创建完成，其内部的内存块大小将不能再做调整。（P48）
- 内存管理运作机制见图（P49）：内存池、空闲内存块链表、分配/释放过程示意。

FreeRTOS 内存管理（P50-P62）

- FreeRTOS 将内存分配保留在可移植层（portable layer），使不同应用程序可以提供适合自身的具体实现。（P50）
- 动态分配不调用 `malloc()`，而是调用 `pvPortMalloc()`；释放 RAM 时内核调用 `vPortFree()` 而不是 `free()`。（P50）

- `void *pvPortMalloc(size_t xWantedSize);` // 分配内存，失败返回 NULL
- `void vPortFree(void *pv);` // 释放内存

- 五种内存分配方案（P51）：

方案	特点	说明
heap_1	最简单，不允许释放内存	确定性；FreeRTOS 加入静态分配支持后不太有用
heap_2	允许释放，但不合并相邻空闲块	视为旧版，较新的 heap_4 是首选
heap_3	简单包装标准 `malloc()`/`free()`	仅保证线程安全
heap_4	合并相邻空闲块以避免碎片化	首选方案
heap_5	如同 heap_4，但可跨越多个不相邻内存区域	支持多段不连续内存

heap_1.c（P52）

- 实现非常基本的 `pvPortMalloc()` 版本，未实现 `vPortFree()`。（P52）
- 若应用程序不需要删除任务、队列或信号量，则适合使用 heap_1，且具有确定性（deterministic）。（P52）
- 将 FreeRTOS 内存堆看成一个简单数组，调用 `pvPortMalloc()` 时将数组细分成为更小的内存块；数组大小在 `FreeRTOSConfig.h` 中由 `configTOTAL_HEAP_SIZE` 定义。（P52）

heap_2.c（P53）

- 采用最佳匹配算法（Best fit）分配内存，并支持内存释放：保证 `pvPortMalloc()` 使用最接近请求大小的空闲块。例如空闲块为 5、25、100 字节，请求 20 字节时选用 25 字节块。（P53）
- 不会把相邻空闲块合并成更大的块，因此会产生内存碎片；若分配和释放的总是相同大小的内存块，碎片不会成为问题。（P53）
- 适合重复创建与删除相同大小任务的应用程序；具有不确定性，但仍比标准 C 的 `malloc()`/`free()` 效率高。（P53）

heap_3.c（P55）

- 对标准 C 库 `malloc()`/`free()` 的简单包装器，仅使其线程安全。（P55）
- 需要编译器提供内存堆，且编译器库要提供 `malloc()`/`free()` 函数；具有不确定性，并会增加代码量。（P55）

heap_4.c（P56-P61）

- 两个特点：①分配时使用首次适应算法（First fit algorithm），从找到的第一个满足要求的空闲块上分配所需内存；②释放内存时将相邻的空闲块合并成较大的空闲块，减少内存碎片，降低多次分配/释放后无法成功分配内存的风险。（P56）

- heap_4 执行时间不确定，但效率高于标准库 `malloc()`/`free()`。（P56）
- 使用链表连接空闲内存块地址，相邻空闲块可合并；释放时遍历链表找到当前块的前一个和后一个空闲块，若地址重叠（相邻）则判定合并。（P56）
- 图示说明（P57-P61）：空闲块链表结构；初始化完成的内存空间（P58）；申请一块内存空间（P59）；前向合并内存块（P60）；后向合并内存块（P61）。

heap_5.c（P62）

- 与 heap_4 使用相同的内存合并算法，但允许堆跨越多个不相邻（非连续）内存区域。（P62）
- 通过链表将多个不连续的内存块串联起来，再使用 heap_4 的算法进行分配和释放。（P62）

Linux 实时内核（RT_PREEMPT）环境中使用 malloc（P63）

- 核心目标是避免内存分配的不确定性延迟，确保实时任务的时间确定性。（P63）
- 使用实时友好的 C 库：普通 glibc 的 `malloc` 为通用性设计，可能存在锁竞争、延迟波动等问题，不适合实时场景，推荐替换为（P63）：
  - musl-libc：轻量，线程安全设计更简单，`malloc` 实现避免了复杂的锁机制，延迟更低；
  - newlib：嵌入式常用库，`malloc` 可配置为使用预分配内存池，减少系统调用依赖。
- 其他做法（P63）：预分配内存池，避免运行时 malloc；锁定内存、禁止页面交换，减少访问时可能出现的毫秒级延迟；避免内存碎片（使用固定大小内存块）；禁止运行时 free（若内存资源充足，可在实时任务结束后统一释放，避免运行时 free 的碎片整理开销）。

第八章 嵌入式系统建模

建模、设计、分析（📌【了解】）

> “建模、设计、分析"在复习文件中标记为”-"，不在笔试考察范围，仍整理供了解（P2）。

- 建模（Modeling）：通过模拟加深对系统理解的过程；模型是对系统的模拟，并反映系统的特性；模型指定了系统的功能。（P2）
- 设计（Design）：创建系统的结构，指定系统如何完成其功能。（P2）
- 分析（Analysis）：通过剖析获得对系统更深入理解的过程，具体说明系统为什么能够完成其功能（或者无法完成模型认为其能完成的功能）。（P2）

模型与特征（正常掌握）

模型的定义（P3）

- 模型是对所研究的系统、过程或概念的一种表达形式。（P3）

- 使用模型的目的是给出系统的抽象视图；每个模型都表示一组对象以及这些对象之间的相互关系。（P3）
- 例如：数学模型是以一组定义和数学公式的形式建立的模型。（P3）

模型应具备的基本特征（P4）

- 简单；经得起理论检验；高表现力；简洁的表示可以提高生产率。（P4）
- 提供逻辑推理能力。（P4）
- 可执行：支持仿真/验证。（P4）
- 可综合。（P4）
- 通常要求设计正交性，能够适应各种不同的任务。（P4）
- 描述不存在歧义，易于理解、易于修改。（P4）

常见的系统建模技术（📌【了解】）（P5）

> “常见的建模技术"在复习文件中标记为”-"，不在笔试考察范围，仍整理供了解。

- 面向状态的建模：有限状态机（Finite State Machines，FSM）。（P5）
- 面向活动的建模：数据流图/过程模型，把系统描绘成一组与数据或者执行的相关性有关的活动的集合。（P5）
- 面向结构的模型：框图，着重强调系统的物理构成。（P5）
- 面向数据的模型：实体-关系图。（P5）
- 异构模型：综合前四种模型的特征，表达复杂系统的多种不同视图。（P5）

基于模型的设计及例子（📌【了解】）

> “基于模型的设计及例子"在复习文件中标记为”-"，不在笔试考察范围，仍整理供了解。

基于模型的设计（P6）

- 创建嵌入式系统各个部分的数学模型：物理世界、控制系统、软件环境、硬件平台、网络、传感器和执行器。（P6）
- 从模型构造实现；目标：构造自动化，就像编译器一样；实际上只有部分是自动构造的。（P6）

嵌入式系统建模实例（P7）

- 系统动态的抽象模型：动态是时间上的演化，即状态如何改变。（P7）
- 例子：

- 建模物理现象：ODE 常微分方程。（P7）
- 反馈控制系统：时域建模。（P7）
- 模态行为建模：FSMs、混合自动机（Hybrid Automata）。（P7）
- 传感器和执行器建模：标定、噪声。（P7）
- 软件建模：并发、实时模型。（P7）
- 网络建模：延迟、错误率、丢包。（P7）

嵌入式系统模型的用途（📌【了解】）（P8）

> “嵌入式系统模型的用途"在复习文件中标记为”-"，不在笔试考察范围，仍整理供了解。

- 通过使用现代建模软件工具，可以离线仿真的方式进行设计和执行初始验证。（P8）
- 模型可以作为所有后续开发阶段的基础。（P8）
- 建模（涉及硬件原型设计）将降低出错风险，通过在整个开发过程中执行验证和确认测试来缩短开发周期。（P8）
- 以系统模型为基础，可以更快、更可靠地进行设计评估和预测。（P8）
- 迭代方法可以在性能和可靠性方面改进设计。（P8）
- 由于模型的可重用性以及对物理原型依赖的减少，降低了资源成本。（P8）
- 通过使用代码自动生成技术，可以减少开发错误和开销。（P8）

嵌入式软件建模（P9）

- 对于工程和设计任务（特别是与嵌入式系统相关的任务），通常使用某种形式的软件建模作为切入或制定应用程序整体设计的最初方法。（P9）
- 软件模型在格式、详细程度和功能方面显著不同：有些软件模型是行为的；有些只是直观地帮助理解和构建；另一些更多地用作框架，以确保类似应用程序之间的一致性，或促进工程师团队之间的沟通。（P9）
- 嵌入式系统设计人员面临的挑战：要知道哪种类型和级别的模型最适合当前的具体情况和问题。（P9）

模型、语言与工具（P10）

- 计算模型（Model of Computation）用于捕获系统行为，包含三要素：一组对象、组合的规则、执行语义。（P10）
- 语言（Language）定义了描述计算模型的语法。（P10）
- 工具（Tool）是将一种语言描述的模型转换为另一种语言描述的模型的"编译器"（Model M1/L1 → Tool → Model M2/L2，两者都承载行为 Behavior）。（P10）

建模语言（正常掌握）

建模语言概述（P11）

- 与编程语言一样，建模语言有明确的定义和标准语法，用于表示结构和功能参与者及其随时间变化的主要关系。（P11）
- 建模语言有多种形式：图形、文本；面向文档、仿真或执行；专注于体系架构层面内容、实现层面内容。（P11）

建模语言的几个关键方面（P12）

- （P12）此页为"建模语言的几个关键方面"示意图，讲义未给出文字要点，展示建模语言在设计时需综合考量的若干维度。

一个典型的嵌入式控制系统模式（P13）

- （P13）图示：典型的嵌入式控制系统由传感器感知被控对象/物理过程、控制器计算控制量、执行器驱动等环节构成，并形成反馈回路。

建模语言例子（P14-P19）

- （P14）建模语言例子总览图，后续各页给出具体例子。
- C 代码：文本、可执行、实现层面的语言（P15）。示例为 PID 控制的循环：每次等待 dt 后计算偏差 `error = setpoint - process_feedback`，累加积分 `integral += error*dt`，计算微分 `derivative = (error - previous_error)/dt`，输出 `output = Kp*error + Ki*integral + Kd*derivative`，更新 previous_error 后跳回循环。（P15）
- 基于时间的仿真：图形、实现、仿真工具。Simulink 是 MATLAB 最重要的组件之一，提供动态系统建模、仿真和综合分析的集成环境。（P16）
- 状态图（Statechart）：图形化、架构和实现工具。（P17）
- URDF（United Robotics Description Format，统一机器人描述格式）：XML 语法框架下用来描述机器人的语言格式（例：Tinymal-B）。（P18）
- STEP（STandard Exchange of Product data model，产品模型数据交换标准）：用于描述整机结构（例：智元灵犀 X1 开源人形机器人）。（P19）

V 图描述嵌入式建模开发周期 📌【了解】（P20）

- V 图（V-model）通常用于描述一个嵌入式建模开发周期。（P20）
- 图形要点：左侧为自上而下的建模/设计分支（需求→系统设计→模块设计→实现），右侧为自下而上的验证分支（单元测试→集成测试→系统测试/验收），底部为编码实现；模型驱动的开发在各阶段以模型为验证依据。
- 相关术语（P20）：

- MIL（model in the loop，模型在环）。（P20）
- HIL（hardware-in-the-loop，硬件在环）。（P20）
- 快速控制原型（rapid control prototype，RCP）。（P20）

何时为嵌入式系统建立模型 📌【了解】（P21）

四种情形（P21）：

1. 任务和安全关键型应用。（P21）
2. 高度复杂的应用程序和系统。（P21）
3. 大型开发团队。（P21）
4. 没有其他选择（当没有原型时）。（P21）

任务和安全关键型应用程序（P22）

- 任务关键特指故障可能导致巨额损失、对企业声誉造成伤害或严重损坏。（P22）
- 对此类系统而言，软件建模、软件工程最优方法和正式行业标准等都要结合起来，以促进和保障嵌入式系统的安全和可靠。（P22）

高度复杂的应用程序和系统（P23）

- 嵌入式设计复杂性演变：1960 年代喷气式战斗机约 5 万行代码 → 现代汽车 10 亿行代码。（P23）
- 现代汽车动力传动控制系统受益于建模技术，因为需要不断调整以满足各种要求：控制废气排放、日益严格的标准、更好的燃油经济性、满足客户要求提供卓越的性能和舒适度等。（P23）
- 上述目标既相互关联又相互冲突：例如稀燃技术可以大大降低燃油消耗，但也降低了三元催化转换效率，造成更多的空气污染。（P23）

大型开发团队需要建模（P24）

- 庞大开发团队面临的关键问题：通信问题；地域分散和语言不同；工具差异。（P24）
- 建模的帮助：帮助不同团队明确沟通（设计一个有用、能准确表示项目动态的系统模型，方便沟通）；提供项目一致性的表达（而不是依赖翻译书面文档）；软件模型可以在工具之间共享。（P24）

建模通常是唯一的选择（P25）

- 当嵌入式系统不存在时。（P25）
- 当设计的硬件尚未完成或芯片尚未准备好（尚未发布的芯片）时，建模、模拟、仿真和原型机设计都是非常有用的方法。（P25）
- 因为项目大小或成本的关系不能开发原型机时，软件模型可以很有帮助。（P25）

建模实例（反馈控制、开环/闭环控制、PID 算法）（正常掌握）

反馈（Feedback）（P28）

- 反馈是控制论的基本概念。（P28）
- 反馈控制：将系统的输出信息返送到输入端，与输入信息进行比较，并利用二者的偏差进行控制的过程。（P28）
- 反馈控制其实是用过去的情况来指导现在和将来。（P28）
- 负反馈：返回信息的作用是抵消输入信息，负反馈可以使系统趋于稳定；正反馈：作用是增强输入信息，正反馈可以使信号得到加强。（P28）

开环控制（Open-loop Control）（P29-P30）

- 开环系统即无反馈系统。（P29）
- Open loop control（开环控制系统）：系统的输入信号不受输出信号影响的控制系统，即不将控制的结果反馈回来影响当前控制的系统。（P30）
- 若组成系统的元件特性和参数值比较稳定，且外界干扰较小，开环控制能够保持一定的精度。（P30）
- 缺点：精度通常较低、无自动纠偏能力。（P30）
- 典型应用：自动售货机、自动洗衣机、产品自动生产线、自动门。（P30）

闭环控制（Closed-loop Control）（P31-P32）

- 闭环控制：作为被控的输出量以一定方式返回到作为控制的输入端，并对输入端施加控制影响的一种控制关系，即带有反馈信息的系统控制方式。（P31）
- 优势：精确、自适应调节、稳定性高。（P31）
- 应用：温控、无人机飞控、电机控制……恒温加热器、太阳能系统、导弹发射器、汽车发动机、自动烤面包机、使用涡轮的水控制系统。（P31）
- （P32）图示：给定值→控制器→被控对象→输出，输出经传感器反馈回输入端与给定值比较，形成闭环回路。

开环与闭环对比（P29-P32）

对比项	开环控制	闭环控制
反馈	无反馈，输入不受输出影响	有反馈，输出量返送到输入端
精度	通常较低	精确
纠偏能力	无自动纠偏能力	自适应调节、稳定性高

对比项	开环控制	闭环控制
典型应用	自动售货机、自动洗衣机、产品自动生产线、自动门	温控、无人机飞控、电机控制、恒温加热器、太阳能系统等

PID 控制器（P33）

- 在过程控制中，按偏差的比例（P）、积分（I）和微分（D）进行控制的 PID 控制器（亦称 PID 调节器）是应用最为广泛的一种自动控制器。（P33）
- 优点：原理简单、易于实现；适用面广；控制参数相互独立、参数整定方式简便；结构改变灵活（PI、PD、…）。（P33）
- 适用场合：当被控对象的结构和参数不能完全掌握，或得不到精确的数学模型时，控制理论的其它技术难以采用时，系统控制器的结构和参数必须依靠经验和现场调试来确定，这时应用 PID 控制技术最为方便；即当我们不完全了解一个系统和被控对象，或不能通过有效的测量手段来获得系统参数时，最适合用 PID 控制技术。（P33）

连续控制系统的理想 PID 控制（P34）

- 理想连续 PID 表达式：$u(t) = Kp \cdot e(t) + Ki \cdot \int e(t)dt + Kd \cdot de(t)/dt$，即 $u(t) = Kp[e(t) + (1/Ti)\int e(t)dt + Td \cdot de(t)/dt]$。（P34，讲义为公式图）
- 式中：Kp——比例增益；Ki——积分增益；Kd——微分增益；Ti——积分时间常数；Td——微分时间常数；u(t)——PID 控制器的输出信号；e(t)——给定值 r(t) 与测量值之差。（P34）

离散化 PID 算法（P35）

- 数字 PID 控制算法在实际应用中可分为两种：位置式 PID 控制算法、增量式 PID 控制算法。（P35）
- 控制理论上两者是相同的，但在数字量化后的实现上会存在差别。（P35）

位置式数字 PID 控制（P36）

- 对连续 PID 作离散化处理：以一系列采样时刻点 k 代表连续时间 t，以矩形法数值积分近似代替积分，以一阶后向差分近似代替微分，得到 k 采样时刻的离散 PID 表达式。（P36）
- 位置式输出 u(k) 由三项组成：比例项 $Kp \cdot e(k)$、积分项（对偏差的累加求和，$Ki \cdot \Sigma e(j)$）、微分项（一阶后向差分，$Kd \cdot (e(k) - e(k-1))$）。（P36）
- 参数：Ki=Kp/Ti，Kd=Kp*Td；T 为采样周期，k 为采样序号（k=1,2,…），e(k−1) 和 e(k) 分别为第（k−1）和第 k 时刻的系统偏差信号。（P36）

增量式 PID 控制（P37）

- 增量式 PID 指控制器的输出是控制量的增量 $\Delta u(k)$；当执行机构需要的是控制量的增量而不是位置量的绝对数值时，可以使用增量式 PID 控制算法进行控制。（P37）

- 增量可由相邻采样时刻的位置式输出之差得到：$\Delta u(k) = u(k) - u(k-1) = Kp[e(k)-e(k-1)] + Ki \cdot e(k) + Kd[e(k)-2e(k-1)+e(k-2)]$。（P37）
- 参数含义同位置式：Ki=Kp/Ti，Kd=Kp*Td，T 为采样周期，e(k−1)、e(k) 分别为第（k−1）和第 k 时刻的系统偏差信号。（P37）

PID 控制器参数整定的一般方法（P38）

- PID 控制器的参数整定是控制系统设计的核心内容；被控过程的特性决定 PID 控制器的比例系数、积分时间和微分时间的大小。（P38）
- 两大类方法：
  - 理论计算整定法：依据系统的数学模型，经过理论计算确定控制器参数；所得到的计算数据未必可以直接用，还必须通过工程实际进行调整和修改。（P38）
  - 工程整定方法：主要依赖工程经验，直接在控制系统的试验中进行，方法简单、易于掌握，在工程实际中被广泛采用；主要有试凑法、临界比例法和衰减曲线法等。（P38）
- 不同方法各有特点，共同点都是通过试验，然后按照工程经验公式对控制器参数进行整定；但无论采用哪一种方法所得到的控制器参数，都需要在实际运行中进行最后的调整与完善。（P38）

齐格勒-尼科尔斯方法（Ziegler–Nichols method）（P39）

- 调试方式：首先将积分和微分增益设置为 0，然后比例增益从零开始逐渐增加，直到到达极限增益 Ku，此时控制器输出值以恒定值振荡。（P39）
- 根据 Ku 和振荡周期 Tu，按不同的控制器类型（P/PI/PD/PID 等），按下表方式设置比例、积分和微分增益。（P39，讲义为表格图）

临界震荡与衰减曲线（P40）

- （P40）临界比例法利用临界震荡示意图：系统在临界等幅振荡时读取极限增益与振荡周期进行整定。
- （P40）衰减曲线法利用 4:1 衰减响应示意图：以 4:1 衰减比下的响应曲线整定参数。

参数整定常用口诀（P41）

- 参数整定找最佳，从小到大顺序查。（P41）
- 先是比例后积分，最后再把微分加。（P41）
- 曲线振荡很频繁，比例度盘要放大。（P41）
- 曲线漂浮绕大湾，比例度盘往小扳。（P41）
- 曲线偏离回复慢，积分时间往下降。（P41）
- 曲线波动周期长，积分时间再加长。（P41）
- 曲线振荡频率快，先把微分降下来。（P41）

- 动差大来波动慢，微分时间应加长。（P41）
- 理想曲线两个波，前高后低 4 比 1。（P41）
- 一看二调多分析，调节质量不会低。（P41）

应用：自动驾驶汽车速度控制（P42）

- 应用场景：自动驾驶汽车需按设定速度（如 60km/h）行驶，应对路况变化（如上坡、顺风）时保持速度稳定。（P42）
- PID 控制逻辑：车速传感器实时反馈实际速度，与目标速度对比产生偏差；PID 控制器调节油门开度（或刹车力度）。（P42）
- 上坡时速度下降（e(t)>0）：P 环节加大油门，D 环节预测速度继续下降趋势、提前增加供油。（P42）
- 下坡时速度上升（e(t)<0）：P 环节轻微刹车，I 环节消除持续下坡导致的微小超速（如长期偏差 0.5km/h）。（P42）
- 回路示意（P42）：[汽车]→车速传感器→偏差 e(t)=60km/h−实际速度→[PID控制器]←反馈；PID 输出调节油门/刹车→[汽车]。速度曲线围绕 60km/h 波动，上坡时短暂降至 58km/h 后快速回升，无长期偏离。

应用：列车精准停车控制（P43）

- 应用场景：高铁或地铁需精准停靠在站台指定位置（误差 < 30cm），避免乘客上下车不便。（P43）
- PID 控制逻辑：位置传感器（如 RFID 标签）检测列车与站台的距离，与目标停车位置对比产生偏差；PID 控制器调节牵引力/制动力。（P43）
- 距离较远时（e(t) 大）：P 环节提供较大牵引力加速。（P43）
- 接近站台时（e(t) 小）：D 环节提前施加制动（预测惯性滑行距离），I 环节消除最后 10cm 的微小偏差。（P43）
- 回路示意（P43）：[列车]→位置传感器→偏差 e(t)=目标位置−实际位置→[PID控制器]；输出调节牵引力/制动力→[列车]。列车速度逐渐降低，最终精准停在站台标线处，无冲过或未到的情况。

扩展阅读：模型预测控制 MPC（P44-P47）

- MPC（Model Predictive Control，模型预测控制）基本思想：通过建立系统的动态模型，在每一个控制时刻用该模型预测系统未来的行为；基于预测生成一个优化控制序列，执行第一个控制动作来调整系统状态，下一时刻重新计算和执行；过程反复进行，使系统在未来一段时间内优化一个特定的性能指标。（P44）
- MPC 的四个基本步骤（P44）：

1. 系统模型化：建立描述系统动态行为的数学模型，通常是差分方程或微分方程。（P44）

2. 预测：在当前时刻基于系统状态和控制输入，用模型预测未来一段时间内的系统响应。（P44）
3. 优化：基于预测的系统响应，通过求解一个优化问题计算最优的控制输入序列，以最大化或最小化一个性能指标（如系统响应时间、能耗等）。（P44）
4. 执行：执行优化得到的控制输入序列中的第一个值，并将实际的系统状态反馈到下一个控制周期中。（P44）

- 滚动优化（receding horizon control，P45）：每个控制时刻都基于最新状态重新预测并优化，优化时域随当前时刻向前滚动。
- （P46）比较图：讲义对 MPC 与 PID 等控制方法进行对比（图示）。
- ACADO Toolkit（P47）：用 C++ 编写的用于自动控制和动态优化的软件环境和算法集合；提供通用框架，使用多种算法进行直接优化控制，包括模型预测控制以及状态和参数估计；作为独立 C++ 代码实现，带用户友好的 MATLAB 界面；面向对象设计，方便耦合现有优化包并使用用户编写的优化例程扩展。

扩展话题：自动紧急制动系统（AEB）（P48-P65）

引言与意义（P48-P49）

- 讲师对行业乱象的评价：连 AEB 是什么，居然有车企的一把手还根本没有搞懂；友商讲了 AEB，认为 99% 是假的、就是造假。（P48）
- AEB 已经成为主动安全领域研究的热点。（P49）
- 94% 的严重撞车事故的主要原因是驾驶员本身的失误（美国国家公路交通安全管理局 NHTSA 数据）。（P49）
- Euro NCAP 研究表明：AEB 可以避免 27% 的碰撞事故；AEB 能够很大程度避免或缓解追尾事故的发生，保护驾乘人员安全。（P49）

难点（P50）

- 故障性风险：车辆零部件的失效。（P50）
- 非故障风险：在特定场景触发下会导致车辆发生危害行为，难以提前预见；系统内部功能组件设计不足（例：2018 年美国自动驾驶测试车辆在夜间与自行车相撞，其决策系统在设计之初未考虑行人会横穿马路）。（P50）
- 性能局限：特斯拉汽车曾发生 3 起与白色货车相撞的事故，感知系统在特定光照条件下将白色车厢识别为天空；幽灵刹车问题——在特殊光照条件、角度及距离情况下，车辆将道路上非障碍物识别为障碍物。（P50）
- 组件间信息交互障碍。（P50）

AEB 定义（P51）

- AEB 系统是通过自动制动来避免或者缓解碰撞的一种主动安全技术，属于先进驾驶辅助系统（advanced driver assistance system，ADAS）的范畴。（P51）
- AEB 系统需要全天候全时段监测车辆前方行驶环境，在碰撞发生前提醒驾驶员，并自动启动车辆制动系统使车辆减速，以避免或减轻碰撞。（P51）
- 主要适用于两车追尾工况，弥补紧急情况下驾驶员制动过慢或者制动力不足的缺陷，主要通过 2 种方式来提高安全：①通过提前辨别危险情况并警告驾驶员来避免事故；②通过主动制动车辆，降低不可避免事故的严重程度。（P51）

AEB 系统功能规范（P52）

- 标准 GB/T 39901-2021《乘用车自动紧急制动系统(AEBS)性能要求及试验方法》规定 AEB 系统应具备以下功能：（P52）
  - 碰撞预警和紧急制动功能：车辆直线行驶遇到静止、低速行驶或紧急减速的障碍物（车辆）时，应以光学、触觉及振动等模式预警，且预警阶段车速不能显著下降，并最终不与障碍物发生碰撞。（P52）
  - 系统失效后的警告信号：当 AEB 系统失效时，系统应启动常亮警告信号，直至 AEB 系统功能恢复。（P52）
  - 驾驶员干扰性能：系统在预警和紧急制动阶段均允许驾驶员通过主动动作中断系统工作。（P52）
  - 相邻车道车辆误响应性能：车辆遇到相邻车道车辆行驶时，系统不应启动。（P52）
  - 车道内铁板误响应性能：车辆行驶时遇到井盖和限高杆等设施时，系统不应启动。（P52）

STPA 控制架构与工作过程（P53-P54）

- 采用系统理论过程分析（system theoretic process analysis，STPA）方法对 AEB 系统进行整车级分析，建立 AEB 系统的 STPA 控制架构。（P53）
- （P54）图示：自动紧急制动系统工作过程（感知前方环境→危险判断→预警→自动制动）。

AEB 系统工况参数（P55）

- 环境参数：照明情况（良好/差）、道路特征（是否路口）、路面状况（干燥/雨天/雪天）。（P55）
- 车辆参数：车速、车辆运动状态（直行/转弯）、视野是否遮挡。（P55）
- 行人参数：行人速度（0-5km/h-10km/h）、行人运动状态（静止/行走/奔跑）、行人行走方向（从左/右侧过马路/沿路直行）。（P55）

车速与制动距离关系（P56）

- 假定路面摩擦系数为 0.85、车型为 C 级车、正常荷载下：车速 20→2.0m、30→4.4m、40→7.9m、50→12.3m、60→17.7m、70→24.1m、80→31.5m、90→39.7m（单位 km/h → 制动距离 m）。（P56）
- 规律：车速越高，制动距离急剧增大（非线性增长）。

AEB 系统的输入/输出（P57）

- AEB 系统输入：环境感知系统提供的障碍物信息、车载传感器提供的自身车辆状态信息以及驾驶员的主动操作信息。（P57）
- AEB 系统输出：光学、声音及振动等预警信息和制动系统的控制命令。（P57）
- 对输入输出的假设：传感器输入到系统的信息真实可信、系统输出的指令可被执行器正确执行。（P57）
- AEB 系统功能启动时对车辆控制的优先级低于驾驶员：当驾驶员通过主动动作控制车辆时，功能关闭，控制权归驾驶员。（P57）

避撞策略（P58）

- 目前国内外汽车厂商及研究机构针对 AEB 避撞策略的制定主要以提高 AEB 系统的综合性能为目标。（P58）
- 两类策略：基于行车间距的安全距离避撞策略；基于驾驶员反应特性的碰撞发生时间避撞策略。（P58）

基于行车间距的安全距离避撞策略（P59-P60）

- 指尽可能避免与前方车辆发生碰撞所要保持的安全距离；从距离尺度将自车与前车的实时车间距作为衡量行车危险状态的指标。（P59）
- 将实时车间距与策略中预警安全距离和紧急制动安全距离进行比较判断，从而指导 AEB 系统做出预警和主动制动的避撞操作。（P59）
- 预警安全距离与紧急制动安全距离主要由如下参数实时确定：自车车速、与目标车的相对车速、自车最大制动减速度、目标车最大制动减速度、系统延迟时间、驾驶员反应时间、车辆停止后需要保持的车间距。（P59）
- Honda 模型（P60）：制动危险距离公式（讲义为公式图）。式中：v 为本车车速，v2 为前车车速，a1、a2 分别为本车和前车的最大减速度，t1、t2 分别为系统延迟时间和制动时间。示例取值 a1=a2=7.8m/s²，t1=0.5s，t2=1.5s。（P60）

基于驾驶员反应特性的碰撞发生时间避撞策略（P61-P64）

- 在符合驾驶员对行车危险情况认知和判断的条件下，从时间尺度将自车当前运动状态到与前车发生碰撞所剩余的碰撞发生时间（time to collision，TTC）作为衡量行车危险状态的指标。（P61）

- 将前端雷达获取到的实际车间距与相对车速相除得到实时 TTC 值，并与系统所设定的碰撞预警 TTC 值和紧急制动 TTC 值比较判断，从而指导 AEB 系统做出预警和主动制动的避撞操作。（P61）
- TTC 模型（P62）：公式中 dbr 为制动距离，D 为两车相对距离，d0 为安全停车距离（取 d0=3m）。算法逻辑：如果 TTC 小于所有延迟时间（系统制动延迟时间与驾驶员反应时间），且驾驶员没有对碰撞预警做出反应，则此时系统应该自动制动。（P62）
- 为什么不用欧氏距离（Euclidean distance，P63）：欧氏距离在数学中被定义为两点之间的距离（欧氏空间中两点间线段的长度），衡量的是空间距离而非碰撞时间，无法直接反映碰撞紧迫性。（P63）
- TTC 计算（P64）：r 是两个物体之间的距离；r' 是这个距离的时间导数，其计算方法是将汽车的相对速度投影到两个物体之间的距离矢量上；运算符 [x]+ 定义为 [x]+ := max(x, 0)。（P64）

AEB 参考文献（P65）

- 何仁、冯海鹏：《自动紧急制动(AEB)技术的研究与进展》，汽车安全与节能学报，第 10 卷第 1 期，2019 年。（P65）
- 胡远志：《基于 PreScan 的 AEB 系统纵向避撞算法及仿真验证》。（P65）
- 标准：GB/T 39901-2021《乘用车自动紧急制动系统（AEBS）性能要求及试验方法》。（P65）

有限状态机 FSM ⭐【重点】

反应式系统（Reactive System）

- **定义**：反应式（reactive）系统是指能够持续地与环境进行交互，并及时地进行响应的系统（P2）。
- **示例**：视频监控系统持续监测，当有陌生人闯入时立刻触发警报；大多数实时系统都是反应式系统，如通信网络、看门狗、家庭应用（洗衣机、微波炉、洗碗机等）（P2）。
- **特征**：事件驱动——系统接收到一个外部事件时，需要对事件进行处理，然后产生响应（P3）。
- 反应式系统可以定义为系统可能的输入/输出事件序列的集合、条件、动作和时序约束（P3）。
- 对反应式系统建模的主要模型：有限状态机（FSM）、行为有限状态机、协同设计有限状态机、UML 状态图（UML StateCharts）、程序状态机、通信交互进程等（P3）。

FSM 定义

- 有限状态机（finite-state machine，FSM）又称有限状态自动机（finite-state automaton，FSA），简称状态机，是表示有限个状态以及这些状态之间的转移和动作等行为的数学计算模型（P4）。
- **状态（State）**：存储关于过去的信息，反映从系统开始到现在时刻的输入变化（P4）。
- **转移（Transition）**：指示状态变更，并用必须满足、确使转移发生的条件来描述（P4）。

- **动作（Action）**：在给定时刻要进行的活动的描述，有多种类型（P4）：
  - 进入动作（entry action）：进入状态时进行；
  - 退出动作（exit action）：退出状态时进行；
  - 输入动作：依赖于当前状态和输入条件进行；
  - 转移动作：在进行特定转移时进行。
- **形式化定义（五元组）**：FSM = ({Input symbols}, {Output Symbols}, {States}, {Initial States}, Transition Relation, Output Function)（P5）：
  - 转移关系（Transition Relation）：将（输入符号，当前状态）映射到下一个状态；
  - 输出函数（Output Function）：将（输入符号，当前状态）映射到当前输出符号。

FSM 的应用

- 有限状态自动机在很多不同领域中都重要，包括电子工程、语言学、计算机科学、哲学、生物学、数学和逻辑学；是自动机理论和计算理论中研究的一类自动机（P6）。
- 在计算机科学中广泛用于：建模应用行为、硬件电路系统设计、软件工程、编译器、网络协议等（P6）。
- **游戏中的 FSM**：角色 AI 可以建模为一系列认知状态，环境事件可以迫使状态发生迁移，认知模型对非程序员也容易掌握；如状态 Flee、Fight、Cornered、Gather Treasure 等，事件如 Monster In Sight、No Monster、Monster Dead（P7）。
- **Pac-Man（吃豆人）**：幽灵有四种行为——在迷宫中随机漫步；当吃豆人在视线范围内时追逐；当吃豆人吃了能量球时逃离；回到中央基地再生（P8-P9）。
- **网络协议**：Internet Protocols 的 TCP 协议可用 FSM 描述（P10）。

FSM 应用实例：汽车安全带警报（P11）

- **场景**：驾驶员用钥匙启动汽车，若 5 秒内没有系好安全带，警报器发出 5 秒警报，直到系好安全带或关闭发动机。
- **形式化规格**：
  - Inputs = {KEY_ON, KEY_OFF, BELT_ON, BELT_OFF, 5_SECONDS_UP, 10_SECONDS_UP}
  - Outputs = {START_TIMER, ALARM_ON, ALARM_OFF}
  - States = {Off, Wait, Alarm}，Initial State = Off
  - NextState: (CurrentState, Inputs) → NextState，例如 NextState(WAIT, KEY_OFF) = OFF
  - Outs: (CurrentState, Inputs) → Outputs，例如 Outs(OFF, KEY_ON) = START_TIMER
- **基于 FSM 的车辆变道决策规划**：用 FSM 描述车辆变道的决策/规划过程（P12）。

FSM 特性

- 统一形式：(next_states, output) = Φ(state, event)（P13）。
- **确定性（Deterministic）**：如果对每个状态、每个输入值最多可激活一个转移，则称状态机具有确定性，意味着 Φ(S, E) 是单一值（P13）。
- **可接受性（Accepting）**：如果对每个状态、每个输入都有至少一个可能的转移，则称状态机是可接受的，定义了每个可能的状态和输入值（P13）。

Moore 机与 Mealy 机

- 状态模型是单线程的：任何时候只有一个状态是有效的（P14）。

对比项	Moore 机	Mealy 机
输出决定因素	完全由当前状态决定，与输入信号的当前值无关	既依赖当前状态，也与输入信号的当前值有关
结构	结构简单，易于组合，易于实现	往往更精简，软件实现代码更简洁
状态数量	大于等于对应的 Mealy 机	相对更少
响应	非反应性，输入对输出的影响要到下一个时钟周期才能反映	即时对输入产生输出，响应速度快
转换	—	可相互转换：Mealy 机可转换成大致等效的 Moore 机，差别只是输出产生于下一个响应，而非当前响应

等价与优化

- 对所有的输入具有相同输出的两个 FSM 称为**等价**（equivalent）（P15）。
- 等价不需要同构（相同形状）：两个等价的 FSM 不需要有相同的图，甚至不需要有相同数量的状态（P15）。
- 等价使优化成为可能：**优化一个 FSM 意味着缩减 FSM 状态数目，同时保证 FSM 实现同样的功能**（P15）。

非确定性 FSM

- 当下一个状态或输出有多个值时，FSM 被称为非确定性的（P16）。
- 五元组中，转移关系表示状态和输入值映射到可能的（下一状态，输出值）对的集合；初始状态不是一个元素而是一个集合，可以有多个初始状态（P16）。
- 在嵌入式系统建模中的主要用途（P16）：
  - 环境建模：隐藏与环境如何运作无关的细节非常有用；

- 机器人不确定的行为：执行的不确定性、状态估计的不确定性。

DFA 与 NFA

- 形式上，确定有限状态自动机（DFA, Deterministic Finite Automaton）和非确定有限状态自动机（NFA）是等价的（通过幂集构造，subset construction）（P17）。
- 在实践中，NFA 通常更简洁（P17）。
- 图例：同一语言可由 DFA（s1→s2→s3，输入 a/b/c 等）与 NFA（如状态集合 {s1,s2,s3} 的转移）分别表示，NFA 的状态图更紧凑（P17）。
- 非确定性 FSM 可用于行为规划（behavior planning）中（P18）。

FSM 实例：定时炸弹（Time-Bomb）

- 定时炸弹控制器用户接口：三个按键 UP、DOWN、ARM，以及定时器 TICK（P21）。
- **UML 状态图描述**（P22）：
  - 状态：setting（设置）、timing（计时）；
  - 事件 Events：UP (Button)、DOWN (Button)、ARM (Button)、TICK；
  - 初始转移 Init/timeout = INIT_TIMEOUT；
  - setting 状态：ARM/code=0 进入 timing；UP [timeout < 60] / ++timeout; display(timeout)；DOWN [timeout > 1] / --timeout; display(timeout)；
  - timing 状态：UP / code <<= 1; code |= 1；DOWN / code <<= 1；ARM [code == defuse] 回到 setting；TICK / --timeout; display(timeout)；[timeout <= 0] / Boom()；[else] 忽略。
- 炸弹示例代码（P24-P27）：枚举 BombSignals（UP_SIG、DOWN_SIG、ARM_SIG、TICK_SIG）与 BombStates（SETTING_STATE、TIMING_STATE）；Event 结构（含 sig）、TickEvt（含 fine_time）；Bomb1 结构（state、timeout、code、defuse）；Bomb1_init 设置 timeout 并进入 SETTING_STATE；Bomb1_dispatch 用两层嵌套 switch 按状态、按事件分发（见下节实现）。
- setup 代码说明：事件来自单一输入队列；事件分配时间需要 2 级嵌套 switch；由于需要在许多子状态中复制超状态的代码，因此没有干净的方法支持状态层次化（P27）。

FSM 实现 📌【了解】

嵌套 switch 的 FSM（Nested-Switch）（P23-P28）

- “State” 是静态类型的枚举，“event” 是类型化信号（也是枚举）；“init” 设置计时器并初始化变量；在每个事件上调用 “dispatch” 将控制传递给 FSM（P23）。
- 结构：外层 switch 按状态（me->state）分发，内层 switch 按事件信号（e->sig）分发，case 内实现动作与状态转移（TRAN(target_) 宏直接赋值 me->state）（P25-P26）。
- **总结**（P28）：

- 优点：简单；内存占用很小（只需一个小的标量状态变量表示当前状态）。
- 缺点：需要枚举信号和状态；无法代码重用（所有元素必须针对手头问题专门编码）；整个状态机被编码为单一函数，很容易变得过大；事件调度时间不是恒定的，取决于两层 switch 的性能，随 case 数量增加而降低（通常 O(log n)）；实现不是分层的；可手动编写 entry/exit 动作，但状态机拓扑变化时容易出错且难维护（与一个状态相关的代码会在导致该状态的每次转换上被分发和重复）。

状态表 FSM（表驱动 State Table）（P29-P37）

- 表驱动设计：将状态和状态转换规则存储在表格中，通过查表方式执行状态转换，减少条件判断和逻辑复杂性，提高执行效率（P29）。
- 对每个事件，描述状态转换规则 (action, next-state)；例如炸弹 FSM 的状态表：行 = 状态（Setting、Timing），列 = 事件（UP、DOWN、ARM、TICK），单元格 = 动作函数 + 下一状态（如 Set_up()→setting、Tim_arm()→Setting* 等）（P29）。
- C 语言实现：用**函数指针的二维数组**实现：`transition state_table[MAX_STATES][MAX_EVENTS] = {{&set_up, ..., &Nothing}, {&tim_up, ..., &tim_tick}}`（P30）。
- 通用状态表事件处理器（StateTable）：结构含 state_table 指针、state、n_states、n_signals、initial 初始转移；接口 StateTable_ctor/init/dispatch/empty；dispatch 通过 `state_table[state * n_signals + sig]` 查表并调用（P31-P33）。
- 应用相关代码：定义 BombSignals/BombStates 枚举、Bomb2 结构（继承 StateTable 并含 timeout、defuse、code），构造时填入 bomb2_state_table 二维表，用户动作函数如 Bomb2_setting_UP/DOWN/ARM、Bomb2_timing_TICK（P34-P36）。
- **状态表实现总结**（P37）：
  - 优点：直接映射成高度规范的状态表；为事件派发提供可确定的性能（O(1)，不包含动作执行时间）；促进通用事件处理器的重用（通常代码短小）。
  - 缺点：需要枚举全部状态和信号（用于检索状态表）；需要大的状态表（通常是稀疏表），但状态表不变，常存于 ROM 而非 RAM；需要复杂的状态表初始化动作，必须绝对符合所枚举的状态和信号，手工维护冗长且易错（如新增一个状态需要增加并初始化一整行单元）。

面向对象的状态设计模式（State Pattern）（P38-P39）

- 严重依赖多态性，需要使用面向对象的语言（如 C++）；每个状态实现为一个类（P39）。
- **总结**（P39）：状态转换有效率（只需重新指定一个指针）；通过后绑定（动态绑定）机制提供非常好的事件派发性能（O(const)，不包括动作执行时间），通常好于检索状态表并调用函数指针的方式；内存高效；如果具体状态对象没有属性（只有操作），它们可以被共享；**不需要枚举各个状态、各个事件**。

常规 FSM 的不足

- 经常**过度指定**：完全指定（即使不关心的事件也要定义）（P42）。
- 由于缺乏组合潜力，**可伸缩性较差**：状态数量可能难以管理（P42）。
- 不支持并发（P42）。
- 经常想要推理组合状态机的局部子属性，但难以关联子状态的行为（P42）。
- **可维护性差**：新增或删除状态时，需要改变所有与之关联的状态，大幅度的修改容易出错（P42）。
- 简单的解决方案：在模型中引入**层次结构**（P42）。

层次有限状态机（Hierarchical FSM，HFSM）

- **定义**：HFSM 最初由 Harel 引入，是由多个嵌套到层次结构中的有限状态机（FSM）组成的；其动机是以模块化的方式方便地对复杂系统建模，从而能够整齐地表示和描述子系统及其相互作用（P44）。
  - 文献： D. Harel. Statecharts: A visual formalism for complex systems. Science of computer programming, 8(3):231–274, 1987.（P44）
- **非确定性 HFSM（马尔可夫模型，Markov Model）**（P46）：
  - 示例：Attack 攻击流程含 Start、Approach、Aim & Shoot、Aim & Slide Left & Shoot、Aim & Slide Right & Shoot 等状态；
  - 增加动作的多样性：对同一个事件有多个转移；给每一个转移标注发生的概率（如 .3/.3/.4）；在运行时随机选择一个转移；
  - 马尔可夫模型：新状态只依赖于前一状态。

并发与状态爆炸（P47-P53）

- 例子：设备可以处于状态 {关闭，启动中，运行中，错误}，同时从 {电源，电池} 运行时，如何安排这些状态？（P47）
- Mealy/Moore 视角下，电池供电与电源供电的操作被视为不同状态，导致**状态爆炸**（state explosion）（P48）。
- 解决方案：允许状态并发操作（拥抱 NFA）（P48）；或使用层次化（组合）模型，将设备状态与电源状态分层/正交组合（P49-P53）。

状态图 StateCharts（P54）

- StateCharts 语言由 David Harel 于 1987 年提出，支持层次结构模型及并发。
- 状态图支持：将状态重复分解为 **AND/OR 子状态**；嵌套状态、并发性、正交；动作（可能有参数）；活动（只要状态处于活动状态就执行的函数）；判定式；历史；同步（即时广播）通信机制。

层次 FSM 模型：OR 状态与 AND 状态（P55）

- 层次结构：状态 a "包含"一个 FSM；处于 a 状态意味着 FSM 处于 a 状态（即 a 的子状态之一）；a 的状态称为 **OR 状态**。
- 并发性：两个或两个以上 FSM 同时处于活动状态；这类状态称为 **AND 状态**。
- 示例图：error/a/recovery 与 odd/even/done 等子状态构成层次，a1/a2 为 a 的子状态（P55）。
- 层次化状态图：FSM 将处于 S 的子状态之一；支持从/到子状态的转移；初始状态（默认）、历史（P56）。

HFSM 术语（定义）（P57）

- FSM 的当前状态也称为**活跃状态**（active state）。
- 包含其他状态的状态被称为**超状态**（superstate）。
- 包含在超状态中的状态被称为超状态的**子状态**（substate）。
- 每个不包含其他状态的状态称为**基本状态**（basic state）。
- 对于每个基本状态 s，包含 s 的超状态被称为 s 的**祖先状态**（ancestor）。
- 任何时候 FSM 只能处于超状态 S 的某个状态中，则该类超状态被称为**或型（OR）超状态**。

默认状态机制（Default-State Mechanism）（P58）

- 指定超状态变为活跃时将要激活的特定子状态；
- 是一个**伪状态（pseudo-state），不是真正的状态**。

历史机制（History Mechanism）（P59-P60）

- 采用该机制，就可能回到超状态退出之前最后一个活跃子状态（P59）。
- 给定输入 m，S 返回到 S 离开之前的状态（可以是 A、B、C、D 或 E）（P59）。
- 在第一次进入 S 时，应用默认机制；**历史和默认机制可以组合使用**（P59-P60）。

并发：AND 超状态（P61-P62）

- 与型超状态（AND-super-state）：无论什么时候系统进入 S 状态，都将进入 S 的**所有**子状态；FSM 处于超状态的所有（直接）子状态（P61）。
- 与或型超状态不同，AND 超状态需要**多个控制点**（P61）。
- 进入 AND 超状态时所进入的子状态可以被单独定义，可以是历史、默认及显式转移的任意组合（P62）。

StateCharts 图中的状态类型（P63）

- 与型状态（AND state）、或型状态（OR state）、基本状态（basic state）。

定时器（Timers）（P64-P65）

- 鉴于嵌入式系统对时间建模的需求，StateCharts 提供了定时器：进入包含定时器的状态一段时间后，超时（timeout）将会发生，且系统将离开这个指定状态（P64）。
- 定时器可以被分层使用；例如：如果事件 a 在系统处于定时器状态的 20 毫秒内没有发生，则会发生超时（P64）。
- 示例：应答机（transponder）层次结构中使用定时器（P65）。

边的标号（P66-P67）

- 生成的输出可用边的标号来指定；标记转移的表达式的一般语法是 `event [condition] / action`（事件[条件]/动作）（P66）：
  - event：触发转移的事件；
  - condition：隐含对变量值的测试或对系统当前状态的测试；
  - action：描述 FSM 对状态转移的反应；
  - 对于每个转移，事件、条件和动作都是可选的。
- 示例（P67）：`on-key/on:=1`（事件测试及变量赋值）；`[on=1]`（对变量值进行条件测试）；`off-key[not in Lproc]/on:=0`（事件测试、对状态的条件测试、变量赋值，当事件发生且条件为真时执行赋值）。
- 条件转移：根据条件选择不同的转移路径（P68）。

StateCharts 的优势与不足（P69-P70）

- **优势**（P69）：层次结构允许任意嵌套 AND 和 OR 超状态；StateMate 的语义定义在足够详细的层次上；大量的商业仿真工具可用（StateMate、StateFlow、BetterState 等）；可在后端将 StateCharts 转换为 C 或 VHDL，从而支持软件或硬件实现。
- **不足**（P70）：生成的 C 程序可能效率低下；生成的硬件可能更糟；难以应用于分布式应用程序；没有结构化层次的描述。

例子：自动饮料售卖机（Vending Machine）（P71-P75）

- 场景（P71）：开机后机器等待投币；投入 25 分硬币后，机器等待另一个 25 分硬币；第二枚硬币存入后进入等待状态；按下"可乐"键就出现可乐；用户拿起瓶子后机器再次进入等待状态；按"雪碧"或"健怡可乐"键则出现相应饮料，拿起瓶子后回到等待状态。
- **V1.0 状态图**（P72）：状态 Idle（空闲）、Gather Cash（收集硬币）、Dispense Product（出货）等；25c/50c 投币转移、cb/cs/cd（可乐/雪碧/健怡可乐按键）出货转移、take bottle 拿起瓶子回到空闲。
- **V1.1 扩展（卡瓶）**（P73-P74）：瓶子会在机器里卡住；卡住时指示器自动通知系统，此时机器不再接受任何钱币或释放任何饮料瓶，直到瓶子被清理干净；清空后再次进入等待状态。

- 问题：状态机需要修改，需要多少个新状态？需要多少新的转移？
- V1.1 在原有基础上新增 Stuck（卡住）状态：stuck bottle 进入、clear bottle 清空后返回。

- **层次 FSM 的优势**（P75）：层次结构允许合理的默认活动；在多个地方可以增强行为；考虑被卡住的恢复行为；所需的状态数量大幅减少；易于添加扩展状态语义；增强状态和条件动作。
  - 注意：层次 FSM 比 FSM 更复杂、成本更高，需要在表现力与复杂性、可维护性之间进行权衡。

行为树（Behavior Tree）⭐【重点】

定义与特征（P78-P79）

- **定义**：行为树是一种结构化的可视化建模技术，常用于系统工程和软件工程领域，用于描绘系统的行为；采用由节点和连接线构成的层级树状图来展示控制流程和系统操作；将各种行为组织成节点，并通过条件来控制节点的执行顺序（P78）。
- **特征**（P79）：
  - 树状结构：从根节点开始，被设计为按特定顺序遍历，直到达到终端状态（成功或失败）为止；
  - 叶节点是可执行行为：每个叶节点都会执行一些操作（简单的检查或复杂的动作），并输出状态（成功、失败或运行中）；叶节点是将行为树连接到特定应用程序的底层代码的地方；
  - 内部节点控制树的遍历：内部（非叶）节点接受其子节点的结果状态，并应用自己的规则来决定下一个应该扩展的节点。

用途（P80）

- 起源于游戏行业，用于为非玩家角色（NPC）定义行为：虚幻引擎（Unreal Engine）和 Unity 都有专门用于编写行为树的工具；
- 机器人的控制与自主决策；
- 自动化系统。

行为树术语（P81）

- **执行节点（Execution node）**：行为树的叶子，可以是 Action 或 Condition 节点；唯一区别在于 Condition 节点在单个 tick 中只能返回 Success 或 Failure，而 Action 节点可以跨越多个 tick 并返回 Running，直到达到最终状态。通常条件节点代表简单检查（如"夹爪是否打开？“），动作节点代表复杂动作（如"打开门”）。
- **控制节点（Control node）**：内部节点，定义如何在其子节点的状态之下遍历行为树；控制节点的子节点可以是执行节点或控制节点本身。顺序（Sequence）、选择（Fallback）和并行（Parallel）节点可以有任意数量的子节点，但处理子节点的方式不同。
- **修饰节点（Decorator）**：必须有一个子节点，使用一些自定义定义的策略修改其行为。

节点类型（P82-P85）

- **顺序节点（Sequence）**：按顺序执行子节点，直到一个子节点返回失败或所有子节点返回成功（即"全部成功才成功"）（P82）。
- **选择节点（Fallback/Selector）**：按顺序执行子节点，直到其中一个返回成功或所有子节点都返回失败（即"一个成功即成功"）（P83）。
- **并行节点（Parallel）**：以"并行"方式执行所有子节点（并非真正并行：每次循环时，每个子节点按顺序单独执行）；如果至少有 M 个子节点（$1 \le M \le N$）成功完成任务，则并行节点返回"成功"；如果所有子节点都失败，则返回"失败"（P84）。
- **装饰器节点（Decorator）**：使用自定义策略修改单个子节点，具有自己的一套规则来改变"被装饰节点"的状态；例如 “Invert” 装饰器将把成功改为失败，反之亦然。虽然装饰器可以增加行为树组合的灵活性，但应尽可能坚持使用标准控制节点和常见装饰器，以便他人容易理解设计（P85）。

例子：寻找物体（P86-P91）

- 场景：移动机器人在家庭环境中寻找特定物体，机器人事先知道所有需要搜索的位置（拥有可操作的世界模型）（P86）。
- 简单开始：只有一个位置 A 时，行为树就是必要行动的简单序列：前往该位置 → 寻找物体（P87）。
- 设计原则：显式成功条件——几乎应该在行动之前先进行检查；例如已到达某位置时，在开始导航操作之前先确认是否已经到达（P88）。
- 多个地点：用选择节点组合多个"前往并寻找"的子树（P89）。
- 并行节点：考虑几个对象（多个搜索目标）时用并行节点组织（P90）。
- **Decorators 和黑板书（blackboard）**：不要为每个位置重复相同的子树，而是使用一个单一的子树，并用"重复行为直到成功"的策略来装饰；黑板用于共享数据（P91）。

BehaviorTree.CPP（P92-P93）

- 轻量级、高性能的 C++ 行为树库，正迅速获得 ROS 开发者的关注（C++ 是流行的机器人开发语言）（P92）。
- 官方的 ROS2 导航栈在其 BT Navigator 功能中使用了这个库（P92）。
- XML 模式：推荐的编写 BT 的方式是通过 XML 文件（P92）。
- 调试与可视化：支持与 Groot（官方可视化工具，新版 Groot2）联动，实时查看树结构、节点状态、执行流程（P92-P93）。

行为树与有限状态机（P94-P98）

- 从理论上讲，任何内容都可以用行为树、FSM、其他抽象模型或纯代码的形式表达；但每种模型都有自身优缺点，体现在它们在大规模设计中的辅助作用上（P94）。

- 就行为树与 FSM 而言，**模块化（modularity）与反应性（reactivity）之间存在权衡**：行为树更容易组合和修改，FSM 在设计反应性行为方面有优势（P94）。
- **机器人示例**（P95）：拾取任务——移动到物体旁、关闭夹持器抓取、返回初始位置；简单设计下行为树与 FSM 两种实现都相对清晰易理解。
- **修改（预抓取修正）**（P96）：若要添加预抓取修正行为（先检查预抓取位置是否有效，无效则修正后再闭合夹具），**行为树非常适合模块化**，修改局部子树即可。
- **反应式问题**（P97）：机器人使用有限电源，电池电量低时必须先返回充电站再继续任务；实现更高响应性会使 BT 设计更复杂，但可借助**反应式序列（Reactive Sequence）** 等结构实现。
- **有限状态机更容易实现反应式**：允许在任意两个状态之间定义转换来实现 reactivity（P98）；HFSM（层次 FSM）可进一步增强（P99）。

行为树的优缺点（P101）

- 优点：
  - 灵活性：可以通过添加、删除、修改节点来轻松调整行为；
  - 模块化：行为可以分离成独立的模块，便于重用和维护。
- 缺点：
  - 复杂性：随着行为树规模增大，树结构可能变得复杂，难以管理；
  - 性能：复杂行为树可能导致性能开销，尤其是在节点层次很多的情况下。

第10章 嵌入式系统设计方法 📌【了解】

> 复习文件标注："10. 嵌入式系统设计方法 -"，整章不在笔试考察范围，通读了解即可；内容仍完整整理。

本章主要内容（P2）：嵌入式系统面临的挑战、嵌入式系统的设计过程、嵌入式系统设计方法学。

10.1 嵌入式系统设计所面临的挑战

- 需要多少硬件？（How much hardware?）（P3）
- 如何满足时限要求，如何处理多项功能在时间上的协调一致关系？（P3）
- 如何降低系统的功耗？（P3）
- 如何设计以保证系统可升级？（P3）
- 如何保证系统可靠地工作？（P3）

对嵌入式系统设计者的要求（P4）：

- 懂得系统的整个构架
- 详细了解硬件的细节
- 软件设计需满足：实时要求、低功耗、代码量小
- 详细了解领域知识

10.2 设计目标

嵌入式系统设计需在多目标间折衷（P5）：

目标	英文	说明
成本	Cost	开发与生产成本
性能	Performance	满足功能与时限要求
功耗	Power	低功耗设计
尺寸	Area	芯片/板卡面积
可伸缩性和可重用性	Scalability and reusability	便于扩展与复用
容错	Fault tolerance	可靠工作

10.3 嵌入式系统的设计过程

- 输入——问题：功能、性能要求、价格、开发周期等（P6）
- 约束：决定设计空间（P6）
- 设计活动：选择、折衷、分析比较、计算、评价（P6）
- 支撑：方法（Methodology）与工具（Tools）（P6）
- 输出：嵌入式系统（P6）

10.4 传统的嵌入式系统设计流程

基本流程（P7）：

- 需求分析 → 规格说明 → 体系结构 → 构件设计 → 系统集成
- 贯穿自顶向下（Top-down）设计与自底向上（Bottom-up）设计两种策略

V 模型设计流（P8）：

- 左侧（自顶向下细化）：需求分析 → 系统体系结构 → 系统设计 → 软件体系结构 → 软件设计 → 单元测试
- 右侧（自底向上集成）：单元测试 → 集成测试 → 系统集成 → 接收与使用

实例：智能驾驶/无人车（P9–P11）：

- 以自动驾驶汽车为例展示需求分析过程，参考 *Autonomous Driving Cars in Smart Cities: Recent Advances, Requirements, and Challenges*
- 图（P9–P11）：无人车场景、自动驾驶系统需求（Advances、Requirements、Challenges）示意图

10.5 体系结构设计与组件设计

体系结构设计——回答"满足规格说明的主要组件是什么？"（P12）：

- 硬件组件：CPU、外设等
- 软件组件：分解为具有接口的软件模块
- 必须同时考虑功能性需求与非功能性需求

硬件和软件组件设计（P13）：

- 在开始编码之前，必须花时间构建系统
- 组件来源分三类：现成的（off-the-shelf）、根据现有设计修改的、必须从头开始设计的

相关示意图（P14–P16）：系统体系结构图（P14）、硬件体系结构图（P15）、软件体系结构图（P16），分别描述系统级、硬件侧、软件侧的组成结构。

10.6 软硬件划分（HW/SW Partitioning）

- 定义：嵌入式系统的设计涉及硬件与软件部件，必须决定什么功能由硬件实现、什么功能由软件实现（P17）
- 硬件和软件具有**双重性**（duality），是划分决策的前提（P17）
- 软硬件变动会对系统决策造成影响；划分和选择需要考虑多种因素（P17）

通常由软件实现的部分（P18）：

- 操作系统功能、任务调度、资源管理、设备驱动
- 协议栈（如 TCP/IP）、应用软件框架
- 原则：除基本系统、物理接口、基本逻辑电路外，许多由硬件实现的功能都可以由软件实现

双重性部分（软硬件均可实现，视性能/成本/功耗权衡而定）（P19）：

- 算法、加密/解密、编码/解码、压缩/解压
- 数学运算：浮点运算、FFT 等

10.7 软硬件技术对系统结构的影响

- 硬软件设计趋势——融合、渗透（P20）
- **硬件设计的软件化**：VHDL、Verilog、HANDEL-C 等硬件描述/高级综合语言（P20）
- **软件实现的硬件化**：将各种算法做成 ASIC（P20）
- 对系统设计的影响——协同设计（Co-design）：增加灵活性，但增加了风险（P20）

嵌入式系统设计方法的演变（P21）：

1. 以 PCB、CAD 和在线仿真器（in-circuit emulator）为主要工具
2. 以 EDA 和 EOS（嵌入式操作系统）为开发平台
3. 以 IP 内核库为设计基础，采用软硬件协同设计技术的系统级设计方法

10.8 传统软硬件设计过程及其问题

基本特征（P22）：

- 系统在一开始就被划分为软件和硬件两大部分
- 软件和硬件独立进行开发设计
- 常采用"硬件优先"方法（“Hardware first” approach）

隐含的问题（P22）：

- 软硬件之间的交互受到很大限制
- 凭经验划分软硬件
- 软硬件之间的相互性能影响很难评估
- 系统集成相对滞后，NRE（一次性工程费用）较大

导致的后果（P22）：

- Poor quality designs（设计质量差）
- Costly modifications（设计修改难）
- Schedule slippages（研制周期不能有效保障）

10.9 软硬件协同设计（HW/SW Co-design）

定义演变（P23）：

- 1994：协同设计指设计一种特殊用途系统，该系统由特定应用的集成电路（IC）与通用处理器上的软件程序协同工作而组成
- 1997：协同设计指通过**并行设计**，利用硬件和软件的协同作用来实现系统级目标
- 2003：协同设计通过提供**分析方法**增加嵌入式系统设计的可预测性（判断系统是否满足性能、功率和尺寸目标），并提供**综合方法**让设计师快速评估许多潜在的设计方案
- 发展：从 90 年代早期的新兴学科，演变为今天的主流技术

简化的协同设计流（P24）：任务并发管理 → 高级转换 → 设计空间探索 → 软硬件划分 → 编译、调度。

完整的协同设计流程（图，P25）：

- 非形式化规格（Informal Specification）→ 系统模型（System Model）→ 系统仿真（System Simulation）
- 软硬件划分（Hardware/Software Partitioning）→ 划分后模型（Partitioned Model）与调度（Schedule）
- 算法设计（Algorithmic Design）与通信综合（Communication Synthesis）
- 分为软件模型（Software Model）与硬件模型（Hardware Model），进行软硬件协同仿真（HW/SW Co-simulation）
- 软件侧：编译（Compilation）→ 二进制执行模型（Binary Executable Model）
- 硬件侧：综合（Synthesis）→ 门级模型（Gate-level Model）
- 最后经仿真或原型验证（Emulate or Prototype）→ 流片制造（Fabrication）
- 整个过程通过细化（Refine）循环迭代，即系统综合（System Synthesis）

10.10 原型与仿真（Prototype & Simulation）

不同层次验证手段的权衡（图/表，P28）——从纯软件模型到真实芯片，精度不断提高，但速度、成本与调试难度也随之变化：

验证手段	速度	精度/时机	特点
纯软件系统仿真（算法/C 模型）	最高速度	忽略硬件细节；过程流中最早	不依赖硬件
指令集仿真器（ISS）	kHz 范围，速度接近原生	精确度稍差；在 RTL 之前	易于调试但细节较少；容易复制
RTL 仿真	MHz 范围	RTL 精确；在获得 RTL 后	出色的硬件调试、易于调试完整细节；很少的软件执行；复制昂贵
FPGA 仿真/原型	十几 MHz	RTL 精确；在获得稳定的 RTL 之后	可调试；比软件复制更昂贵
芯片原型（流片后）	实时速度	完全精确；芯片之后	难于调试；有时难以复制

- 图（P26、P27、P29）：协同设计流程及原型验证相关的示意插图，展示设计流中各阶段（系统仿真、协同仿真、原型验证）的衔接关系

第11章 物联网 ⭐【重点】

1) 基本

物联网的起源

- 物联网并非新词：1999 年，美国从事传感器实证研究的凯文·阿什顿（Kevin Ashton）提出物联网概念，出自自动识别中心研究联盟（P2）。
- 物联网（IoT）由具有唯一身份并连接到互联网上的物组成；重点是传统上与互联网无关的设备或物（例如泵、电表、汽车发动机）通过互联网进行配置、控制和网络化（P2）。
- 物联网是"连接到互联网的端点能力"方面的一场新革命（P2）。
- 物联网的范围并不局限于仅仅将物（设备、家电、机器）连接到互联网：还允许这些物通信和交换数据（控制和信息）；对这些数据的处理将提供面向一般用户或机器目标的各种应用程序（P3）。

术语：M2M（机器对机器）

- M2M（Machine to Machine，机器对机器）：机器设备之间在无需人为干预的情形下，直接通过网络沟通而自行完成任务的一种模式或系统（P4）。
- 2000 年以后 M2M 被广泛应用；其概念与物联网基本相同——都是机器与机器之间通信，根本点都是"机器与机器之间联网、没有人类介入采集信息"（P4）。

M2M 与 IoT 的不同（P5）

对比维度	M2M（机器对机器通信）	IoT（物联网）
定义本质	聚焦"机器与机器之间的直接数据交互"，是一种通信技术/模式	聚焦"万物互联的生态系统"，是一种网络架构/理念
连接范围	封闭、局部的"点到点 / 点到小范围"连接（如两个设备直接通信）	开放、全局的"万物互联"（设备、人、系统跨网络互通）
数据处理方式	数据多在本地/专用系统处理，不依赖云端（如工业设备本地监控）	数据多上传至云端/边缘端，通过算法分析产生决策（如智能家居云端控制）
系统复杂度	简单、单一的"通信链路"（如智能电表→电力公司终端的单向数据传输）	复杂、分层的"生态系统"（感知层→网络层→平台层→应用层全链路）

IoT 定义（P6）

- 定义一（国内教材）：物联网（Internet of Things, IoT）通过射频识别（RFID）、红外感应器、全球定位系统、激光扫描器等信息传感设备，按约定的协议，把任何物品与互联网相连接，进行信

息交换和通信，以实现智能化识别、定位、跟踪、监控和管理的一种网络概念。

- 定义二（Wikipedia）：物联网是计算设备、机械、数字机器相互关系的系统，具备通用唯一识别码（UID），并具有通过网络传输数据的能力，无需人与人、或是人与设备的交互。
- 定义三（IoT 2008）："物联网"这个术语用来描述一系列技术和研究学科，这些技术和研究学科使互联网能够延伸到物理对象的真实世界。
- 定义四（IoT in 2020）：具有身份和虚拟人格的事物在智能空间中运行，使用智能接口在社会、环境和用户环境中连接和交流。

术语（ITU-T，P7）

- 设备：在物联网中，具有强制性通信能力和选择性传感、激励、数据捕获、数据存储和数据处理能力的设备。
- 物：指物理世界（物理装置）或信息世界（虚拟事物）中的对象，可以被标识并整合入通信网。
- 依据：ITU-T Y.2060 建议书（06/2012）。

从人联网到物联网（P8）

- 互联网出现在世界各地，但主要是人与人之间的联系；理念是从人联网到物联网。
- 物联网是一个使用相同媒介连接物的规划。

物联网引入的新层面（P11）

- 物联网允许人们和事物在任何时间、任何地点、通过任何事物和任何人连接，理想情况下可以使用任何路径/网络和任何服务（Perera et al. 2014）。
- 相关图示：全球联网物联网设备数量统计（P9）、LPWAN 市场（P10），均来源 iot-analytics.com。

IOT 特征（P12）

- 智能：从生成的数据中提取知识
- 架构：一个支持许多其他架构的混合架构
- 复杂的系统：一组动态变化的对象
- 规模：可伸缩性
- 时间：数十亿并行和同时发生的事件
- 空间：定位
- 一切都是服务：将资源作为服务消费

IoT 的优势（P13）

- 技术优化：物联网技术有助于技术的改进和提高。例如制造商从汽车传感器收集数据并分析，以改进设计、提高效率。

- 改进的数据采集：传统数据采集有局限性（设计为被动使用），物联网促进了对数据的即时行动。
- 减少浪费：实时信息使资源被有效管理。例如发现多个发动机的同一问题时，可跟踪制造计划并用制造皮带解决。
- 提高客户参与度：通过发现问题和改进流程改善客户体验。

IoT 的不足（P14）

- 安全：连接设备的生态系统可能缺乏足够的认证控制。
- 隐私：在没有用户积极参与的情况下暴露大量个人数据，产生很多隐私问题。
- 灵活性：主要涉及与另一个系统的集成，过程中涉及许多不同的系统。
- 复杂性：物联网系统的设计相当复杂，部署和维护也不容易。
- 合规性：物联网有自己的一套规则和法规，法规遵循的复杂性使任务相当具有挑战性。

IoT 的应用

- 消费者物联网用例（P16）：智能家居类（智能灌溉、智能车库门、智能门锁、智能灯、智能恒温器、智能安全系统）；可穿戴设备类（健康和运动跟踪器、智能服装/可穿戴设备）；宠物类（宠物定位系统、智能狗狗门）。
- 智能家居（P17）：行业发展分为三个阶段——单品连接 → 物物联动 → 平台集成。
- 零售、金融和营销（P18）：任何以消费者为基础的商业交易场所；用例包括定向广告、信标（近距离感知客户）作为营销分析、资产跟踪（库存控制、损失控制、供应链优化）、资产保险跟踪、冷库监控、数字标牌、娱乐场所/会议/音乐会/游乐场/博物馆的信标系统。
- 医疗健康（P19-P20）：用例包括家庭患者护理、预测性和预防性医疗的学习模型、痴呆症患者和老年人护理与跟踪、医院设备和供应资产跟踪、药物跟踪和安全、偏远地区远程医疗、药物研究、患者跌倒指示灯。
- 智慧城市 Smart City（P21-P22）：环境感知的污染控制和监管分析、城市范围小气候天气预报、按需废物管理服务、智能交通灯控制和模式化（改善交通流量和燃油经济性）、按需城市照明、智能扫雪、智能灌溉、智能监控、智能停车场、桥梁街道和基础设施的磨损与使用情况监测。
- 智慧安防（P23）：物联网普及使城市安防从简单的安全防护系统向城市综合化体系演变；通过无线移动、跟踪定位等手段建立全方位立体防护；涵盖街道社区、楼宇建筑、银行邮局、道路监控、机动车辆、警务人员、移动物体、船只等，特别针对机场、码头、水电气厂、桥梁大坝、河道、地铁等重要场所；兼顾整体城市管理系统、环保监测系统、交通管理系统、应急指挥系统等应用的综合体系。
- 智慧交通（P24）：汇集交通信息，对交通管理、交通运输、公众出行等全方位管控支撑，使交通系统具备感知、互联、分析、预测、控制等能力；典型应用包括智能公交车、共享单车、汽车联网、智慧停车、智能红绿灯、充电桩、高速无感收费等。

- 智慧物流（P25-P26）：通过智能软硬件、物联网、大数据等智慧化技术手段，实现物流各环节精细化、动态化、可视化管理；集仓储、运输、配送、信息服务等多功能于一体，将分散的物流资源集中处理，发挥整体和规模优势，共享基础设施、配套服务和信息，降低运营成本、获得规模效益。用例：车队跟踪和位置感知、市政车辆规划/路线/监控、冷藏运输和食品配送安全、轨道车辆识别和跟踪、车队内资产和包裹跟踪、道路车辆预防性维护。
- 智能电网 Smart Grids（P27-P28）：以自动化方式提取消费者和电力供应商的行为信息，提高配电的效率、经济和可靠性，优化资源使用。用例：石油钻井平台数千个传感器和数据点分析、远程太阳能电池板监控和维护、核设施危险分析、智能电表/气表/水表、电价定价分析、根据天气对风力涡轮机实时调整叶片。
- 智能制造（P29-P30）：智能工厂和工业 4.0 项目需要安全可靠的物联网设计；数字化工厂的核心特点是产品的智能化、生产的自动化、信息流和物资流合一。企业数字化、智能化改造大体分 4 个阶段：自动化产线与生产装备 → 设备联网与数据采集 → 数据的打通与直接应用 → 数据智能分析与应用（注：4 个阶段不严格按顺序进行，边界较模糊）。用例：对新旧设备预防性维护、通过实时需求提高产量、节能、安全系统（热感应、压力感应、气体泄漏）、工厂车间专家系统。
- 智慧农业 Smart Farming（P31-P32）：集成计算机与网络技术、物联网技术、音视频技术、3S 技术、无线通信技术及专家智慧与知识，依托部署在农业生产现场的传感节点（环境温湿度、土壤水分、二氧化碳、图像等）和无线通信网络，实现农业环境的智能感知、智能预警、智能决策、智能分析、专家在线指导，提供精准化种植、可视化管理、智能化决策；广义上还包括农业电子商务、食品溯源防伪、农业休闲旅游、农业信息服务。用例：智能灌溉和施肥、智能照明（巢式/家禽养殖）、牲畜监控和资产跟踪、远程农业设备预防性维护、基于无人机的土地调查、农场到市场的供应链效率、机器人耕作、火山和断层线监测（预防灾害）。

2) IoT 技术

物联网体系结构

- 讲义以图示给出物联网体系结构总览（P36）；结合 M2M 与 IoT 对比可知，典型结构为"感知层 → 网络层 → 平台层 → 应用层"的全链路分层生态系统（P5）。
- 物联网技术：云（传感即服务模型，Sensing-as-a-Service）（P39-P40 图示）：云平台作为传感数据的汇聚与服务提供方，将传感能力以"服务"的形式开放给上层应用。

IoT 多样化的技术环境（P37）

- 硬件（端设备）：用于开发设备软件、固件和 API 的集成开发环境（IDE）。
- 通信：RFID、NFC、6LowPAN、ZigBee、蓝牙、WiFi、2G/4G/5G。
- IoT 协议：CoAP、RESTful HTTP、MQTT、XMPP。
- 网络骨干网：IPv4、IPv6、UDP、6LowPAN。
- 软件：RIOT OS、Contiki OS、Eclipse IoT。

- 云平台/数据中心；机器学习算法和软件。

主要协议（P38）

- CoAP：受限应用协议（Constrained Application Protocol）。
- MQTT：消息队列遥测传输（Message Queue Telemetry Transport）。
- XMPP：可扩展通讯和表示协议（Extensible Messaging and Presence Protocol）。
- AMQP：高级消息队列协议（Advanced Message Queuing Protocol）。
- DDS：数据分发服务（Data Distribution Service）。

IoT 软件、硬件、连接（主流的几个）📌【了解】

- 成熟度概览（P41）：物联网软件——边缘人工智能、基于物联网的流分析、监督和非监督机器学习已接近成熟；物联网硬件——CPU、MCU、GPU、安全芯片、FPGA 和边缘网关被归类为相当成熟或主流；物联网连接——eSIM、Mesh 网络、5G 和 Wi-Fi 6 接近成熟。
- IoT 软件技术及采用进度（P43-P45）：
  - 主流（Mainstream）：云计算 Cloud Computing；IoT 平台 IoT Platforms（相当成熟）。
  - 接近成熟（Nearing maturity）：边缘 AI/分析 Edge AI（边缘计算与 AI 结合，算法在设备本地或附近服务器处理）、容器 Containers、基于 IoT 的流分析、监督机器学习 Supervised ML、云原生应用设计、云原生数据仓库、实时数据库、低代码/无代码开发平台。
  - 即将出现（Coming up）：非监督机器学习、Serverless/FaaS、深度学习、IoT 市场（Marketplaces）、数字孪生 Digital Twins、IoT 安全平台、IoT 边缘数据与应用平台、ML Ops。
  - 数年后（Years out）：自动化 ML（Automated ML）、数据生态系统（Data ecosystems）；遥远未来（Far on the horizon）：双向脑机接口 BMI。
- IoT 硬件技术（部分，P46）：CPU、MCU、GPU（主流）；安全芯片、边缘网关（连接云与控制器/传感器/智能设备的物理设备）、FPGA（相当成熟）；智能传感器（感知到相应输入时采取预定义动作，接近成熟）；云连接传感器（即将出现）。
- IoT 连接技术（部分，P47）：蜂窝 IoT 2G/3G/4G、LPWAN（Sigfox、LoRa、NB-IoT、LTE-M）（主流）；eSIM（嵌入式 SIM，支持远程配置、多运营商配置文件远程切换）、Mesh 网络、5G（2019 年商用）、Wi-Fi 6（IEEE 802.11ax）（接近成熟）；MQTT、OPC UA（工业自动化 M2M 通信协议）、卫星 IoT（即将出现）；TSN 时间敏感网络（IEEE 确定性以太网标准）、SASE 安全访问服务边缘（数年后）。

进步最快的技术 📌【了解】（P42）

- 智能传感器：过去三年技术发展出现高潮，旨在解决边缘应用的延迟、数据吞吐量和安全性问题；与旧一代相比嵌入了数据处理功能，可在更靠近传感器的地方处理数据并对用户界面或执行器作出响应；推动应用包括可穿戴医疗设备（如血糖监测器）。

- Wi-Fi 6：相当成熟，显著提高速度和网络容量，吞吐能力几乎是 Wi-Fi 5 的 4 倍；路由器、网关和客户场所设备（CPE）是其快速普及的关键设备。
- GPU：优化 GPU 以训练 AI 深度学习模型，为物联网应用同时处理多个计算；因其并行处理能力被引入数据中心，使其更加成熟。

IOT 平台

- 概念（P48）：物联网平台是一种多层技术，能够在物联网领域中直接提供、管理和自动化连接设备；通过灵活的连接选项、企业级安全机制和广泛的数据处理能力，无论硬件如何多样化都将硬件连接到云；对开发者提供一套随时可用的功能，极大地加快联网设备应用程序的开发，同时兼顾可扩展性和跨设备兼容性。
- 作为中间件（P49-P50）：物联网平台起源于物联网中间件的形式，充当硬件层和应用层之间的中介；主要任务包括通过不同协议和网络拓扑从设备收集数据、远程设备配置和控制、设备管理和空中固件更新；在异构物联网生态系统中，中间件期望支持几乎所有可连接设备并集成、与设备使用的第三方应用程序融合；这种独立于底层硬件和软件的特性，允许一个物联网平台以同样直接的方式管理任何类型的连接设备。
- 层次化功能与技术栈（P51-P52 图示）：平台功能按层次组织，形成完整的技术栈，支撑从设备接入到上层应用的各环节。
- 物联网平台覆盖的功能领域（P53）：

  1. 使设备连接管理简单；
  2. 接收、存储和发送数据；
  3. 帮助企业可视化设备数据并提供关键见解；
  4. 提供特定于应用程序的支持和数据；
  5. 提供知识渊博、反应灵敏的支持团队；
  6. 提供健壮的安全；
  7. 提供支持物联网解决方案特定需求的定制；
  8. 一致的产品交付和平台更新。

- 各功能领域要点：
  - 设备连接管理（P54）：设备连接性是任何物联网产品的重要组成部分，连通性管理是许多平台的主要功能；登录后可检查各设备连接状态并排除问题；随着 eUICC（eSIM）普及，应选择支持空中传输（OTA）供应和软件更新的平台；还可提供工具跟踪数据消耗、在不同网络或服务线路之间切换设备。
  - 数据管理（P55）：平台作为终端设备生成数据的接收点，可存储在平台中，或路由到第三方应用程序进行处理和分析。
  - 可视化与见解（P56）：配备易于使用的数据仪表板，可自定义查看数据、快速了解部署情况；可展示账单信息、覆盖范围和连接问题以及数据使用随时间的变化；蜂窝平台在检测到

异常使用或设备损坏时可给出视觉警报，便于发现可疑设备并暂停连接；地理空间可视化提供位置信息（如一组小型摩托车最后被检测到的位置及移动路径）；目标是快速提供系统状态洞察，在异常成为问题之前处理。

  - 特定于应用的支持和数据（P57）：利用多个信息源的输入，与特定应用上下文连接；例如物联网冰箱故障时，客服登录应用平台查看设备错误或连接问题并根据数据解释。
  - 支持团队（P58）：需要平台提供商持续支持，自服务平台可监督和排除大多数小问题，必要时可提供网络运营团队的高级支持；由于物联网项目启动运行学习曲线陡峭，应选择有能力将业务整合到平台的供应商。
  - 健壮的安全（P59）：最小权限原则同样适用于平台级别，旧模式"授予全面管理访问权、假设只有一人管理系统"在规模化部署中不现实；应选择支持基于角色的访问控制（RBAC）的平台，为团队中每个人分配不同职责和权限（如 IT 成员负责设备管理、财务成员访问数据使用情况）；数据安全包括加密、全面的身份管理和灵活的部署，端到端数据流加密（静态数据、设备认证、用户访问权限管理、敏感数据的私有云基础设施）。
  - 定制（P60）：每个物联网用例需求略有不同；可寻找支持实时数据导出或流向其他系统的平台；API 允许开发特定于应用程序的工具；任务管理工具简化团队协作、减少意外的重复工作。
  - 产品交付与平台更新（P61）：选择定期更新升级的平台提供商（跟踪新出现的安全威胁、接受新功能）；部分平台提供主动管理功能；协作工作流管理工具可定义并编码工作流；机器学习工具提供设备见解、突出使用/安全性和异常事件趋势，工作流引擎将操作项推送给最合适的人员。

- 平台示例：AWS IoT（P63）：兼具广度与深度（跨越边缘直到云端，覆盖几乎任何使用案例和互联设备）；多层安全性（预防性安全机制、连续监控、警报和远程缓解）；卓越 AI 集成（结合 AI 与 IoT，用机器学习在云中构建、训练和优化模型并部署到设备）；大规模下的成熟体验（可扩展支持数千万设备和数十亿条消息，基于可扩展、安全且成熟的云基础设施）。

第11章 物联网（续）⭐【重点】

2) IoT 技术（续）：IoT 平台工作原理

- （P64）本页为 IoT 平台"工作原理"示意图，展示平台如何衔接设备接入与上层应用，实现设备、平台、应用之间的数据流转与协同运行机制。
- （P65–P67）后续为本章第 2 节参考文献及结束页（“Thank you!”），随后进入第 3 节内容。

3) 设备管理（Device Management）

什么是设备管理

- 设备管理服务可帮助对所有连接的设备在全球范围内进行规模化的注册、查看及远程管理（P69）。
- 可在 IoT 平台内查看所有设备队列，进行信息跟踪、运行状态查看、管理连接、查看日志数据等操作（P69）。
- 通过固件标准上报的版本信息，可定位每个设备当下运行的固件信息，便于确认设备稳定并排查问题（P69）。
- 可通过无线下载技术（OTA）发送固件升级，确保设备始终运行最新版本软件（P69）。
- 设备管理在设备接入基础上提供了更丰富完备的管理能力，简化海量设备管理复杂性、节省人工操作、提升管理效率（P69）。

主要功能

- 物联网平台提供功能丰富的设备管理服务，包括但不限于（P70）：生命周期、设备分组、设备影子、物模型、数据解析、数据存储、在线调试、固件升级、远程配置、实时监控等。

设备注册与接入鉴权

- 为保证接入物联网平台的设备安全可信，需进行设备注册和接入鉴权（P71）。
- 设备注册：用户通过控制台或调用注册设备 API 在 IoT 平台注册设备信息；平台中存在设备信息后，再接入真实实体设备，实现平台与终端实体设备的连接和通信（P71）。
- 设备接入鉴权：IoT 平台对接入平台的设备进行鉴权认证，保障设备接入信息的完整性和安全性，以及设备与平台消息传输的完整性和安全性（P71）。

命令下发

- 设备的产品模型中定义了 IoT 平台可向设备下发的命令；平台向设备下发命令，修改设备的服务属性，实现对设备的控制（P72）。
- 平台向设备下发命令包括立即下发和缓存下发两种情况（P72）：

机制	定义	适用场景
立即下发	不管设备是否在线，平台收到命令后立即下发给设备；设备不在线或未收到命令则下发失败	适合对命令实时性有要求的场景，如路灯开关灯、燃气表开关阀
缓存下发	平台收到命令后先缓存，等设备上线或上报数据时再下发；单设备存在多条缓存命令时排队串行下发	适合对命令实时性要求不高的场景，如配置水表参数

设备联动规则

- 规则引擎的使用对象是终端用户，系统预置支持的规则场景，终端用户通过方便、易理解的友好界面在自有设备下制定自动化规则（P73）。
- 规则可以将设备、应用、告警绑定，当绑定的信息满足条件时，规则可自动化执行响应动作（P73）。
- 规则引擎定位处理各种事件，利用规则引擎可完成异常事件的及时通知和快速处理，帮助终端用户维护、监控设备，保证系统业务的及时恢复（P73）。
- 阈值超限、范围超限、位置跟踪等事件也可定义为规则引擎输入条件，并关联对应的处理动作（P73）。
- （P74）本页为设备联动规则示例图，展示设备、应用、告警绑定后按条件自动触发响应动作的流程。

设备批操作

- 设备批操作向物联网平台提供对终端设备统一管理的通道，能提升终端设备的管理效率，满足用户批量管理设备的需求（P75）。
- 当接入设备数量过多，或需要对全部/某一群组的设备进行相同操作时，可采用设备批操作（P75）。
- 支持对设备的批量操作，包括：批量注册设备、批量命令下发、批量位置上传、批量设备配置和批量软固件升级（P75）。

物模型（Thing Model）

- 物模型（Thing Model）是对设备在云端的功能描述，包括设备的属性、数据、服务和事件（P76）。
- 物联网平台通过定义一种物的描述语言来描述物模型，称之为 TSL（Thing Specification Language），采用 JSON 格式，可根据 TSL 组装上报设备的数据（P76）。
- 物模型的功能类型（阿里定义，P77）：

功能类型	说明	是什么/做什么
属性（Property）	一般用于描述设备运行时的状态，如环境监测设备读取的当前环境温度；属性支持 GET 和 SET 请求方式，应用系统可发起对属性的读取和设置请求	是什么
服务（Service）	设备可被外部调用的能力或方法，可设置输入参数和输出参数；相比属性，服务可通过一条指令实现更复杂的业务逻辑，如执行某项特定任务	做什么

功能类型	说明	是什么/做什么
事件（Event）	设备运行时的事件，可被订阅和推送，一般包含需要被外部感知和处理的通知信息，可包含多个输出参数，如任务完成信息、设备故障或告警时的温度	可对外提供什么

- （P78）物模型 JSON 结构示例：包含 schema（物模型结构定义的访问 URL）、profile（productKey 产品标识）、properties（属性：identifier 唯一标识、name 名称、accessMode 读写类型只读 r/读写 rw、required 是否必选、dataType 类型及 min/max 取值范围等）、events（事件：identifier、type 类型 info/alert/error、outputData 输出参数、method 方法名）、services（服务：callType 异步 async/同步 sync 调用、inputData、outputData、method）。

设备影子（Device Shadow）

- 物联网平台提供设备影子功能，用于缓存设备状态（P79）。
- 设备在线时可以直接获取云端指令；设备离线后，再次上线可主动拉取云端指令（P79）。
- 设备影子是一个 JSON 文档，存储设备上报状态和应用期望状态信息（P79）。
- 每个设备有且只有一个设备影子，设备可以通过 MQTT 获取和设置设备影子来同步状态；该同步可以是影子同步给设备，也可以是设备同步给影子（P79）。
- 应用场景（P80）：① 网络不稳定、设备频繁上下线；② 多程序同时请求获取设备状态；③ 设备掉线（指令携带时间戳）。
- （P81）设备影子 JSON 示例：state 中 desired（期望状态，如 color: RED、sequence: RED/GREEN/BLUE）与 reported（上报状态，如 color: GREEN）并列，metadata 记录各属性时间戳，顶层含 timestamp 与 version。

关于模板

- 模板定义了一类设备的 schema，Schema 中定义了各设备各属性的显示名称、类型、默认值等信息（P82）。
- 通过模板可定义设备的视图（view）（P82）。
- 需要支持模板的 CRUD 操作，IoT Device Management Schema API 主要包含管理设备模板的相关接口（P82）。
- 示例：物模型模板（中国移动 IoT）、Device template library in IoT Central（Azure）（P82）。

数字孪生（Digital Twin）

- 数字孪生（Digital Twin，数字双胞胎），又称数字映射、数字镜像（P83）。
- 起源（P83）：
  - 工程实践：2009 年 AFRL（美国空军研究实验室）发起"机身数字孪生"项目；NASA 于 2010 年首次提出数字孪生（Digital Twins）概念。

    - 理论色彩：2002 年 Michael Grieves 在密歇根大学提出 PLM（产品生命周期管理）概念模型。
    - 数字孪生起源于制造业，现已广泛应用到智慧城市、智慧交通、智慧农业、智慧医疗、智能家居等行业，可谓无处不在。
- 官方定义（P85）：数字孪生是充分利用物理模型、传感器更新、运行历史等数据，集成多学科、多物理量、多尺度、多概率的仿真过程，在虚拟空间中完成映射，从而反映相对应实体装备的全生命周期过程。
- 数字孪生的价值（P86）：
    - 可见性：实现机器操作的可见性，以及制造工厂或机场中大型互联系统的可见性。
    - 预测性：使用多种建模技术（基于物理和基于数学的），用于预测机器未来的状态。
    - 假设分析：通过适当设计的接口很容易与模型交互，对模型询问假设问题，模拟现实中无法创建的各种条件。
    - 记录与沟通机制：对单独机器或机器集合的行为进行理解和解释。
    - 连接不同的系统：如后端的业务应用，设计正确时可连接后端业务应用，在供应链运作（制造、采购、仓储、运输、物流、现场服务等）中实现业务成果。
- 数字孪生与仿真（Simulation）的区别（P87）：

对比项	仿真（Simulation）	数字孪生（Digital Twin）
本质	将包含确定性规律和完整机理的模型转化成软硬件方式来模拟物理世界的方法；通过仿真实验、数值计算和问题求解反映系统行为或过程的模型技术	依靠仿真、实测、数据分析等手段对物理实体状态进行感知、诊断和预测，进而优化物理实体，同时进化自身的数字模型
特征	仅能以离线方式模拟物理世界，不具备分析优化功能，因此不具备数字孪生的实时性、闭环性等特征	依托并集成其他新技术，与传感器共同在线以保证其保真性、实时性与闭环性
关系	仿真技术是创建和运行数字孪生的核心技术，是数字孪生实现数据交互与融合的基础	在仿真基础之上集成更多新技术

- 数字孪生与信息物理系统（CPS）的异同（P88）：
    - 相同：数字孪生与 CPS 都是利用数字化手段构建系统为现实服务。
    - 不同：CPS 属于系统实现，而数字孪生侧重于模型的构建等技术实现；相比于综合了计算、网络、物理环境的多维复杂系统 CPS，数字孪生的构建是建设 CPS 系统的使能技术基础，是 CPS 具体的物化体现。
- 数字孪生与元宇宙（P89）：
    - 元宇宙：通过虚拟增强的物理现实，是呈现收敛性和物理持久性特征的、基于未来互联网的、具有连接感知和共享特征的 3D 虚拟空间。

- 共同点：都以数字技术为基础，再造高仿真的数字对象和事件，进行可视化感知交互和运行；底层支撑技术可通用。
- 不同点：元宇宙可以现实物理世界为数字框架，也可以完全塑造全新的理念数字世界，终极形态是基于数字世界实现的原生社会；而数字孪生则以信息世界严格、精确映射物理世界和事件过程为框架和基础，无论是工业制造还是城市管理，都基于实时客观数据的动态进程，与人工智能结合进行挖掘分析和深度学习，并进一步模拟情境和决策，以改进现实或更好适应现实，最终实现自动控制或自主决策控制，终极形态是自主孪生。

- 行业实现（P90–P92）：
  - ① 简单的设备模型（P90）：使用包含两个主要属性集的 JSON 文档——一组观测值/报告值（设备上探测器读取当前值并更新观测属性，如机器当前转速 1000 RPM）和一组期望值（控制程序希望在设备上设置的值，如应用设置引擎转速 1200 RPM）；此外还存储设备名称/序列号、当前位置等相关信息。
  - ② 工业孪生（P91）：通常被工业物联网供应商采用，包括 PLM 工具设计机器的信息和设备模型；一些工业供应商关注物理属性、设计信息和实时数据，并将其展现在资产/设备模型图中；这些模型通常基于机器的物理属性。
  - ③ 预测孪生示例（P92）：行为和预测建模可通过两种方式实现——基于物理方式（利用资产精确设计的知识和制造参数建模，如有限元分析建立高精度模型回答 “What-if” 问题，例如根据给定负载条件评估机器各部分的应力模式）和基于分析/计算模型（用机器学习技术创建预测模型，无需引入原始设计人员，数据分析师仅基于对机器设备的外部观察即可创建预测模型）。

- 智慧城市领域的数字孪生应用（P93）：2019 年中国新型智慧城市规模超过 9000 亿元，预计到 2023 年市场规模将超过 1.3 万亿元；安全综治、智慧园区、智慧交通是智慧城市建设投入重点，三大细分场景规模占智慧城市建设总规模的 71%；城市级平台、机器人等新技术和产品快速落地。
- （P94）本页为典型应用场景介绍图（智慧城市等数字孪生典型应用场景展示）。

4) 边缘计算（Edge Computing）⭐【重点】

基于云的 IoT 解决方案不足

- 大规模边缘的多源异构数据处理要求，无法在集中式计算线性增长的计算能力下得到满足；IoT 感知层是海量数据，数据具有很强的冗余性、相关性、实时性和多源异构性，数据之间存在着频繁的冲突与合作，需要实时处理（P97）。
- 数据在用户和云数据中心之间的长距离传输将导致高网络时延和计算资源浪费（P97）。
- 大多数终端用户处于网络边缘，通常使用资源有限的移动设备（低存储和计算能力、有限的电池容量），因此有必要将一些不需要长距离传输到云数据中心的任务分摊到网络边缘端（P97）。

- 云计算中数据安全性和隐私保护在远程传输和外包机制中将面临很大挑战，使用边缘计算处理数据可以降低隐私泄漏的风险（P97）。

为何需要边缘计算

- 云服务推：与高速发展的数据处理速度相比，网络的带宽却停滞不前（P98）。
- 物联网拉：边缘处的数据量过大；隐私保护要求；物联网中的大多数终端节点都是能量受限的（P98）。
- 从数据消费者到数据生产者的转变：如 YouTube、Facebook、Twitter；可穿戴健康设备（隐私）（P98）。
- （P99）本页为边缘计算发展历程示意图，展示边缘计算从提出到演进的发展脉络。

边缘计算是什么

- 将"边缘"定义为数据源和云数据中心之间的任何计算和网络资源（P100）。
- 边缘计算与雾计算（Fog Computing）是可以互换的（P100）。

边缘计算的定义

- ISO/IEC JTC1/SC38：边缘计算是一种将主要处理和数据存储放在网络的边缘节点的分布式计算形式（P101）。
- 国际标准组织 ETSI 的定义：在移动网络边缘提供 IT 服务环境和计算能力，强调靠近移动用户，以减少网络操作和服务交付的时延，提高用户体验（P101）。

边缘计算 2.0

- 主要包括云边缘、边缘云和边缘网关三类落地形态（P102）。
- 以"边云协同"和"边缘智能"为核心能力发展方向（P102）。
- 软件平台需要考虑导入云理念、云架构、云技术，提供端到端实时、协同式智能、可信赖、可动态重置等能力（P102）。
- 硬件平台需要考虑异构计算能力，如鲲鹏、ARM、X86、GPU、NPU、FPGA 等（P102）。

特征：云与边缘/雾计算对比

- 云（Cloud）与边缘/雾（Edge/Fog）的对比（P103）：

对比项	**Cloud**	**Edge/Fog**
位置	集中	分布在不同的地理位置
容量	非常大的数据中心	许多小规模的雾节点，形成一个大系统
能耗	高	低

对比项	**Cloud**	**Edge/Fog**
延迟	高，因为终端用户和云之间的距离很大	低，因为边缘和最终用户之间的距离很短
资源和服务的邻近性	远离终端用户，在数据中心	靠近终端用户，处于网络边缘
应用	支持不需要短延迟的应用程序，主流云应用	支持大多数类型的应用程序，如 VR、智能家居、智能车辆、智能城市
服务成本	高，因为数据中心被大公司垄断	更低，由于在网络边缘处理数据

边云协同放大各自价值

- 边缘计算的 CROSS 价值（Connectivity 连接、Realtime 实时、Optimization 数据优化、Smart 智能、Security 安全）推动计算模型从集中式的云计算走向更加分布式的边缘计算（P104）。
- 云计算擅长全局性、非实时、长周期的大数据处理与分析，能够在长周期维护、业务决策支撑等领域发挥优势（P104）。
- 边缘计算更适用于局部性、实时、短周期数据的处理与分析，能更好地支撑本地业务的实时智能化决策与执行（P104）。
- 边缘计算与云计算之间不是替代关系，而是互补协同关系（P104）。
- （P105）两张对比图：Figure 1 边缘计算在靠近数据源处处理图像数据，系统延迟低；Figure 2 云计算中图像数据的信号路径长，增加了系统延迟。

边缘计算例子

- 边缘计算是一种网络模型，其中数据处理发生在网络的边缘，靠近数据源（P106）。
- 边缘计算可以消除将图像数据发送到中央服务器或云服务进行处理的需要（P106）。
- 例：用于道路收费的边缘计算使系统能够在靠近摄像头的低功耗单板计算机上进行车牌识别，只传输车牌号码，而不传输车辆或道路的完整图像（P106）。

边缘计算的主要优势

- 减少带宽：在源头处理数据，消除了将图像传输回中央服务器的需要；由于只发送可操作的信息，所需带宽要少得多（P107）。
- 减少延迟：减少从边缘发送的数据量可以加快系统速度，并最大限度地减少捕获图像和到达信息之间的延迟（P107）。
- 改善隐私、提升安全性：车牌、人脸等敏感信息不会传输到云端（P107）。

何时使用边缘计算

- 适用应用及优势（P108）：

应用	优势
智能交通系统	更低的带宽消耗，提高系统安全性，降低隐私风险
工业自动化	更低的延迟和抖动，以实现更高的吞吐量
自动驾驶车辆导航	最大限度地减少系统延迟，使高速车辆能够快速做出决策，同时消除对始终在线的数据连接的依赖

模型压缩技术 📌【了解】

- 学术界和工业界普遍采用模型压缩技术来提高边缘设备上的学习效率，用于减小模型规模，能够提高计算速度并降低边缘设备的计算负担（P109）。
- 模型裁剪：可移除不必要的模型参数或层以简化模型（P109）。
- 参数量化：减少参数所需的位数来降低模型的存储和计算需求（P109）。
- 这些方法有效缓解了边缘设备上模型部署的约束条件（如计算能力、存储空间的限制）（P109）。

智能边缘平台

- 智能边缘平台（Intelligent EdgeFabric）通过纳管边缘节点，提供将云上应用延伸到边缘的能力，联动边缘和云端的数据，满足客户对边缘计算资源的远程管控、数据处理、分析决策、智能化的诉求（P110）。
- 在云端提供统一的边缘节点/应用监控、日志采集等运维能力，为企业提供边、云协同的一体化边缘计算解决方案（P110）。
- （P111）本页为智能边缘平台例子图；（P112）应用场景——工业视觉（边缘侧视觉质检等场景）；（P113）华为车路协同解决方案架构（V2X）；（P114）华为监控场景（智能边缘平台IEF）；（P115）IoT 解决方案示意图。

面向物联网端的边缘计算开源平台 📌【了解】

- 面向物联网端的边缘计算开源平台，致力于解决在开发和部署物联网应用过程中存在的问题，例如设备接入方式多样性问题等（P116）。
- 这些平台部署于网关、路由器和交换机等边缘设备，为物联网边缘计算应用提供支持（P116）。
- 代表性平台：Linux 基金会发布的 EdgeXFoundry 和 Apache 软件基金会的 Apache Edgent（P116）。
- （P117）EdgeXFoundry 的架构图。
- EdgeXFoundry 服务层描述（P118）：
    - 设备服务层：主要提供设备接入功能，由多个设备服务组成；每个设备服务是用户根据设备服务软件开发工具包（SDK）编写生成的一个微服务；EdgeXFoundry 使用设备文件定义南侧设备的相关信息（源数据格式、存储在 EdgeXFoundry 中的数据格式及对该设备的操作命令

等）；设备服务将来自设备的数据进行格式转换并发送至核心服务层；目前提供 MQTT、ModBus 串行通信协议和低功耗蓝牙协议（BLE）等多种接入方式。

- 核心服务层：由核心数据、命令、元数据、注册表和配置 4 个微服务组件组成；核心数据微服务存储和管理来自南侧设备的数据；元数据微服务存储和管理设备的元数据；命令微服务将定义在设备文件中的操作命令转换成通用 API，提供给用户以监测控制设备；注册表和配置微服务存储设备服务的相关信息。
- 支持服务层：提供边缘分析和智能服务；以规则引擎微服务为例，允许用户设定规则，当检测到数据满足规则要求时触发特定操作，如检测到温度低于 25 度时触发对空调的关闭操作。
- 导出服务层：用于将数据传输至云计算中心，由客户端注册和分发等微服务组件组成；前者记录已注册的后端系统相关信息，后者将对应数据从核心服务层导出至指定客户端。
- 系统管理和安全服务：系统管理服务提供安装、升级、启动、停止和监测控制 EdgeXFoundry 微服务的功能；安全服务用以保障来自设备的数据和对设备的操作安全。

设备端学习（On-device Learning）

- 设备端学习指在部署硬件设备上本地自适应或训练机器学习模型，无需持续依赖中心服务器（P119）。
- 通过现场采集的用户数据，实现个性化、隐私保护和自主运行（P119）。
- 设备端学习系统必须在计算、内存、能耗和数据可用性等严苛约束下运行，需采用专门的模型优化、训练高效和数据表达方法（P119）。
- 随着技术成熟，设备端学习正融合联邦协作、终身自适应和安全执行，推动智能边缘计算的前沿（P119）。
- 动因（P120）：
  - 传统机器学习系统依赖集中式训练流水线：模型在云端利用大规模数据和强大算力训练，随后下发到客户端仅用于推理。
  - 这种训练与部署分离的架构虽适用于多数场景，但在本地数据动态、隐私或高度个性化的现代应用中已显局限。
  - 设备端学习反映了对响应性、个性化和隐私保护的现实需求，打破了传统模式，使系统能在本地直接训练或自适应，无需持续云端连接。
- 设备端学习的优势（P121）：
  - 个性化：本地自适应能捕捉用户独特行为、环境和偏好，解决单一全局模型难以覆盖的多样性问题。
  - 时延与可用性：边缘场景下云端连接可能不稳定或受限，本地学习可在离线或低延迟场景下自主提升模型。
  - 隐私：许多应用涉及敏感数据（如生物、输入、定位、健康），本地学习可避免数据外泄，助力合规。

    - 基础设施效率：集中式训练需庞大后端收集和处理用户数据，分布式学习可降低通信和后端压力，提升可扩展性。
- 决策标准（P122）：
    - 设备端学习工程复杂、投入大，未必总是最佳选择。
    - 团队应评估是否有更简单的替代方案能满足需求，避免不必要的复杂性；只有在确有隐私、网络、时延或性能等硬性需求时，才应采用设备端学习。
    - 对于有严格时延需求的应用（摄像头 33ms、语音 500ms、AR/VR 20ms、控制系统 10ms），云端方案因网络延迟（通常 50–200ms）而不可行，此时设备端学习成为唯一选择。
- 真实应用示例（P123）：移动输入预测是最成熟的设备端学习应用——智能手机键盘通过本地自适应持续优化预测和纠错；以 Gboard 为例，采用联邦学习在保证数据本地化的前提下提升全局模型；Gboard 结合下一词预测与智能补全，实时本地自适应用户输入，提升个性化与隐私，无需数据上传即可优化移动输入体验。

板级支持包（BSP）与系统引导（Bootloader）

本章主要内容：嵌入式软件运行流程、板级支持包 BSP、Bootloader 三部分（P2）。

嵌入式软件运行流程（系统的启动过程）📌【了解】

总体流程（P3）

- 基于多任务操作系统的嵌入式软件（单核）主要运行流程：上电复位 → 板级初始化 → 系统引导/升级（远程升级或本地升级）→ 系统初始化 → 应用初始化 → 多任务应用（P3）。
- 图（启动流程）：引导与升级为同一阶段的两个分支，其中升级又分远程升级、本地升级（P3）。

上电复位与板级初始化（P4）

- 上电复位后先完成板级初始化工作；板级初始化程序具有完全的硬件特性，一般采用汇编语言实现（P4）。
- 不同系统的板级初始化内容有特殊性，但以下工作一般必须完成：
    - CPU 中堆栈指针寄存器的初始化；
    - BSS 段（Block Storage Space，未被初始化的数据段）的初始化；
    - CPU 芯片级的初始化，如中断控制器、内存等的初始化（P4）。

系统引导/升级阶段（P5）

- 根据需要分别进入系统软件引导阶段或系统升级阶段（P5）。
- 软件可通过测试通信端口数据或判断特定开关的方式，选择进入不同阶段（P5）。

系统引导的几种情况（**P6**）

- 从 NOR Flash 读取系统软件加载到 RAM 中运行：解决成本及 Flash 速度比 RAM 慢的问题，软件可压缩存储在 Flash 中（P6）。
- 不需加载到 RAM，而是直接在 NOR Flash 上运行，进入系统初始化阶段（P6）。
- 从外存（如 NAND Flash）读取软件加载到 RAM 中运行：这种方式成本更低（P6）。

系统升级阶段（**P7**）

- 远程升级：通过网络进行，一般支持 TFTP、FTP、HTTP 等方式（P7）。
- 本地升级：通过串口进行，可在 Console 口使用超级终端或特定的升级软件（P7）。

系统初始化、应用初始化与多任务运行（**P8**）

- 系统初始化：完成操作系统等系统软件各功能部分必需的初始化工作，如按系统配置初始化数据空间、初始化系统所需的接口和外设（P8）。
- 初始化需按特定顺序进行：先完成内核的初始化，再完成网络、文件系统等的初始化，最后完成中间件等的初始化（P8）。
- 应用初始化：进行应用任务的创建、信号量/消息队列的创建及应用相关的其它初始化工作（P8）。
- 多任务应用运行：初始化完成后系统进入多任务状态，操作系统按已确定的算法进行任务调度，各应用任务分别完成特定功能（P8）。

板级支持包 **BSP**（正常掌握）

BSP 的定义与作用（**P10**）

- BSP 全称"板级支持包"（Board Support Packages），是嵌入式系统中连接硬件平台和操作系统（OS）的关键软件层（P10）。
- 简单说就是为特定硬件电路板定制的"驱动和配置集合"（P10）。
- BSP 的作用就是填补通用 OS 和特定硬件之间的"鸿沟"（P10）。
- 图（BSP 所处位置）：BSP 位于 Custom-Hardware（定制硬件）与 RTOS-kernel（RTOS 内核）之间，向下屏蔽硬件差异，向上支撑 Applications（应用层）（P11）。

BSP 的职责与组成（**P12**）

职责：

- 提供对特定硬件平台的支持，使操作系统能够在该平台上运行（P12）。
- 包含初始化代码（如启动代码和引导加载程序）和硬件抽象层（HAL）（P12）。
- 集成各种驱动程序，确保操作系统能够正确识别和使用硬件功能（P12）。

组成：

- 启动代码：负责系统上电后的硬件初始化，如设置时钟、初始化存储器等（P12）。
- 硬件抽象层（HAL）：提供硬件接口的抽象层（P12）。
- 设备驱动：针对特定硬件的驱动程序（P12）。

示例：用于特定单片机或开发板的 BSP，如 STM32 的 BSP 包，包含启动代码、时钟配置、外设驱动等（P12）。

BSP 中的驱动程序（P13）

- 驱动程序是直接与硬件设备交互的软件组件，为操作系统或应用程序提供控制硬件的接口（P13）。
- 职责：
  - 控制和管理硬件设备（P13）。
  - 处理设备特定的操作，如初始化、读写数据、中断处理等（P13）。
  - 提供标准化的接口，使上层应用无需关心硬件的具体实现细节（P13）。
- 示例：I2C 驱动、SPI 驱动、UART 驱动、GPIO 驱动等（P13）。

硬件抽象层（HAL, Hardware Abstraction Layer）（P14）

- 定义：HAL 是介于底层硬件和上层软件之间的一层抽象层，用于隐藏硬件的具体实现细节，提供统一的接口（P14）。
- 职责：
  - 抽象硬件细节，提供易于使用的 API（P14）。
  - 提高代码的可移植性，使上层应用和中间件无需关心底层硬件的具体实现（P14）。
  - 通常由硬件厂商提供，帮助开发者快速开发和移植应用（P14）。
- 组成：各种外设的抽象层接口函数，例如 GPIO、UART、I2C、SPI 等（P14）。
- 示例：STM32 HAL 库，提供了对 STM32 系列微控制器外设的抽象和访问（P14）。

BSP 与 BIOS/UEFI 的区别（P15）

- BIOS：负责在电脑开启时检测、初始化系统设备（设置栈指针、中断分配、内存初始化等）、装入操作系统并调度操作系统向硬件发出的指令（P15）。
- UEFI：“统一的可扩展固件接口”，旨在代替 BIOS，提高软件互操作性和解决 BIOS 的局限性（P15）。
- BSP 是和操作系统绑定在一起运行的，虽然 BSP 的开始部分和 BIOS 所做的工作类似，但 BSP 还包含和系统有关的基本驱动（P15）。

- BIOS 程序用户不能更改、编译编程，只能对参数进行修改设置；而程序员可以编程修改 BSP，任意添加与系统无关的驱动或程序，甚至把上层开发的内容统统放到 BSP 中（P15）。

不同系统中的 BSP（P16）

- 一个嵌入式操作系统针对不同的 CPU，会有不同的 BSP（P16）。
- 即使同一种 CPU，由于外设的一点差别，BSP 相应的部分也不一样（P16）。

BSP 的特点与功能（P17）

- 硬件相关性：嵌入式实时系统的硬件环境具有应用相关性，作为高层软件与硬件之间的接口，BSP 必须为操作系统提供操作和控制具体硬件的方法（P17）。
- 操作系统相关性：不同的操作系统具有各自的软件层次结构，因此不同的操作系统具有特定的硬件接口形式（P17）。

RTOS 的引导模式（正常掌握）（P19）

- 操作系统引导概念：将操作系统装入内存并开始执行的过程（P19）。
- 按时间效率和空间效率的不同要求，分为两种模式：
  - 不需要 BootLoader 的引导模式：时间效率高、系统快速启动，直接在 NOR Flash 或 ROM 系列非易失性存储介质中运行，但不满足运行速度的要求（P19）。
  - 需要 BootLoader 的引导模式：节省空间、牺牲时间，适用于硬件成本低、运行速度快、启动速度相对慢的场景；数据在 RAM、代码在 Flash，或都放在 Flash 再加载到 RAM（P19）。

Bootloader 📌【了解】

BootLoader 定义（P20）

- BootLoader 是嵌入式系统中的 OS 启动加载程序（引导加载程序）（P20）。
- 包括固化在固件（firmware）中的 boot 代码（可选）和 Boot Loader 两大部分（P20）。
- 是系统加电后运行的第一段软件代码，实际上是裸机程序（P20）。

嵌入式系统中的引导加载程序（P21）

- 嵌入式系统没有 BIOS/UEFI 那样的固件程序；有的嵌入式 CPU 会内嵌一段短小的启动程序（P21）。
- 系统的加载启动任务完全由 Boot Loader 完成（P21）。
- 如 ARM7TDMI：系统在上电或复位时从地址 0x00000000 处开始执行，该地址存放的即是 BootLoader 程序（P21）。
- 存储位置：BootLoader 需要存放在可按地址随机存取的永久性存储器中；也需要支持 NAND Flash 启动的 SOC（P21）。

支持 NAND Flash 启动的 SOC（P22）

- 问题：NAND Flash 需要控制器才能访问数据，而控制器又需要驱动程序（P22）。
- 复制过程（两种方式）：
  - 硬件方式：硬件实现对存储设备控制器的控制，读取指定大小的数据（P22）。
  - 软件方式：SOC 自带片内 ROM，装有驱动程序，负责将数据复制到 SDRAM 或 RAM 后跳转到 BootLoader 运行，主要负责小量数据的复制（P22）。
- 内存映射：可使用跳线选择启动方式，将其内存地址映射到 CPU 启动地址；若配置为 NOR Flash 启动，则将其原本地址映射到 CPU 的启动地址（P22）。
- 启动后需将映射取消；需要有按地址随机存取存储器的支持（P22）。

固态存储设备的典型空间分配（P23）

- 图（空间分配结构）：固态存储设备中依次存放 Bootloader、启动参数、内核、根文件系统（P23）。

与主机的连接（P24）

- 用来控制 Boot Loader 的设备或机制：主机和目标机之间一般通过串口建立连接（P24）。
- Boot Loader 执行时通常通过串口进行 I/O，如输出打印信息到串口、从串口读取用户控制字符等（P24）。

单阶段还是多阶段启动（P25）

- 多阶段的 Boot Loader 提供更为复杂的功能以及更好的可移植性（P25）。
- 从固态存储设备上启动的 Boot Loader 大多是 2 阶段的启动过程（P25）。
- Bootloader 一般分为两部分：汇编部分执行简单的硬件初始化；C 语言部分负责复制数据、设置启动参数、串口通信等功能（P25）。
- Bootloader 的生命周期：
  1. 初始化硬件：如设置 UART（至少设置一个）、检测存储器等（P25）。
  2. 设置启动参数：告诉内核硬件的信息，如用哪个启动界面、波特率（P25）。
  3. 跳转到操作系统的首地址（P25）。
  4. 消亡（P25）。

Boot Loader 的操作模式（P26）

- 启动加载模式（自主 / Autonomous 模式）：从目标机上的某个固态存储设备将操作系统加载到 RAM 中运行，是 Boot Loader 的正常工作模式（P26）。
- 下载模式：通过串口连接或网络连接等通信手段从主机（Host）下载文件，如内核映像和根文件系统映像（P26）。

- 从主机下载的文件通常先被保存到目标机的 RAM 中，再被写到目标机上的 Flash 类固态存储设备中（P26）。
- 通常在第一次安装内核与根文件系统时使用；系统更新也会使用该模式（P26）。
- 通常都会向终端用户提供一个简单的命令行接口（P26）。

文件传输的通信设备与协议（P27）

- 通常目标机上的 Boot Loader 通过串口与主机之间进行文件传输（P27）。
- 传输协议通常是 xmodem/ymodem/zmodem 协议中的一种（P27）。
- 也可通过以太网连接并借助 TFTP 协议来下载文件（主机提供 TFTP 服务）（P27）。
- 串口传输的速度是有限的（P27）。

Boot Loader 的主要任务（P28）

- stage1 通常包括以下步骤：硬件设备初始化；为加载 stage2 准备 RAM 空间；拷贝 stage2 到 RAM 空间中；设置好堆栈；跳转到 stage2 的 C 入口点（P28）。
- stage2 通常包括以下步骤：初始化本阶段要使用的硬件设备；检测系统内存映射（memory map）；将 kernel 映像和根文件系统映像从 Flash 读到 RAM 空间；为内核设置启动参数；调用内核（P28）。
- 图：Boot Loader 的 stage2 可执行映像被拷贝到内存后的内存布局情况（P29）。

BootLoader 的调试（P30）

- 从交叉调试的技术实现途径及应用场合看，分为：
  - 硬件调试：调试方便，但价格昂贵（P30）。
  - 源码软件调试：点灯或串口输出（P30）。
- 串口终端显示乱码或根本没有显示的常见原因：
  - Boot Loader 对串口的初始化设置不正确（P30）。
  - 运行在 host 端的终端仿真程序对串口的设置不正确，包括波特率、奇偶校验、数据位和停止位等方面的设置（P30）。

BootLoader 例子：U-Boot（P31-32）

- U-Boot（Universal Boot Loader）：功能最全、适配最广的开源 Boot Loader，遵循 GPL 条款的开放源码项目（P32）。
- 其源码目录、编译形式与 Linux 内核很相似；源码是相应 Linux 内核源程序的简化，尤其是设备驱动程序（P32）。
- 支持几乎所有主流 CPU 架构（ARM、x86、MIPS、PowerPC、RISC-V 等）（P32）。
- 适配各类存储介质（NAND/NOR Flash、eMMC、SD 卡、SPI Flash、硬盘等）（P32）。

- 丰富的功能：网络启动（TFTP）、USB 下载、固件升级、环境变量管理、设备树（Device Tree）支持（P32）。
- 模块化设计，可灵活裁剪，从极简嵌入式设备到复杂工业板卡都能适配（P32）。

附录A 期末重点速查表

> 依据《90-期末重点》整理的"章 → 知识点 → 重要程度"速查表。⭐=重点关注，📌=不在笔试范围，无标记=正常掌握。

章节	知识点	标注
1 嵌入式系统概述	定义、术语、组成、特征、分类、典型应用、发展趋势	正常掌握
2 嵌入式微处理器	主流处理器与分类、特点、选择准则与步骤、ARM 架构、流水线清空、分支预测、设备分类	📌 整章不在笔试范围
3 存储器架构	复杂性；**存储器系统的层次结构**；**存储器访问时间难以预测**；固件/可固化(ROMable)、Flash、易失性/非易失性存储器及其应用、存储墙与功耗墙	层次结构、访问时间：正常掌握；其余 📌
4 基于总线的计算机系统	总线及分类/常用总线、IO 设备、可编程 I/O、中断、可重入、竞态条件	📌 整章不在笔试范围
5 嵌入式软件系统	嵌入式软件与桌面软件对比、架构模式、实时软件设计模式（轮询/FSM/中断前后台/DMA、RTOS、同步、发布订阅）、低功耗、多核、中断数据模式（线性/双缓冲/环形/消息队列）	正常掌握（"软件主导硬件、软硬件权衡"📌）
6 嵌入式操作系统	**6.1 RTOS/IoT OS**（实时系统与 RTOS 术语/分类、RTOS 与 GPOS 异同、关键要求、内核服务、IoT OS）；**6.2 实时调度**（任务模型、时间、时限约束、调度程序度量）；**6.3 RMS/EDF 及改进**；6.4 调度异常（优先级反转/继承/天花板、Richard 异常）；6.5 实时调度实例（ucOS-II/FreeRTOS）	整章 ⭐；其中 6.3 ⭐；6.4、6.5 及 6.2 的调度策略/调度程序分类 📌
7 同步、通信与存储管理	任务间同步与通信机制（信号量/互斥/消息队列等）；**动态内存管理缺陷**、**内存管理方法**、**ucOS/FreeRTOS 内存管理**	内存管理：正常掌握；同步通信机制 📌

章节	知识点	标注
8 建模	**模型与特征、建模语言、建模实例（反馈控制/开闭环/PID）**；V 图开发周期、何时建模、建模设计分析、常见建模技术、基于模型的设计、模型用途	粗体部分：正常掌握；其余 📌
9 FSM	**反应式系统、FSM 定义/应用/分类、常规 FSM 不足、层次 FSM（定义/术语/示例）、行为树（术语/应用/优缺点）**；FSM 实现	整章 ⭐；行为树 ⭐；FSM 实现 📌
10 嵌入式系统设计方法	设计挑战、设计目标、传统设计流程、软硬件划分、软硬件协同设计、原型	📌 整章不在笔试范围
11 物联网	**1) 基本**（定义/术语/特征/优劣/应用）；**2) IoT 技术**（体系结构、技术环境、IoT 平台概念/中间件/功能领域）；**3) 设备管理**（主要功能、物模型、设备影子、数字孪生）；**4) 边缘计算**（云方案不足、概念、云边比较、优势用途、设备端学习）	整章 ⭐；其中 4) 边缘计算 ⭐；软件/硬件/连接、进步最快技术、模型压缩、边缘开源平台 📌
12 BSP、Bootloader	**BSP 职责与组成、驱动程序、HAL、RTOS 引导模式**；启动过程、bootloader 启动过程	粗体部分：正常掌握；其余 📌
实验部分	MCU 裸机程序开发、ROS 应用开发	📌 不在笔试范围

附录B 复习文件原文摘录

> 以下为《90-期末重点.pdf》知识点部分原文（“-”=不在本次笔试考察范围，“+”=重点关注）。

1. 嵌入式系统概述：定义、术语、组成、特征、分类、典型应用、发展趋势

2. 嵌入式微处理器 -：主流的嵌入式微处理器，分类；特点；选择微处理器的准则、步骤；ARM 架构；流水线清空，分支预测技术；根据其能力和性能，设备分类

3. 存储器架构：复杂性；易失性存储器 -；非易失性存储器 -；在嵌入式系统中的应用 -；固件、可固化（ROMable）-；Flash -；存储器系统的层次结构；存储器访问时间难以预测；存储墙与功耗墙 -

4. 基于总线的计算机系统 -：总线，分类，常用总线 -；IO 设备 -；可编程 I/O；中断 –；可重入；竞态条件

5. 嵌入式软件系统：嵌入式软件与桌面软件的对比；软件主导硬件、软硬件权衡 -；嵌入式软件架构模式；实时嵌入式软件常用的设计模式——单核（轮询、有限状态机、中断处理（前后台）、DMA）；RTOS 模式，同步（资源、活动）；发布和订阅模型；低功耗设计模式；多核；数据获取/存储相关的中断设计模式（线性、双缓冲、环形、消息队列等）

6. 嵌入式操作系统 +

- 6.1 RTOS、IOT OS：实时系统，实时操作系统，术语，分类；RTOS and GPOS 相似/不同；RTOS 关键要求；为何使用 RTOS；RTOS 类别，对于标准操作系统的实时扩展；RTOS 内核系统服务；物联网操作系统，要求，通用架构，非技术属性，分类
- 6.2 实时调度：实时系统所需的调度策略 -；调度程序及分类 -；任务模型；与任务执行相关的时间，时限约束；调度程序的度量
- 6.3 RMS、EDF 及改进 +
- 6.4 调度异常 -：优先级反转；优先级继承；优先级天花板；Richard 异常
- 6.5 实时调度实例：实时任务调度（结合 ucOS-II 和 freertos）-

7. 同步与通信、存储管理（静态、动态）：实时系统任务间的同步与通信常用的机制 -；动态内存管理缺陷；内存管理方法；ucOS、freertos 内存管理

8. 建模：建模、设计、分析 -；模型、特征；常见的建模技术 -；基于模型的设计及例子 -；嵌入式系统模型的用途 -；建模语言；V 图描述一个嵌入式建模开发周期 -；何时为嵌入式系统建立模型 -；建模实例（反馈控制、开环/闭环控制、PID 算法）

9. FSM +：反应式系统，特征、示例；FSM 定义，应用，分类；FSM 实现 -；常规的 FSM 不足；层次 FSM，定义，术语，示例；+ 行为树，术语，应用，优缺点

10. 嵌入式系统设计方法 -：嵌入式系统设计所面临的挑战；设计目标；传统的嵌入式系统设计流程；软硬件划分；软硬件协同设计；原型

11. 物联网 +

- 1）基本：定义，术语；IOT 特征；IOT 的优势和不足；应用
- 2）IoT 技术：物联网体系结构；IoT 多样化的技术环境；物联网软件、硬件、连接（主流的几个）-；进步最快的技术 -；IOT 平台：概念，中间件，主要功能领域
- 3）设备管理：主要功能；物模型、设备影子；数字孪生（概念、价值、相关概念异同、应用）
- 4）边缘计算 +：基于云的 IOT 解决方案不足；概念；云/边比较；优势，用途；模型压缩技术 -；面向物联网端的边缘计算开源平台 -；设备端学习，动因，优势

12. bsp, bootloader：嵌入式系统的启动过程 -；bsp，职责与组成，驱动程序，HAL；RTOS 的引导模式；bootloader 及其启动过程 -

实验部分：MCU 裸机程序开发 -；ROS 应用开发 -

附录C 考试形式与成绩构成

笔试（期末）

- **形式**：闭卷
- **内容**：基本概念、基本原理、设计/应用技术，开放式问题
- **范围**：以课件和平时作业涵盖内容为主，**重点在于授课时强调的内容**
- **题型**：问答题，共 10 个

成绩构成

项目	占比
平时成绩（实验完成情况、出勤）	30%
期末考试成绩	40%
课程项目	30%

附录D 实验部分简介（不在笔试范围）

> 复习文件明确"实验部分：MCU 裸机程序开发 -、ROS 应用开发 -"，即实验内容不进入笔试，但计入平时成绩。各实验如下（供回顾）：

实验	主题	内容概要
Lab0	交叉编译	交叉编译工具链与环境搭建
Lab1	Arduino / MCU	Arduino 与 MCU 裸机程序开发入门
Lab2	跌倒检测应用	PlatformIO 集成开发环境、Wokwi 仿真、跌倒检测 key 实验
Lab3	ROS	ROS 安装及开发环境、ROS 应用开发
Lab4	tf	ROS 坐标变换 tf 实验
Lab5	PID / 沿墙运动	PID 控制实验与机器人沿墙运动
Lab6	TinyML / 端智能	TinyML 实验手册、边缘端智能应用

附录E 参考书目

1. [德] 彼得·马韦德尔（Peter Marwedel）著，张凯龙 译.《嵌入式系统设计：CPS 与物联网应用》（原书第 3 版），机械工业出版社，2020。
2. [美] Edward Ashford Lee, Sanjit Arunkumar Seshia 著，李实英、贺蓉、李仁发 译.《嵌入式系统导论：CPS 方法》（原书第 2 版），机械工业出版社，2018。
3. Robert Oshana、Mark Kraeling 著，单波等 译.《嵌入式系统软件工程：方法、实用技术及应用》，清华大学出版社，2016。
4. [美] 哈桑·戈玛（Hassan Gomaa）著，郭文海、林金龙 译.《实时嵌入式系统软件设计》，机械工业出版社，2018。
5. [美] 王孔敞（K.C.Wang）著，徐坚、李佳蓓、吴文峰 译.《嵌入式与实时操作系统》，机械工业出版社，2020。
6. Wayne Wolf 著，李仁发 译.《嵌入式计算系统设计原理》（第 3 版），机械工业出版社。

本文档由课件文本自动整理生成，页码 (Pxx) 对应原 PDF 课件页；标注依据《90-期末重点.pdf》，请以老师课堂强调内容为准。